Lean 社区文档

Lean 及其数学库

Lean 定理证明器 是一个主要由 Leonardo de Moura 开发的证明助手。

Lean 数学库 mathlib 是一项由社区驱动的工作，旨在构建一个在 Lean 证明助手中形式化的统一数学库。该库还包含对编程有用的定义。本项目非常活跃，拥有众多固定贡献者，每天都有新的进展。

你可以通过阅读库概览来鸟瞰 mathlib 库中的内容，并在我们的博客上了解最近的新增内容。mathlib 的设计与社区组织在 2020 年的文章 The Lean mathematical library 中有所描述，不过自该文章发表以来，库的规模已经增长了一个数量级以上。还有一篇论文（arXiv 版本）从技术和社会两个层面探讨了 mathlib 增长所带来的挑战。

你还可以查看我们的仓库统计，了解库是如何增长的以及谁在为它做贡献。

什么是证明助手?

证明助手是一种软件，它提供了一种语言，用于定义对象、指定这些对象的性质，并证明这些规约成立。系统会一直检查这些证明的正确性，直至其逻辑基础。

这些工具常被用于验证程序的正确性。但它们也可用于抽象数学，这正是 mathlib 社区所关注的内容。在形式化中，所有定义都被精确地指定，所有证明的正确性都几乎得到保证。

使用 Lean 的几种方式

你可以通过两种方式开始与 Lean 交互。在自己的计算机上安装它会给你带来最令人满意的体验。然而，有些 Lean 项目提供了通过云端与其交互的方式，无需在本地安装。

无需本地安装

如果你只想尝试 Lean 而不想安装它，有几个选择。如果你想做一个快速的一次性实验，可以使用在线 Lean 编辑器。如果你想开展一个更大的项目，可以使用 GitHub Codespaces（或者 Gitpod，但自 2025 年 4 月起已弃用）。例如，你可以一边阅读《Mathematics in Lean》这本书，一边在 GitHub Codespaces 上完成其中的练习。注意，这需要你在 GitHub 上创建一个账户。

安装 Lean

点击此处查看安装 Lean 的说明。

在你按照这些说明操作之后，你大概会想要学习 Lean！

学习 Lean 4

学习 Lean 有许多途径，具体取决于你的背景和偏好。它们都既有趣又有收获，但同样也颇有难度，偶尔还会令人沮丧。证明助手目前仍然不易上手，你不能指望通过一个下午的学习就变得熟练。

本页列出的所有资源都关于 Lean 4。其中有些有 Lean 3 版本，但现阶段已没有必要学习 Lean 3。

动手实践

- 无论你的背景如何，如果你想立刻上手，可以来玩 Natural Number Game。这是一个在线交互式 Lean 教程，专注于证明自然数上基本运算的各种性质。Lean Game Server 托管了多种学习游戏，包括集合论、逻辑以及 Robo（一个关于本科数学的故事）。
- 若想更快节奏地入门，你可以获取 Glimpse of Lean 教程。它包含四个基础文件，涵盖使用 Lean 进行证明的一些基本方面，随后是关于初等分析、抽象拓扑和数理逻辑的若干独立专题文件。
- 你可以下载 策略速查表（PDF），作为最常用策略的参考。
- 如果你希望直接从源代码学习，Lean API 文档 不仅包含 `Mathlib`，还涵盖 `Std`、`Batteries`、`Lake` 以及核心编译器。由于 Lean 的许多部分都是以语法扩展的形式定义的，这是现存最接近完整参考手册的资料。
- 如果你想亲自动手并为 mathlib 做贡献，但不知道有什么合适的项目可以入手，那么在 GitHub Issues 上有一长串容易上手的 issue。如果你正在处理某个 issue，请在该 GitHub issue 下回复，说明你正在处理它，以尽量减少重复劳动。

书籍

如果你更喜欢阅读书籍（带练习），有许多免费可得的 Lean 书籍已被证明对初学者很有帮助。它们以 HTML 或 PDF 形式提供，但通常意在 VSCode 中交互式阅读，随读随做 Lean 练习：

- 面向数学的标准参考是 Mathematics in Lean。你可以将其下载为 PDF，但也请参阅 VSCode 使用说明。
- The Mechanics of Proof 同样面向数学。它的节奏比 Mathematics in Lean 更平缓，面向数学经验较少的读者。
- 如果你更偏好关于类型论基础的内容，标准参考是 Theorem Proving in Lean。
- 一本面向计算机科学/编程的书籍是 The Hitchhiker’s Guide to Logical Verification。它也包含关于 Lean 类型论的有用信息，并配有一个带练习的 VSCode 项目。

如果你想要更专注于 Lean 本身而非如何使用 Lean 的内容，那么你可以阅读参考手册（旧版手册）。

（元）编程与策略编写

- 如果你对 Lean 作为一门编程语言感兴趣，那么你应该阅读 Functional programming in Lean。
- 如果你特别想实现自定义策略，那么你可以从对初学者友好的 Tactic Programming Guide 开始。
- 若要更深入地学习策略和元编程，我们推荐 Metaprogramming in Lean 4 （至少在确认你已掌握 Functional programming in Lean 中关于单子的章节之后）。

关于基础的更多内容

如果你对 Lean 的基础感兴趣，可以先阅读 这里 一份非常粗略的概述。如果你想要更多细节，可以阅读 Rijke 的 Introduction to Homotopy Type Theory 的第一部分，或 HoTT book 的第一章，并忽略其中任何提到一价性（univalence）的内容。

如果你对 Lean 内核的具体细节感兴趣，想为 Lean 编写自己的外部类型检查器，或导出证明，你可以在 Type Checking in Lean 4 中阅读更多内容。

另一个可能有用的资源是 Coq 文档中的这个页面。Coq 的基础与 Lean 的基础极为接近。需要牢记的最相关的差异是：* Lean 的 `Prop` 是证明无关的（proof-irrelevant），因此它更接近于上述页面中的 `SProp`。* Lean 中的宇宙不是累积的（cumulative）。然而，任何类型都可以被提升到更高的宇宙。* Lean 原生支持商类型及其相关的约简规则（参见 Theorem proving in Lean 的这一节）。

如果你能读懂上述 Coq 文档，那么你就已经准备好阅读 Mario Carneiro 的这篇论文，它精确地描述了 Lean 的类型论。

请注意，理解类型论基础对于使用 Lean 而言完全不是必需的。

聚会

许多聚会曾帮助欢迎新人加入 Lean 社区。以下这些附有在线讲座的链接及其他可能感兴趣的材料。请注意, 2022 年以前的所有项目都使用 Lean 3, 但它们可能仍包含相关信息。* Lean for the Curious Mathematician 2023 * Formalization of mathematics 2023 * Lean for the Curious Mathematician 2022 * Lean for the Curious Mathematician 2020

更多活动可在活动页面找到。我们还有一个 YouTube 频道, 其中包含来自上述会议的视频播放列表, 以及其他包含 Lean 相关内容的会议。

Lean 术语表

本文档收集了在 Lean 社区中可能遇到的术语的简短解释。

尽管下文中的许多条目都有精确的技术定义，但我们更倾向于解释它们在日常交流中的用法，并附上额外的参考链接以供进一步了解。

下文中的少数内部链接对应的是尚未添加的条目，因此在它们完成之前，这些链接还不会指向任何条目定义。若想请求向本术语表添加新的条目，欢迎提交一个 issue。

本页中的条目可以通过锚点链接来引用(例如 `https://leanprover-community.github.io/glossary.html#wid`
对于某些条目，在条目标题之前还提供了更易于输入的额外锚点——例如`#heavy-rfl` 会指向“heavy `rfl` / heavy `refl`”条目。新术语条目的作者如果条目标题较长、含有反引号或者难以输入，应当考虑添加这些额外的锚点。

attribute（属性）

可以应用于某个 Lean 声明的一个或多个标签或标记，它可能影响该声明本身的行为，或者影响与之交互的其他 Lean 对象的行为。属性既可以在core Lean 中定义，也可以在Mathlib 中定义，或在任何 Lean 代码中定义。

应用一个属性的方式是：在声明命令前加上 `@[name-of-attribute]` 前缀，或者事后使用 `attribute` 命令，如 `attribute [name-of-attribute] name-of-declaration`。

例如，`@[simp]` 属性会将一个声明（通常是 `lemma`、`theorem` 或 `def`）标记为一个simp 引理。

另见

- The Lean Reference Manual 第 5.4 节，其中列出了在core lean 中定义的属性

beta reduction（β 归约）

依值类型论（以及 Lean 对它的实现）中的一种特定的化简操作，它可以作为判定两个项是否定义相等过程的一部分来执行。

更确切地说，它是将诸如 `(λ x, t) a` 这样的表达式化简为 `t[a/x]` 的过程，其中 `t` 中出现的变量 `x` 已被替换为 `a`。

另见

- Theorem Proving in Lean 第 2.3 节

big operators（大型运算符）

这指的是使用 ∑ 和 ∏ 字符表示的求和与求积记号，例如 Finset.sum 和 Finset.prod。

binder（绑定子）

诸如 `(a : α)`、`[a : α]` 或 `{a : α}` 这样的表达式（其中 `a` 为任意标识符，`α` 为类型），它们作为各种 Lean 语法元素——声明、`fun`、量词及其他——的一部分，表示将在该语法元素或声明的主体中被绑定的标识符。

每种绑定子在标识符是被隐式绑定（无需调用方传入）、显式绑定，还是通过类型类推断绑定方面都有不同的含义。

在某些场合，特别是在 `def` 中，允许定义不带外围括号的“简单”绑定子，例如对某个标识符 `a` 使用绑定子 `a`（不带显式类型）。

bundled vs unbundled（捆绑式与非捆绑式）

给定一个具有性质 `P` 的数学对象 `O`，捆绑 `P` 指的是创建一个 Lean 结构，在其字段中除了结构性地定义 `O` 所需的字段外，还包含一个 `P` 的证明作为其字段之一。

相反，非捆绑式结构只包含 `O` 的定义，并单独创建一个可应用于 `O` 的项的 `is_P` 命题。

举一个具体的例子，一个群同态可以看作群之间的映射 `φ : G → H`，连同一个证明 `h : φ(a * b) = φ(a) * φ(b)`。一个捆绑式群同态会同时将 `φ` 和 `h` 作为字段，而一个非捆绑式群同态则只包含 `φ`，并配有一个单独的用于证明 `h` 的 `IsGroupHomomorphism` 声明。

在选择捆绑或非捆绑时，存在性能、风格或实现方面的考量，此外还有介于两者之间的灰色地带，即部分捆绑结构的某些部分而将其余部分保持非捆绑。Mathlib 中的类型类主要是半捆绑式的，通常只将载

体类型本身非捆绑。Mathlib 中的态射则更常采用完全捆绑式，不过两种方法的痕迹都有所存在，并在下面的资源中有所讨论。

另见

- The Lean Mathematical Library（PDF）的第 4.1.1 节（Bundled Type Classes）和第 4.1.2 节（Bundled Morphisms），这是一篇由 Mathlib 社区撰写的论文，描述了Mathlib 的许多架构与设计选择。

cache（缓存）

通常指的是由Mathlib 的持续集成在每次拉取请求被合并或获得新提交时构建的一组共享的、预先构建好的`olean` 文件。

它的目的是减轻每个Mathlib 用户在本地构建（或重新构建）相同 Lean 文件的需要，因为这样做可能花费大量时间（在一台普通计算机上要数小时）。

缓存是在上述持续集成中构建的，通常 Mathlib 用户使用 `lake exe cache get` 来获取其构建好的文件。

`calc` mode（`calc` 模式）

一种模式，它由对表达式进行的一系列连续变换组成，这些变换涉及诸如 `=`、`<` 等传递关系，或其他被标记了 `trans` 属性的关系。它通过 `calc` 关键字进入。

另见

- Theorem Proving in Lean 第 4.3 节
- `calc` 模式社区文档

carrier（载体）

对于一个将类型 `T` 与某种以附加字段表示的额外数学结构捆绑在一起的 Lean 结构（例如 `Group`），载体即底层元素的类型 `T`。

对于一个将集合 `S` 与某种以附加字段表示的额外性质捆绑在一起的 Lean 结构（例如 `Subgroup`），载体即底层集合 `S`。

class（类型类）

更完整地说，是一个 typeclass（或 type class，类型类）。

一个 Lean 结构，其实例可以通过类型类推断来检索。

它不同于面向对象语言中 class 的用法——这个词在函数式编程语言中的用法源自 Haskell 的类型类。

conv mode（conv 模式）

策略模式的一个子模式，它便于在假设或目标内部进行导航，以便重写或化简它们的目标部分。它从策略模式经由 `conv` 关键字进入。

另见

- `conv` 模式社区文档

core Lean（核心 Lean）

与Mathlib 或其他社区编写的 Lean 代码相区别，core Lean（或“核心库”）指的是随 Lean 自身发行版一同提供的那部分 Lean。

从历史上看，Mathlib 项目本身曾是 core Lean 的一部分，后来为了便于加快开发速度，被拆分为一个单独维护的项目。

即使在拆分之后，一些基础的声明仍然是 core Lean 的一部分。

core Lean 当前的各部分内容可以在 Lean 4 仓库中找到。

declaration（声明）

Lean 环境中的单个 Lean 运行时对象。

或者，含糊地说，指可以定义或声明此类对象的若干 Lean 命令中的任意一个。

此类命令的例子包括 `def`、`lemma`、`theorem`、`constant` 或 `example` 命令等。

更多细节可在 Lean 文档中找到。

defeq（定义相等）

定义相等（Definitional equality）。两个项 `a b : α` 是定义相等的，如果内核能够自动证明 `rfl : a = b`。这比命题相等更强，但不如句法相等那么强。

另见 Equality, specifications and implementations，来自 Xena Project

dependent type theory（依值类型论）

一种类型论，在其中还可以拥有依赖于参数的类型，例如“某个日历月份中的天数”这一类型，其具体取值依赖于具体的月份，因为不同的月份天数不同。更多例子可在下面的资源中找到。Lean 对依值类型论的实现基于所谓的构造演算（Calculus of Constructions），使其既可用于复杂的数学推理，也可用于软件验证。

另见

- Theorem Proving in Lean 第 2 节，其中讨论了 Lean 特定版本的依值类型论
- 构造演算，来自 Wikipedia，对构造演算的进一步概述
- Dependent Type Theory，来自 nLab，对依值类型论的一般性论述
- Mike Shulman 的 In Praise of Dependent Types
- Andrej Bauer 对“What makes dependent type theory more suitable than set theory for proof assistants?”的回答

diamond（菱形）

在类型类实例图中，存在某个类型类的多个相互冲突的项。菱形很可能在类型类推断过程中引发问题，因为推断试图构造该类型类的单一项，因而可能无法做到。在不加限定时，“菱形”最常指这种不良情形，即菱形中非定义相等的项可能导致错误，导致无法通过 `refl` 证明的目标，甚至可能导致根本无法证明相等的目标。在Mathlib 中，由于其众多的层级，菱形大量存在。修复或缓解菱形通常涉及重构出问题类型类的字段或实例优先级。跨库边界的菱形——例如类型类图的一部分位于 Mathlib 中，另一部分位于某个依赖 Mathlib 并添加新实例或类型类的库中——可能特别难以在不进行修改的情况下修复或避免。

另见

- Mathlib 关于 `AddMonoid` 和 `Monoid` 的设计注记，一个规避菱形的具体例子
- Forgetful Inheritance，同样来自 Mathlib 文档，讨论了在类型上的“更丰富”与较贫乏结构这一情形下避免菱形的一般模式

dot notation / generalized field notation / generalized projections（点号记法 / 广义字段记法 / 广义投影）

允许诸如 `((foo a b c).bar x.y).baz` 这样记号的语法糖。

Lean 为语法 `foo.bar` 提供了两种解释：它可以表示 `foo` 命名空间中的声明 `bar`，也可以是广义字段记法。我们在此对后者展开说明。

假设 `foo` 的类型为 `C x1 ... xn`，其中 `C` 是某个常量，`x1 ... xn` 是任意的，并假设上下文中存在一个名为 `C.bar` 的声明，它接受一个类型为 `C x1 ... xn` 的参数。那么 `foo.bar` 就是 `C.bar foo` 的语法糖。对于形如 `foo.bar _ ... _` 的、带有（隐式或显式）参数的调用，Lean 足够智能，能够展开为 `C.bar _ ... foo _ ... _`，从而使一切都能通过类型检查。在前面这些例子中，`foo` 也可以是更复杂的表达式，例如 `(foo bar baz).quux` 中的函数应用。

environment linter（环境检查器）

环境检查器是一种通过分析导入声明与本地声明所构成的整个环境，来寻找 Lean 声明中无意错误的检查器。此外还有句法检查器（对单个声明运行）和风格检查器（对字面源文本运行）。要对当前文件运行环境检查器，请使用 `#lint` 命令。偶尔会引入新的检查器以检测更多类别的错误。Mathlib 的持续集成确保任何此类新代码都能通过既定的检查器。

可以通过使用 `nolint` 属性对某一段特定代码禁用环境检查。一些早于 CI 句法检查器的检查器失败被记录在一个自动生成的 nolint 文件中；随着这些错误被修复，该文件的长度应当随时间趋于零。

`equiv`（等价）

与数学上的相等不同，`equiv` 允许定义类型之间的等价或同余。需要注意的一点是，`equiv` 携带数据，而不仅仅是一个证明。

goal（目标）

在 Lean 中交互式地证明定理的语境下，指每个证明正在进行中的目标陈述。

或者更广泛地，从类型论的角度看，指需要为之给出一个项的单个类型。对于命题而言，给出一个项等价地归结为前述的证明这一概念。

golfing（代码高尔夫）

试图或努力使某一段可工作的代码尽可能简短；在 Lean 的语境下，常常是对证明的长度进行高尔夫优化。经过高尔夫优化的证明常常使用项模式，并且总体上将简洁性置于可读性之上。这种混淆常被有意或有利地用来向读者表明某个证明是机械性的或平凡的。

heavy `rfl` / heavy `refl`（重型 `rfl` / 重型 `refl`）

一种由 Lean 求值时运行缓慢的 `rfl`（或 `refl`）用法。当 `rfl` 被要求一次性执行许多步定义性归约时，就会出现重型 `rfl`，其结果是一个很小的证明项，但需要 Lean 进行大量计算才能确保其通过类型检查。

这个 Zulip 讨论给出了一个特别慢的例子。

hierarchy（层级）

数学某个相关领域内一系列约束逐渐增强的类型类的集合。在Mathlib 中, 我们有代数层级（`semiring`、`ring`、`field`……）、序层级（`preorder`、`partial_order`、`linear_order`……）、拓扑层级（`t1_space`、`t2_space`、`normal_space`……）、范畴层级（`preadditive`、`abelian`、`monoidal`……），还有标量层级（`mul_action`、`distrib_mul_action`、`module`……）、范数层级，以及前述层级的交集，如序-代数层级、拓扑-代数层级等等。

HoTT（同伦类型论）

同伦类型论（Homotopy Type Theory），一种类型论，其特点是包含了一条额外的单价公理（univalence axiom），它使得这样一个概念变得精确：被认为等价的某类型的两个不同实现，在数学意义上也是相等的。

在 Lean 2 中，Lean 内核可以被实例化为标准模式和 HoTT 模式。标准库与同伦类型论库是并行开发的。这两种实例化彼此并不兼容, 因为（标准模式中的）单例消去与 HoTT 的单价公理是不相容的。Lean 3 的内核放弃了对 HoTT 模式的支持。因此，从 Lean 3 起，Lean 不再随附 HoTT 库。

举一个说明性的具体例子，自然数类型有许多定义，它们都可以看作是在构造等价的数学对象。core Lean 拥有一个皮亚诺式的实现，而Mathlib 还拥有一个基于二进制表示的实现，并证明了它与前者等价。鉴于 Lean 不具有单价公理，这两个类型是等价的，但它们作为类型并不能被证明相等。而一种基于 HoTT 的语言（或库）则会进一步将这些类型称为相等的。

`Icc`, `Ico`, `Ioc`, `Ioo`, `Ici`, `Ioi`, `Iic`, `Iio`

在Mathlib 中用于指代 8 种数学区间之一的简写记号。共有 `8` 种区间类型，取决于该区间在每一端是闭的（closed）、开的（open）还是延伸至无穷（infinity）。这些名称被设计得很紧凑，方法是将每个区间记为 `I` + 它在左端的端点情况 + 它在右端的端点情况。`Iii`（按命名约定将指代两端皆无穷的区间）并未使用。

另见

- `Mathlib.Order.Interval.Set.UnorderedInterval`, 它在这些区间之上构建了无序区间（其端点可以按任意顺序给定）。

infoview（信息视图）

在交互式编辑 Lean 文件的语境下，指一个显示增量目标状态、诊断信息、错误以及 Lean 小部件输出的窗口或界面。

instance（实例）

两个密切相关概念之一:

- 由 `def`、`lemma` 或其他声明所接受的、用方括号（`[]`）括起来的类型类参数，使其在使用该声明时由类型类推断系统解析。
- 用同名的 `instance` 命令创建的声明，或等价地用 `instance` 属性标记的声明，二者都会将该声明注册到类型类推断系统中，以供上述用途使用。举一个具体的例子，Mathlib 为 `ℝ` 定义了一个 `linear_order` 实例，使得实数可以用 `<` 进行比较。

Lean 3

Lean 的一个已弃用版本，已被 Lean 4 取代。Mathlib 在 2022—2023 年间完全从 Lean 3 移植到了 Lean 4。

kernel（内核）

在 Lean 的实现（以及更广泛地的证明助手）的语境下，内核是验证每个证明正确性的核心组件。

相比于 Lean 的策略实现代码的规模1^，更不用说相比于Mathlib，Lean 的内核实现相对较小，从而使我们能够更有信心地确信证明没有错误，而无需依赖庞大或复杂的高层构造。

另见

- Lean 的内核实现
- The challenge of computer mathematics，作者 Henk Barendregt 与 Freek Wiedijk (2005)
- 证明助手的组成部分，来自 Andrej Bauer 对“What makes dependent type theory more suitable than set theory for proof assistants?”的回答

Lean Together

面向 Lean 用户社区、其Mathlib 库用户以及更广泛的定理证明器社区的年度会议。

往届活动的演讲或报告已分享在社区 YouTube 频道上。

lint（检查）

检查器（linter）是一种用于查找代码中难以发现的错误的小程序。在提交拉取请求时，Mathlib 会在每次CI 运行中接受检查。

我们有多种机制来检查代码的不同方面：风格与格式、单个声明的句法，以及声明在环境中相互组合的方式。对于这几种情形中的每一种，检查器的使用和定义方式都大不相同。

- 风格检查器由 `lake exe lint-style` 运行，它们在 Mathlib 中定义并在 Mathlib 中的某个特定文件中声明，以文件的 `String` 内容作为输入，并通过在 `scripts/nolints-style.txt` 中添加例外来将其静默。
- 句法检查器自动运行，它们在 core Lean 中定义并用 `add_linter` 命令声明，以声明的 `Lean.Syntax` 作为输入，并通过 `set_option` 命令将其静默。
- 环境检查器由 `#lint` 或 `lake exe lintAll` 运行，它们在 Batteries 中定义并用 `@[env_linter]` 属性声明，以文件的 `Environment` 作为输入，并通过 `@[nolint]` 属性将其静默。

Mathlib

一个面向 Lean 4 的大型、由社区维护的数学集合。

mode（模式）

在编写 Lean 代码的语境下，指一组相关的语法元素或关键字，它们使某种特定风格的形式化推理或证明项的构造变得高效。Lean，尤其是限定于Mathlib 范围内的 Lean，拥有少数几种这样的模式——策

略模式、项模式、calc 模式和conv 模式。某种特定的模式可能使针对特定类型目标的推进变得更容易。然而，在构造一个证明项的过程中，一个证明往往会混合使用各种模式。

module（模块）

包含 Lean 源代码的单个文件。

不要将其与数学中的 `module`（模）混淆，即向量空间的推广。

module docstring（模块文档字符串）

模块级别的注释，概述文件中可以找到的内容。我们要求每个文件都有一个，但一些旧文件仍然没有。

MWE（最小可工作示例）

最小可工作示例（Minimal Working Example），一种通过将一段 Lean 代码精简到其本质部分、同时仍可被他人运行，从而更容易就该代码获得帮助的方式。

更多信息可在 MWE 页面上找到。

non-terminal `simp`（非终结性 `simp`）

`simp` 策略的一次调用，它既不是在某个特定子目标上调用的最后一个策略，也不使用 `simp only` 来显式限制它所考虑的simp 引理。应避免使用非终结性 `simp`，因为它们难以维护，原因在于随着 Mathlib 随时间推移添加或修改 `simp` 引理集，它们的行为或运行时会发生变化。

另见 `simp` 文档的“非终结性 `simp`”一节

olean file（olean 文件）

Lean 在构建一个 Lean 模块时所产生的一种缓存的、已编译的二进制文件。`olean` 文件与构建它们的特定 Lean 版本绑定，其名称与对应的模块相匹配（因此一个名为 `Foo.lean` 的文件将在 `.lake` 文件夹中有一个对应的 `Foo.olean` 文件）。构建一个 `olean` 文件可以手动完成（例如通过 `lake build`），但对于像Mathlib 这样的协作项目，它们是通过CI 构建为一个共享的olean 缓存，然后每个 Mathlib 用户只需检索即可。

orange bar of hell（地狱橙条）

在 VSCode（或其他编辑器）中交互式编辑 Lean 通常相当流畅。

然而，地狱橙条指的是偶尔出现的、Lean 文件旁侧边栏中的橙色条不消失或不更新的情形。通常这些条指示文件中 Lean 仍在求值的部分，但如果这些条一直存在，则表明进度没有推进。修复这些条通常可以通过关闭任何不活动的编辑器标签页，然后打开 VSCode 命令面板（`ctrl-shift-p` 或 `cmd-shift-p`）并运行 `Lean: Restart` 来完成。出现这种情况的另一个常见原因是，由于缓存的 olean 文件集不匹配，Lean 不得不（重新）编译所有导入的Mathlib 文件。在这种情况下，通过 `lake exe cache get` 确保正确下载 Mathlib 缓存应当能解决该问题。

propeq（命题相等）

命题相等（Propositional equality）。两个项 `a b : α` 是命题相等的，如果我们能够证明 `a = b`。这比定义相等和句法相等都更弱。

另见 Equality, specifications and implementations，来自 Xena Project

simp lemma（simp 引理）

一个被标记了某个属性的引理，该属性使其可被 `simp` 策略使用。

好的 `simp` 引理会引导 `simp` 策略将复杂的表达式归约为更简单的表达式，常常达到 `simp` 策略本身就能闭合许多目标的程度。

另见

- Mathlib 的 `simp` 文档。
- Theorem Proving in Lean 第 5.7 节。

simp-normal form（simp 范式）

Mathlib 中的一种约定，用于将具有多种等价形式的命题表达为单一的约定形式。

例子和更多细节可在 `simp` 页面上找到。

syntax linter（句法检查器）

句法检查器是一种试图查找 Lean 声明中无意错误的检查器。此外还有环境检查器（对整个环境运行）和风格检查器（对字面源文本运行）。偶尔会引入新的检查器以检测更多类别的错误。Mathlib 的持续集成确保任何此类新代码都能通过既定的检查器。可以通过使用 `set_option` 命令对某一段特定代码禁用句法检查。

style linter（风格检查器）

风格检查器是一种试图确保Mathlib 内代码外观或风格统一、而不影响其工作行为的检查器。此外还有环境检查器（对整个环境运行）和句法检查器（对单个声明运行）。具体而言，Mathlib 包含一些简短的 Lean 程序，例如检查行末是否不以空白结尾，等等。还有一些用 Python 实现的风格检查器，它们正被用 Lean 编写的检查器所取代。Mathlib 的持续集成确保新代码都能通过既定的检查器。允许的风格检查例外被存储在仓库内的一个风格例外文件中。

syntactical equality（句法相等）

两个项 `a b : α` 是句法相等的，如果它们字面上是同一个表达式。这比定义相等和句法相等都更强。

另见 Equality, specifications and implementations，来自 Xena Project

tactic mode（策略模式）

一种 Lean 模式，其特点是依赖于一系列策略，这些策略常常便于产生与基于纸笔的推理颇为相似的证明，尽管往往要使用复杂的策略来自动化证明中繁琐的部分。有多种进入策略模式的方式。可以从项模式使用 `by` 关键字进入它。其他模式也可以穿插在其中，常常是为了协同产生一个易于理解、高效、简短或可读的整体证明。最终，一个策略模式块的结果是一个项，它是通过其中的策略组装而成的。

另见

- Theorem Proving in Lean 第 5 节，其中讨论了策略，以及进出策略模式
- `show_term` 策略，它可以揭示组装而成的项
- Mathlib 策略文档，其中有一份全面的策略列表

term（项）

项是 Lean 类型论中表示某个（数学）对象的基本组成部分。

项这个词也可以用来与类型形成对照：如果我们写 `x : X`，那么 `x` 是项，而 `X` 是它的类型。在像 Lean 这样的依值类型论中，类型本身也由项来表示。

证明项是表示一个数学证明的一种方式。我们也称这样的证明是用项模式（与策略模式相对）写成的。

另见

- Lean Language Reference 第 4 节

term mode（项模式）

一种 Lean 模式，它通过使用函数式子表达式来组装单个项。与策略模式相比，项模式的证明往往长度较短，尽管对人类来说可能更难阅读。有多种进入项模式的方式。一个声明的主体以项模式开始，或者在策略模式中常常使用 `exact` 策略进入它。高效的项模式证明常常有助于代码高尔夫。

诸如 `have`、`suffices` 和 `show` 这样的命令可以用来编写结构化的项模式证明，它们比裸的证明项更易于阅读。

TPIL

“Theorem Proving in Lean”，一本由 Jeremy Avigad、Leonardo de Moura、Soonho Kong 和 Sebastian Ullrich 编写、并有 Lean 社区贡献的免费在线教科书，“旨在教你在 Lean 中开发和验证证明”。该书从对 Lean 所使用的类型论的简单介绍开始，进而阐释诸如策略、归纳类型和类型类等主题。

type theory（类型论）

一种带有两类基本对象——项和类型——的形式系统。它也可以指对这类语言的研究领域，因为某种特定的类型论可能包含的各种附加性质或特征之间存在着细微差别。

Lean 的依值类型论是其数学基础系统的根基，与集合论形成对照。在集合论的基础上，所有对象在形式意义上都是由单一种类的原始对象——集合——构建出来的。人们推理所构造的集合是否是其他集合的成员。这种成员关系的概念可以对任意两个形式集合提出，这可能使一些陈述在形式上有意义，即便它们对人类而言在数学上毫无意义——例如询问数 2 是否是数 37 的成员（数按集合论方式定义）。相比之下，在类型论中，每个项都有一个相关联的类型，陈述或命题本身也是如此。例如，人们有自然数类型（在Mathlib 中为 `ℕ`），或命题类型（`Prop`），或 `2 + 2 = 4` 的证明的类型，或从 `ℕ` 到 `Prop` 的函数

的类型（`ℕ → Prop`），并且可以构造这些类型的项——分别可能是项 `2`、`2 + 2 = 37`、`rfl` 和 `λ n, true`。然而并非每个项都是一个类型，因此与前述集合论的困难不同，如果询问 37 是否具有类型 2 是没有意义的，那么人们就无法构造出这样的陈述。

unicode abbreviation（Unicode 缩写）

在编辑 Lean 文件的语境下，缩写是一种使用描述性快捷方式来输入标准键盘布局上通常没有的符号的方法。

例如，不等号“≠”可以用序列 `\neq` 输入。

完整的缩写列表（及其替换内容）可在 `vscode-lean4` 仓库中找到。

whnf（弱头范式）

如果一个 Lean 表达式满足下面所链接资源中提到的若干归约判据，则称它处于弱头范式（weak head normal form），常缩写为 `whnf`。非正式地说，处于 whnf 的表达式其最外层部分已被求值，尽管内部的子表达式可能尚未被求值。

它也可以指一个将表达式归约为此形式的命令。

另见

- What is weak head normal form? - Stack Overflow
- Weak head normal form - The Haskell wiki

widget（小部件）

一个可扩展的框架，用于通过 Lean 代码定义交互式的、组件化的图形元素，这些元素在交互式定理证明期间渲染于信息视图中。

小部件也可以指由上述框架渲染的单个图形元素。

小部件提供了一种机制，用于显示在交互式定理证明过程中会更新的额外上下文或信息。

另见

- Lean Together 2021: Widgets, interactive output in VSCode，一场由 Edward Ayers 在Lean Together 2021 上所作的报告，展示了小部件所支持的一些功能
- core Lean 中的小部件源代码

你证明它了吗?

Lean 是一个计算机程序，它能够以超越人类的水平验证数学证明。但 Lean 是一个复杂的程序。某人声称自己证明了一条定理，不能仅仅附上一段 Lean 代码；这段代码必须符合若干基本准则。下面我们简要列出这些准则，然后再逐一详细说明。

验证 Lean 证明的准则

- 你的代码是否位于一个格式正确的 Lean 仓库中？单独一个 Lean 文件是不够的。
- 仓库能否编译？换言之，`lake build` 是否无错误地返回?
- 证明是否正在被检查？换言之，证明主定理的那个文件是否被构建过程所编译?
- 证明所用的内容是否不超出 Lean 的标准公理？换言之，`#print axioms my_proof` 是否返回 `[propext, Classical.choice, Quot.sound]` 的一个子集?
- 你的工作是否证明了你所声称证明的内容?

下面我们逐一详细说明这些准则。

你的代码是否位于一个仓库中?

Lean 是一个发展迅速的软件，每月都有新的发布。Lean 的数学库 `mathlib` 通常每天都会合并许多提交。在当前阶段，该软件仍处于“快速推进”阶段，对向后兼容性几乎没有保证。这在实践中意味着，你电脑上某个随机目录下的一个 Lean 文件中的独立代码片段，可能在你的机器上能够编译，但在别人的机器上却不行（甚至在以后的某个时刻在你自己的机器上也不行）——因为所用的 Lean 或 `mathlib` 版本不同。

为解决这个问题，Lean 代码需要成为一个仓库（也称为项目）的一部分。例如，mathlib 就是一个存储在 GitHub 上的 Lean 项目。一个 Lean 项目附带各种系统文件，这些文件精确地确定了你的代码所使用的 Lean 版本（以及诸如 Mathlib 之类的其他依赖库的版本），这意味着其他人可以独立地编译你的代码。

如果你在 VS Code 中使用 Lean，创建新 Lean 项目最简单的方法是点击由 Lean 扩展提供的 ∀ 符号，然后选择“New Project”。

或者，你也可以在命令行中创建新的 Lean 项目。命令

```
lake new my_project math
```

会创建一个名为 `my_project` 的新项目，并依赖于 Lean 的数学库。

仓库能否编译？证明是否正在被检查?

一个名为 `Foobar` 的项目的典型设置是：它在项目的根目录下有一个 `Foobar.lean` 文件，该文件导入项目中所有相关的文件。如果你定理的实际证明位于 `Foobar/MainResult.lean` 中，那么文件 `Foobar.lean` 中应该有一行写着 `import Foobar.MainResult`。在通常情况下，`lake build` 随后就会构建 `Foobar/MainResult.lean`，而该命令需要无错误地编译。

还有其他更高级的方式来设置仓库，但如果你了解这些方式，那么你也会非常清楚构建过程检查你的证明意味着什么。

证明是否只使用了数学的公理?

Lean 是一个灵活的软件。可以用 `axiom` 命令向系统中添加新的公理（包括错误的公理），或用 `sorry` 策略跳过证明。此外还有其他滥用系统的方法。归根结底，在执行 `#print axioms my_proof` 之后，系统应当返回 `[propext, Classical.choice, Quot.sound]`（或这些公理的某个子集）。用户自定义的公理，或 `sorryAx`（表明某个证明被省略了）都说明你的证明是不完整的。

你的工作是否证明了你所声称证明的内容?

这是一个重要的问题，而且比看上去更为复杂。

- 用户有可能混淆一个结果的陈述和它的证明。在 Lean 中很容易定义并命名 `2+2=5` 这一陈述；但这并不构成对 `2+2=5` 的证明！
- 把一个看起来很复杂的陈述定义并命名为 `TheRiemannHypothesis` 是非常容易的，而它尽管名字如此，实际上却并不是黎曼猜想的陈述。因此，对该陈述的证明当然也不是对真正的黎曼猜想的证明。
- Lean 的语法极其灵活。可以覆盖 Lean 对自然数或其上基本运算的标准定义，然后声称你证明了一个看起来像费马大定理的陈述，但它其实根本不是。

Lean 的数学库 `Mathlib` 提供了若干著名数学定理和猜想的陈述，例如费马大定理 和黎曼猜想。这些陈述都已经过 Mathlib 维护者团队的检查，确认它们是相应数学陈述在 Lean 语言中的正确翻译。

如果你声称证明了一条 `Mathlib` 中尚未陈述的定理，那么验证过程中必不可少的一部分就是要有一位 Lean 专家能够确认：你所证明内容的陈述确实对应于你所提出的数学论断，并且你在陈述该命题的同时也确实证明了它。

最小工作示例

摘要

在 Zulip 上发布代码时，请包含所有的 `import`、`open`、`universe` 和 `variable`，这样他人只需复制粘贴你发布的内容，就能看到与你相同的问题。

确保你做到这一点的最佳方式，是把你打算发布的代码片段复制粘贴到一个空的 Lean 文件中，或粘贴到 lean web 编辑器中，并检查它能否编译通过。

示例

不好的示例：

```
#check (univ : Set X)
```

好的示例：

```
import Mathlib

universe u

variable (X : Type u)

open Set

#check (univ : Set X)
```

不好的示例：

```
Goal state:
/-
a b : blah,
h : a.fst < b.fst,
h2 : a.fst < b.snd
⊢ false
-/
```

好的示例：

```
def blah : Type := Nat × Nat

example (a b : blah) (h : a.fst < b.fst) (h2 : a.fst < b.snd) : False := by
  /-
  a b : blah,
  h : a.fst < b.fst,
  h2 : a.fst < b.snd
  ⊢ False
  -/
  done
```

提示：如果你正在使用 mathlib（例如使用 `import Mathlib`），有一个名为 `extract_goal` 的策略，可以帮助你把当前目标格式化为一个独立的示例。你可以从 `extract_goal` 的输出中移除多余的变量和假设，以进一步精简你的示例。

注意，你仍然需要按上文所述包含相应的 `import`、`open`、`universe` 和 `variable`。

形式化定义

最小工作示例是一段代码片段，它可以被复制粘贴到一个空的 Lean 文件中，并仍然保持相同的特性（工作），且不包含不必要的细节（最小）。

这里是 StackOverflow 关于制作 MWE 的指南。

请确保你的代码片段具备：

- 正确的 imports；以及
- 所有相关的定义 / 定理。

你的示例抛出编译器错误或警告是没有问题的。特别地，你的代码包含关键字 `sorry` 也是没有问题的（事实上，用 `sorry` 替换无关的证明是精简示例的一种好方法）。MWE 的要点在于，你的代码在空白文件中应当抛出与你看到的相同的错误，这样人们才能针对你困惑的那个确切错误来帮助你。

我如何知道我的代码是不是 MWE?

你应当通过把代码片段复制粘贴到一个新的 Lean 文件中，或粘贴到 lean web 编辑器中，看看是否得到预期的行为，来测试这一点。这正是那些试图帮助你的人会做的事！

如果我问的是关于自然数游戏（Natural Number Game）之类游戏的问题怎么办?

如果你的示例来自自然数游戏或任何此类基于浏览器的 Lean 演示，那么你可以添加一个指向该网页的链接，而不必去寻找正确的 imports。例如，说“我正处于自然数游戏的这个关卡，我的证明脚本是 某某”，会比说“我正处于自然数游戏的 Addition World 第 2 关，我的证明脚本是 某某”有用得多。

如果你在 Zulip 上发布代码片段，请确保它被三重反引号包围。

```
def myNat : Nat := 5
```

精简代码的技巧

- 在 MWE 中，简单的 `import Mathlib` 是一个完全合适的 import。
- 出于制作 MWE 的目的，你可以把所有 `theorem` 和 `lemma`（你正在处理的那个之外）的证明替换为 `sorry`。Lean 会给你一些额外的警告，但这些警告是无害的。
- 移除所有与你看到的问题无关的声明（`def`、`theorem`、`lemma`、`example` 等）。一般来说，如果你能注释掉某段代码而不抛出错误，那么它就可以被移除。
- 删除一些代码之后，你接着可以删除所有仅在那里被引用的声明。通过重复这一过程几次，你或许能把一个很长的文件缩短到只有几行。
- 最后，你可以在代码中加入像 `-- HERE`、`-- TODO`、`-- ERROR: yada yada` 之类的注释，以将注意力引导到你 MWE 中的某个特定部分。

Lean 的缺陷报告和功能请求

本节面向为 Lean 本身提交缺陷报告或功能请求的资深用户。如果你是在 Zulip 上提问的初学者，上文的建议已经足够——你无需关心本节内容。

在报告 Lean 中的意外行为时（无论是缺陷还是功能请求），让你的示例尽可能精确且可复现会很有帮助。有两种技术尤其有价值：`#guard_msgs` 和 `lean-minimizer` 项目。

使用 `#guard_msgs` 来捕获预期输出

`#guard_msgs` 命令让你能够将预期的编译器输出直接嵌入到代码中。这使你的示例毫不含糊：任何运行它的人都会立即看到他们能否复现该问题。

基本用法

预期输出放在 `#guard_msgs` **之前的一个文档注释**中，并且必须完全匹配：

```
/-- info: Nat.add : Nat → Nat → Nat -/
#guard_msgs in
#check Nat.add
```

如果输出不匹配，`#guard_msgs` 会向你展示一个 diff：

```
-- This fails because #check Nat.mul produces different output
/-- info: Nat.add : Nat → Nat → Nat -/
#guard_msgs in
#check Nat.mul
```

`#guard_msgs` 提供了一个代码操作（code action），当文档注释与输出不匹配时，它会添加或更新该文档注释。（特别地，在设置时无需手动复制粘贴文本！）

记录错误

```
/--
error: Application type mismatch: The argument
  true
has type
  Bool
but is expected to have type
  Nat
```

```
in the application
  Nat.succ true
---
info: sorry.succ : Nat
-/
#guard_msgs in
#check Nat.succ true
```

使用 drop 来忽略消息类别

使用 `drop info` 或 `drop warning` 来忽略整类消息：

```
#guard_msgs (drop info) in
#check Nat.add
```

使用 substring 进行部分匹配

当你只关心输出的一部分时，使用 `#guard_msgs (substring := true)`。文档注释只需包含出现在实际输出中某处的文本即可：

```
/-- Nat.add -/
#guard_msgs (substring := true) in
#check Nat.add
```

记录超时

将 `set_option maxHeartbeats` 与 `#guard_msgs (substring := true)` 结合使用：

```
set_option maxHeartbeats 1 in
/-- maximum number of heartbeats (1) has been reached -/
#guard_msgs (substring := true) in
example : True := by trivial
```

用 pp.mvars 稳定元变量名

错误消息中常常包含像 `?m.47` 这样的元变量名，它们在不同的运行之间会发生变化。使用 `set_option pp.mvars.anonymous false` 将它们替换为稳定的 `?_` 占位符：

```
set_option pp.mvars false in
/--
error: Type mismatch
  rfl
has type
  ?_ = ?_
but is expected to have type
  1 + 1 = 3
-/
#guard_msgs in
example : 1 + 1 = 3 := rfl
```

不依赖 Mathlib 的精简

对于提交给 Lean 仓库的缺陷报告，不依赖 Mathlib 的示例要可操作得多。一个不依赖 Mathlib 的示例：

- 使得二分查找哪个 Lean 提交引入了回归（regression）变得更容易
- 排除了缺陷在 Mathlib 而非 Lean 中的可能性
- 在开发者迭代修复时编译速度更快

当然，制作一个不依赖 Mathlib 的示例可能很繁琐。**lean-minimizer** 可以帮助自动化这一过程。它的工作方式是反复尝试移除或简化你代码中的各个部分——包括内联和移除 imports——同时保留你试图演示的那个错误。这个过程可能需要一段时间，但其结果往往是一个极小的、自包含的示例，使缺陷一目了然。

对于依赖 Mathlib 的代码，**mathlib-minimizer** 是一个便利项目，它把 lean-minimizer 与作为依赖的 Mathlib 捆绑在一起，因此你可以运行该精简器而无需自己设置依赖项。

我该如何引用 mathlib?

如果你在工作中使用了 mathlib，请使用以下文献进行引用：

> The mathlib Community. The Lean Mathematical Library. In Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP ' 20), January 20–21, 2020, New Orleans, LA, USA. ACM, 2020. DOI: 10.1145/3372885.3373824

许可证

mathlib 库与本网站均以开源许可证发布：mathlib 采用 Apache 2.0 许可证，本网站采用 MIT 许可证。

BibTeX

```bibtex
@inproceedings{mathlib2020,
  author = {The mathlib {C}ommunity},
  title = {The {L}ean {M}athematical {L}ibrary},
  booktitle = {Proceedings of the 9th {ACM} {SIGPLAN} {I}nternational
               {C}onference on {C}ertified {P}rograms and {P}roofs},
  series = { {CPP} '20},
  year = {2020},
  month = {{J}anuary},
  location = {{N}ew {O}rleans, {LA}, {USA}},
  publisher = { {ACM} },
  doi = {10.1145/3372885.3373824},
  url = {https://doi.org/10.1145/3372885.3373824}
}
```

arXiv 预印本

arXiv 预印本可在 arxiv.org/abs/1910.09336 获取。

GitHub 引用

mathlib4 仓库 中还包含一个 `CITATION.md` 文件，GitHub 能够识别该文件。你也可以点击仓库页面上的“Cite this repository”链接，在那里查看上述 BibTeX 引用。

LaTeX 示例

本测试页面汇集了若干在 Markdown 中使用 LaTeX 的示例。此页面仅用于开发用途。

对照：https://math.meta.stackexchange.com/revisions/9386/164

来自 doc-gen issue 的示例

来自 doc-gen#10，如果我在 TeX 中写入 $f[*a, *b](cd)$ 会怎样？ $g_{x_0}(y)$？

来自 doc-gen#62：

- 可能表示 $R[X_i : i \in \sigma]$。
- 示例：$x \in \sigma$，另外单独写 `y`
- 示例：$x \in \sigma$，另外单独写 `s`
- 示例：$x \in \sigma$，另外单独写 `sigm`
- 示例：$x \in \sigma$，另外单独写 `simg`
- 示例：单独写 `s`，以及 $x \in \sigma$

来自 dynamics.circle.rotation_number.translation_number 的示例：

对任意 `x : ℝ`，序列 $\frac{f^n(x)-x}{n}$ 趋向于 `f` 的平移数。特别地，这一极限不依赖于 `x`。

mathlib#3776

PR 之前（有一处多余的 `cc`），行距过大：

$$J = \begin{bmatrix} cc0_l1_l \\ \\ 1_l0_l \end{bmatrix}$$

PR 之后，行距过大：

$$J = \begin{bmatrix} 0_l & 1_l \\ \\ 1_l & 0_l \end{bmatrix}$$

实际已修复：

$$J = \begin{bmatrix} 0_l & 1_l \\ 1_l & 0_l \end{bmatrix}$$

mathlib#6175

PR 之前（渲染正确）：

如果一个函数在圆盘 `D(x, R)` 内解析，那么它在该圆盘所包含的任意圆盘内也解析。事实上，可以写出

$$f(x+y+z) = \sum_n p_n (y+z)^n = \sum_{n,k} \binom{n}{k} p_n y^{n-k} z^k = \sum_k \Big(\sum_n \binom{n}{k} p_n y^{n-k} \Big) z^k.$$

于是相应的幂级数的第 `k` 个系数等于 $\sum_n \binom{n}{k} p_n y^{n-k}$。在 `pₙ` 为多重线性映射的一般情形中，这需要被恰当地解释：不再使用二项式系数，而应对 `fin n` 中所有可能的、基数为 `k` 的子集 `s` 求和，并将 `z` 赋给 `s` 中的指标，将 `y` 赋给 `s` 之外的指标。本段落中我们实现这一点。新的幂级数记作 `p.change_origin y`。然后我们验证它的收敛性，以及它的和与原来的和一致这一事实。这一讨论的结论是：一个函数解析的点集是开集。

PR 之后（在 `sum` 前加了两个反斜杠作为变通；应当渲染失败）：

如果一个函数在圆盘 `D(x, R)` 内解析，那么它在该圆盘所包含的任意圆盘内也解析。事实上，可以写出

$$f(x+y+z) = sum_n p_n (y+z)^n = sum_{n,k} \binom{n}{k} p_n y^{n-k} z^k = sum_k \Big(sum_n \binom{n}{k} p_n y^{n-k} \Big) z^k.$$

于是相应的幂级数的第 `k` 个系数等于
$sum_n \binom{n}{k} p_n y^{n-k}$。在 `pₙ` 为多重线性映射的一般情形中，这需要被恰当地解释：不再使用二项式系数，而应对 `fin n` 中所有可能的、基数为 `k` 的子集 `s` 求和，并将 `z` 赋给 `s` 中的指标，将 `y` 赋给 `s` 之外的指标。

行距测试

（每个示例上方给出 math.stackexchange.com 的结果）

渲染正常 ☒：$\alpha\alpha$

不应渲染 ☒：$\alpha

\alpha$

不应渲染 ☒：$$\alpha

\alpha$$

不应渲染 ☒：$$

\alpha \alpha

$$

渲染正常 ☒：

$$\alpha\alpha$$

渲染正常 ☒：

$$\alpha\alpha[hi](345)b$$

不应渲染 ☒：$$ \alpha\alpha hi \beta

$$

渲染正常 ☒：\begin{aligned} \begin{matrix} a & b & c \end{matrix} \end{aligned}

只有内层的 matrix 环境应被渲染：\begin{aligned}

hi \begin{matrix} a & b & c \end{matrix} \end{aligned}

不应渲染 ☒：\begin{aligned}

\begin{matrix}

a & b & c \end{matrix} \end{aligned}

渲染正常 ☒:

hi there \begin{aligned} 3 \ 3 \end{aligned}

不应渲染 ☒: \begin{aligned} \end{blah}

不应渲染 ☒: \begin{aligned} link { {\alpha} \end{aligned}

嵌套环境

来自 http://web.archive.org/web/20120617014306/http://www.st.fmph.uniba.sk/~kiselak1/pdfka/tex/latexMath_align.pdf

\begin{aligned}T(n) & \leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \T(n) & \leq 2(cn/2) \lg(n/2) + n \T(n) & = cn (\lg n - 1) + n \T(n) & \leq cn \lg n\end{aligned}

\begin{aligned} T(n) & \leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \ T(n) & \leq 2(cn/2) \lg(n/2) + n \ T(n) & = cn (\lg n - 1) + n \ T(n) & \leq cn \lg n \end{aligned}

\begin{gather} \begin{aligned}T(n) & \leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \T(n) & \leq 2(cn/2) \lg(n/2) + n \T(n) & = cn (\lg n - 1) + n \T(n) & \leq cn \lg n\end{aligned}\end{gather}

\begin{gather} \begin{aligned} T(n) & \leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \ T(n) & \leq 2(cn/2) \lg(n/2) + n \ T(n) & = cn (\lg n - 1) + n \ T(n) & \leq cn \lg n \end{aligned} \end{gather}

\begin{aligned}\begin{aligned}T(n) & \leq 2(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \T(n) & \leq 2(cn/2) \lg(n/2) + n \T(n) & = cn (\lg n - 1) + n \T(n) & \leq cn \lg n\end{aligned}\end{aligned}

MathJax 基础教程与快速参考

来自 https://math.meta.stackexchange.com/questions/5020/mathjax-basic-tutorial-and-quick-reference

（德语版：MathJax: LaTeX Basic Tutorial und Referenz）

1. 要查看任意提问或回答中（包括本文）某个公式是如何写成的，在该表达式上右键单击并选择“Show Math As > TeX Commands”。（这样做时，`$` 不会显示出来。请确保你自己添加上它们。参见下一条。还有其他方式可以查看公式或整篇帖子的代码。）

2. **对于行内公式，将公式包在 `$...$` 中。对于独立显示的公式，使用 `$$...$$`。**二者的渲染方式不同。例如，键入 `$\sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6}$` 会显示 $\sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6}$（这是行内模式），或键入 `$$\sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6}$$` 会显示

$$\sum_{i=0}^n i^2 = \frac{(n^2+n)(2n+1)}{6}$$

（这是显示模式）。

3. 对于**希腊字母**，使用 `\alpha`、`\beta`、⋯、`\omega`：$\alpha, \beta, ...\omega$。对于大写形式，使用 `\Gamma`、`\Delta`、⋯、`\Omega`：$\Gamma, \Delta, ..., \Omega$。某些希腊字母有变体形式：`\epsilon \varepsilon` ϵ、ε，`\phi \varphi` ϕ、φ，等等。

4. 对于**上标和下标**，使用 `^` 和 `_`。例如，`x_i^2`：x_i^2，`\log_2 x`：$\log_2 x$。

5. **分组**。上标、下标以及其他运算只作用于紧接其后的“分组”。所谓“分组”，要么是单个符号，要么是用花括号 `{⋯}` 括起来的任意公式。如果你写 `10^10`，你会得到一个意外的结果：10^10。但 `10^{10}` 会得到你大概想要的结果：10^{10}。用花括号来界定上标或下标所作用的公式：`x^5^6` 是错误的；`{x^y}^z` 是 x^{yz}，而 `x^{y^z}` 是 x^{y^z}。注意 `x_i^2` x_i^2 与 `x_{i^2}` x_{i^2} 之间的区别。

6. **括号** 普通符号 `()[]` 生成圆括号和方括号 $(2+3)[4+4]$。使用 `\{` 和 `\}` 来生成花括号 $\{\}$。

   这些括号**不会**随中间的公式自动缩放，因此如果你写 `(\frac{\sqrt x}{y^3})`，括号会显得太小：$(\frac{\sqrt{x}}{y^3})$。使用 `\left(⋯\right)` 会使括号大小自动适配其所包含的公式：`\left(\frac{\sqrt x}{y^3}\right)` 是 $\left(\frac{\sqrt{x}}{y^3}\right)$。

   `\left` 和 `\right` 适用于以下各类括号：`(` 和 `)` (x)、`[` 和 `]` $[x]$、`\{` 和 `\}` $\{x\}$、`|` $|x|$、`\vert` $\vert x\vert$、`\Vert` $\Vert x\Vert$、`\langle` 和 `\rangle` $\langle x\rangle$、`\lceil` 和 `\rceil` $\lceil x\rceil$，以及 `\lfloor` 和 `\rfloor` $\lfloor x\rfloor$。`\middle` 可用于添加额外的分隔符。还有不可见的括号，用 `.` 表示：`\left.\frac12\right\rbrace` 是 $\left.\frac12\right\rbrace$。

   若需要手动调整大小：`\Biggl(\biggl(\Bigl(\bigl((x)\bigr)\Bigr)\biggr)\Biggr)` 给出 $\Biggl(\biggl(\Bigl(\bigl((x)\bigr)\Bigr)\biggr)\Biggr)$。

7. **求和与积分** `\sum` 和 `\int`；下标是下限，上标是上限，例如 `\sum_1^n` $\sum_1^n$。如果上下限不止一个符号，别忘了用 `{⋯}`。例如，`\sum_{i=0}^\infty i^2` 是 $\sum_{i=0}^\infty i^2$。类似地，`\prod` $\prod$、`\int` $\int$、`\bigcup` $\bigcup$、`\bigcap` $\bigcap$、`\iint` $\iint$、`\iiint` $\iiint$、`\idotsint` $\int\cdots\int$。

8. **分数** 有三种生成方式。`\frac ab` 作用于紧接其后的两个分组，生成 $\frac{a}{b}$；对于较复杂的分子和分母，使用 `{⋯}`：`\frac{a+1}{b+1}` 是 $\frac{a+1}{b+1}$。如果分子和分母都很复杂，你可能更倾向于使用 `\over`，它会拆分它所在的分组：`{a+1\over b+1}` 是 $\frac{a+1}{b+1}$。使用 `\cfrac{a}{b}` 命令对于连分数很有用 $\cfrac{a}{b}$，更多细节见此子文章。

9. **字体**

- 使用 `\mathbb` 或 `\Bbb` 表示“黑板粗体”：$\mathbb{CHNQRZ}$。
- 使用 `\mathbf` 表示粗体：$\mathbf{ABCDEFGHIJKLMNOPQRSTUVWXYZ\,abcdefghijklmnopqrstuvwxyz}$。
  - 对于基于表达式的字符，改用 `\boldsymbol`：$\boldsymbol{\alpha}$
- 使用 `\mathit` 表示斜体：$\mathit{ABCDEFGHIJKLMNOPQRSTUVWXYZ\,abcdefghijklmnopqrstuvwxyz}$。
- 使用 `\pmb` 表示粗斜体：$\pmb{ABCDEFGHIJKLMNOPQRSTUVWXYZ\,abcdefghijklmnopqrstuvw}$
- 使用 `\mathtt` 表示“打字机”字体：$\mathtt{ABCDEFGHIJKLMNOPQRSTUVWXYZ\,abcdefghijklmnopqrstuvwxyz}$。
- 使用 `\mathrm` 表示罗马字体：$\mathrm{ABCDEFGHIJKLMNOPQRSTUVWXYZ\,abcdefghijklmnopqrstuvwxyz}$。
- 使用 `\mathsf` 表示无衬线字体：$\mathsf{ABCDEFGHIJKLMNOPQRSTUVWXYZ\,abcdefghijklmnopqrstuvwxyz}$。
- 使用 `\mathcal` 表示“花体”字母：$\mathcal{ABCDEFGHIJKLMNOPQRSTUVWXYZ}$
- 使用 `\mathscr` 表示手写体字母：$\mathscr{ABCDEFGHIJKLMNOPQRSTUVWXYZ}$
- 使用 `\mathfrak` 表示“Fraktur”(旧式德文风格)字母：$\mathfrak{ABCDEFGHIJKLMNOPQRSTUVWXYZabc}$

10. **根号 / 方根** 使用 `sqrt`，它会自适应其参数的大小：`\sqrt{x^3}` $\sqrt{x^3}$；`\sqrt[3]{\frac xy}` $\sqrt[3]{\frac{x}{y}}$。对于复杂的表达式，可以考虑改用 `{...}^{1/2}`。

11. 某些**特殊函数**，如“lim”、“sin”、“max”、“ln”等，通常以罗马字体而非斜体字体排版。使用 `\lim`、`\sin` 等来生成它们：`\sin x` $\sin x$，而不是 `sin x` $sinx$。使用下标将记号附加到 `\lim`：`\lim_{x\to 0}`

$$\lim_{x\to 0}$$

非标准的函数名可以用 `\operatorname{foo}(x)` $\operatorname{foo}(x)$ 来设置。

12. 还有数量极其庞大的**特殊符号与记号**，多到无法在此一一列出；参见这份较短的列表，或这份详尽的列表。其中最常见的包括：

- `\lt \gt \le \leq \leqq \leqslant \ge \geq \geqq \geqslant \neq` $<$、$>$、$\le$、$\leq$、$\leqq$、$\leqslant$、$\ge$、$\geq$、$\geqq$、$\geqslant$、$\neq$。你可以用 `\not` 在几乎任何符号上加一条斜杠：`\not\lt` $\not<$，但它常常看起来不太好。
- `\times \div \pm \mp` $\times$、$\div$、$\pm$、$\mp$。`\cdot` 是居中的点：$x \cdot y$
- `\cup \cap \setminus \subset \subseteq \subsetneq \supset \in \notin \emptyset \varnothing` $\cup$、$\cap$、$\setminus$、$\subset$、$\subseteq$、$\subsetneq$、$\supset$、$\in$、$\notin$、$\emptyset$、$\varnothing$
- `{n+1 \choose 2k}` 或 `\binom{n+1}{2k}` $\binom{n+1}{2k}$
- `\to \rightarrow \leftarrow \Rightarrow \Leftarrow \mapsto` $\to$、$\rightarrow$、$\leftarrow$、$\Rightarrow$、$\Leftarrow$、$\mapsto$

- `\land \lor \lnot \forall \exists \top \bot \vdash \vDash` $\land$、$\lor$、$\lnot$、$\forall$、$\exists$、$\top$、$\bot$、$\vdash$、$\vDash$
- `\star \ast \oplus \circ \bullet` $\star$、$\ast$、$\oplus$、$\circ$、$\bullet$
- `\approx \sim \simeq \cong \equiv \prec \lhd \therefore` $\approx$、$\sim$、$\simeq$、$\cong$、$\equiv$、$\prec$、$\lhd$、$\therefore$
- `\infty \aleph_0` ∞ $\aleph_0$ `\nabla \partial` ∇、∂ `\Im \Re` $\Im$、$\Re$
- 对于模同余，按如下方式使用 `\pmod`：`a\equiv b\pmod n` $a \equiv b \pmod{n}$。
- 对于二元取模运算符，按如下方式使用 `\bmod`：`a\bmod 17` $a \bmod 17$。
- 避免使用 `\mod`，因为它会产生额外的空白：将上面的结果与 `a\mod 17` $a \mod 17$ 对比。
- `\ldots` 是 $a_1, a_2, \ldots, a_n$ 中的点 `\cdots` 是 $a_1 + a_2 + \cdots + a_n$ 中的点
- 手写体小写 l 是 `\ell` ℓ。

Detexify 让你在网页上画出一个符号，然后列出看起来与之相似的 TeX 符号。这些不保证在 MathJax 中可用，但是一个很好的起点。要检查某个命令是否受支持，请注意 MathJax.org 维护着一份当前受支持的 LaTeX 命令列表，你也可以查阅 Dr. Carol JVF Burns 的 TeX Commands Available in MathJax 页面。

13. **空白** MathJax 通常会依据一套复杂的规则自行决定公式中的间距。在公式中放入额外的字面空格不会改变 MathJax 所插入的间距量：`a␣b` 和 `a␣␣␣␣b` 都是 ab。要增加更多空白，使用 `\,` 表示窄空白 $a\,b$；`\;` 表示更宽的空白 $a\;b$。`\quad` 和 `\qquad` 是大空白：$a\quad b$、$a\qquad b$。

要排版纯文本，使用 `\text{⋯}`：$\{x \in s \mid x \text{ is extra large}\}$。你可以在 `\text{⋯}` 内嵌套 `$⋯$`，例如用于插入空格。

14. **重音符号与变音符号** 对单个符号使用 `\hat` $\hat{x}$，对较大的公式使用 `\widehat` $\widehat{xy}$。如果你把它弄得太宽，就会显得很滑稽。类似地，还有 `\bar` $\bar{x}$ 和 `\overline` $\overline{xyz}$，以及 `\vec` $\vec{x}$、`\overrightarrow` $\overrightarrow{xy}$ 和 `\overleftrightarrow` $\overleftrightarrow{xy}$。对于点，如 $\frac{d}{dx}x\dot{x} = \dot{x}^2 + x\ddot{x}$，使用 `\dot` 和 `\ddot`。

15. MathJax 解析时所用的特殊字符可以用 `\` 字符转义：`\$` $\$$、`\{` $\{$、`\_ _` 等。如果你想要 `\` 本身，应使用 `\backslash`（符号）或 `\setminus`（二元运算）来表示 $\setminus$，因为 `\\` 是用于换行的。

（教程到此结束。）

重要的是，本注记应保持合理的篇幅，不要过度膨胀。要收录更多主题，请撰写简短的附录并将其作为回答发布，而不要将它们插入本帖。

目录

按标题字母顺序排列的 MathJax 主题链接列表：

· Absolute values and norms · Additional symbolic decorations · Aligning Equations

- Alternative Ways of Writing in LaTeX · Annotations of reasoning · Arbitrary operators
- Arrays · Big braces · Colors
- Commutative diagrams · Continued fractions · Crossing things out
- Definitions by cases (piecewise functions) · Degree symbol · Display style
- Equation numbering · Fussy spacing issues · Highlighting expressions
- Left and right arrows · Limits · Linear programming
- Long division · Math Programming · Matrices
- Markov Chains · Mixing code and MathJax formatting on lines · The \newcommand function
- Numbering Equations · Overlaying Symbols · Packs of cards
- Symbols · System of equations · Tables
- Tags and references · Tensor indices · Units
- Vertical spacing

对齐的方程

https://math.meta.stackexchange.com/questions/5020/mathjax-basic-tutorial-and-quick-reference/5024#5024

人们常常希望排列一系列方程，使其等号对齐。要实现这一点，使用 `\begin{aligned}`…`\end{aligned}`。每一行都应以 `\\` 结尾，并在需要对齐之处（通常是紧接等号之前）放置一个 & 符号。

例如，

$$\begin{aligned} \sqrt{37} &= \sqrt{\frac{73^{2-1}{}^{12}2}} \\ &= \sqrt{\frac{73^{2}{}^{12}2}\cdot\frac{73^{2-1}{}^{73}2}} \\ &= \sqrt{\frac{73^{2}{}^{12}2}}\sqrt{\frac{73^{2-1}{}^{73}2}} \\ &= \frac{73}{12}\sqrt{1 - \frac{1}{73^2}} \\ &\approx \frac{73}{12}\left(1 - \frac{1}{2\cdot73^2}\right) \end{aligned}$$

是由以下代码生成的

```
\begin{aligned}
\sqrt{37} &= \sqrt{\frac{73^{2-1}{}^{12}2}} \\ &= \sqrt{\frac{73^{2}{}^{12}2}\cdot\frac{73^{2-1}{}^{73}2}} \\ &= \sqrt{\frac{73^{2}{}^{12}2}}\sqrt{\frac{73^{2-1}{}^{73}2}} \\ &= \frac{73}{12}\sqrt{1 - \frac{1}{73^2}} \\ &\approx \frac{73}{12}\left(1 - \frac{1}{2\cdot73^2}\right) \end{aligned}
```

这里通常用于界定显示公式的 `$$` 标记可以省略。

分情形定义（分段函数）

https://math.meta.stackexchange.com/a/5025

使用 `\begin{cases}⋯\end{cases}`。每种情形以 `\\` 结尾，并在应当对齐的部分之前使用 `&`。

例如，你会得到这个：

$$f(n) = \begin{cases} n/2, & \text{if } n \text{ is even} \\ 3n+1, & \text{if } n \text{ is odd} \end{cases}$$

写法如下：

```
  f(n) =
\begin{cases}
```

n/2, & \text{if n is even} \ 3n+1, & \text{if n is odd} \end{cases}

花括号可以移到右侧：

$$\left.\begin{array}{ll} \text{if } n \text{ is even:} & n/2 \\ \text{if } n \text{ is odd:} & 3n+1 \end{array}\right\} = f(n)$$

写法如下：

```
\left.
\begin{array}{ll}
```

\text{if n is even:}&n/2\ \text{if n is odd:}&3n+1 \end{array} \right} =f(n)

要在各情形之间获得更大的垂直间距，我们可以用 `\\[2ex]` 代替 `\\`。例如，你会得到这个：

$$f(n) = \begin{cases} \frac{n}{2}, & \text{if } n \text{ is even} \\[2ex] 3n+1, & \text{if } n \text{ is odd} \end{cases}$$

写法如下：

```
f(n) =
\begin{cases}
```

\frac{n}{2}, & \text{if n is even} \[2ex] 3n+1, & \text{if n is odd} \end{cases}

（一个‘ex’是等于字母 `x` 高度的长度；这里的 `2ex` 意味着该空白应为两个 ex 高。）

矩阵

https://math.meta.stackexchange.com/a/5023/

1. 使用 `$$\begin{matrix}⋯\end{matrix}$$`。在 `\begin` 和 `\end` 之间放入矩阵元素。每一行矩阵以 `\\` 结尾，并用 `&` 分隔矩阵元素。例如，

   ```
   $$
   \begin{matrix}
   1 & x & x^2 \\
   1 & y & y^2 \\
   1 & z & z^2 \\
   \end{matrix}
   $$
   ```

   生成：

   $$\begin{matrix}
   1 & x & x^2 \\
   1 & y & y^2 \\
   1 & z & z^2 \\
   \end{matrix}$$

MathJax 会调整各行各列的大小，使一切都能容纳。

2. 要添加括号，可以像教程第 6 节那样使用 `\left`⋯`\right`，或者将 `matrix` 替换为 `pmatrix` $\begin{pmatrix}1 & 2\\3 & 4\end{pmatrix}$、`bmatrix` $\begin{bmatrix}1 & 2\\3 & 4\end{bmatrix}$、`Bmatrix` $\begin{Bmatrix}1 & 2\\3 & 4\end{Bmatrix}$、`vmatrix` $\begin{vmatrix}1 & 2\\3 & 4\end{vmatrix}$、`Vmatrix` $\begin{Vmatrix}1 & 2\\3 & 4\end{Vmatrix}$。

3. 当你想省略某些条目时，使用 `\cdots` ⋯ `\ddots` ⋱ `\vdots` ⋮：

   $$\begin{pmatrix}
   1 & a_1 & a_1^2 & \cdots & a_1^n \\
   1 & a_2 & a_2^2 & \cdots & a_2^n \\
   \vdots & \vdots & \vdots & \ddots & \vdots \\
   1 & a_m & a_m^2 & \cdots & a_m^n
   \end{pmatrix}$$

4. 对于水平“增广”矩阵，在一个格式适当的表格周围加上圆括号或方括号；详见下文的数组。这里是一个例子：

   $$\left[\begin{array}{cc|c}
   1 & 2 & 3 \\
   4 & 5 & 6
   \end{array}\right]$$

由以下代码生成：

```
    $$ \left[
\begin{array}{cc|c}
  1&2&3\\
  4&5&6
\end{array}
```

\right] $$

这里 cc|c 是关键部分；它表示有三个居中的列，并在第二列和第三列之间有一条竖线。

5. 对于垂直“增广”矩阵，使用 \hline。例如

$$\left(\begin{array}{cc|c} a & b \\ c & d \\ \hline 1 & 0 \\ 0 & 1 \end{array}\right)$$

由以下代码生成

```
$$
  \begin{pmatrix}
    a & b\\
    c & d\\
  \hline
    1 & 0\\
    0 & 1
  \end{pmatrix}
$$
```

6. 对于小型行内矩阵，使用 \bigl(\begin{smallmatrix} ... \end{smallmatrix}\bigr)，例如 $\bigl(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix} \bigr)$ 由以下代码生成：

```
$\bigl( \begin{smallmatrix} a & b \\ c & d \end{smallmatrix} \bigr)$
```

来自 tactic_writing.md

- return：在 monad 中产生一个值（类型：A → m A）
- ma >>= f：从 ma : m A 中取出类型为 A 的值并将其传给 f : A → m B。等价写法：do a ← ma, f a
- f <$> ma：将函数 f : A → B 作用于 ma : m A 中的值，得到一个 m B。等同于 do a ← ma, return (f a)

- `ma >> mb`：等同于 `do a ← ma, mb`；这里 `ma` 的返回值被忽略，然后调用 `mb`。等价写法：`do ma, mb`
- `mf <*> ma`：等同于 `do f ← mf, f <$> ma`，或 `do f ← mf, a ← ma, return (f a)`
- `ma <* mb`：等同于 `do a ← ma, mb, return a`
- `ma *> mb`：等同于 `do ma, mb`，或 `ma >> mb`。为何同一件事有两种记法？历史原因。
- `pure`：等同于 `return`。同样是历史原因。
- `failure`：失败值（具体的 monad 通常有更有用的形式，如策略的 `fail` 和 `failed`）。
- `ma <|> ma'` 从失败中恢复：运行 `ma`，若失败则运行 `ma'`。
- `a $> mb`：等同于 `do mb, return a`
- `ma <$ b`：等同于 `do ma, return b`

Lean 数学理论

下列文档介绍了若干跨越多个文件的理论。

- 集合与类集对象
- 范畴论
- 线性代数
- 自然数
- 拓扑空间、一致空间与度量空间

Lean 中的数学：自然数

自然数从零开始，这与计算机科学中的标准约定一致。你可以称之为 `Nat` 或 `ℕ`（在 VS Code 中输入 `\N` 即可得到后一个符号）。

自然数是所谓的归纳类型，具有两个构造子。第一个是 `Nat.zero`，在实践中通常写作 `0` 或 `(0 : ℕ)`，即零。另一个构造子是 `Nat.succ`，它以一个自然数作为输入，并输出下一个自然数。

加法和乘法是对第二个变量作递归来定义的，核心库中许多基本结论的证明也是对第二个变量作归纳得到的。记号 `+`、`-`、`*` 是函数 `Nat.add`、`Nat.sub` 和 `Nat.mul` 的简写，而其他记号（≤、<、|）表示通常的含义（用 `\|` 输入“整除”符号）。符号 `%` 表示取模（除法后的余数）。

下面是核心 Lean 中用于处理 `Nat` 的一些函数。

```
open nat

example : Nat.succ (Nat.succ 4) = 6 := rfl

example : 4 - 3 = 1 := rfl

example : 5 - 6 = 0 := rfl -- these are naturals

example : 1 ≠ 0 := one_ne_zero

example : 7 * 4 = 28 := rfl

example (m n p : ℕ) : m + p = n + p → m = n := add_right_cancel

example (a b c : ℕ) : a * (b + c) = a * b + a * c := left_distrib a b c

example (m n : ℕ) : succ m ≤ succ n → m ≤ n := Nat.le_of_succ_le_succ

example (a b: ℕ) : a < b → ∀ n, 0 < n → a ^ n < b ^ n := pow_lt_pow_of_lt_left
```

在 mathlib 中还有更多关于自然数的基本函数，例如阶乘、最小公倍数、素数、平方根，以及一些模运

算。

```
import Mathlib.Data.Nat.Dist -- distance function
import Mathlib.Data.Nat.GCD.Basic -- gcd
import Mathlib.Data.Nat.ModEq -- modular arithmetic
import Mathlib.Data.Nat.Prime.Basic -- prime number stuff
import Mathlib.Data.Nat.Factors -- factors
import Mathlib.Tactic.NormNum.Prime -- a tactic for fast computations

open Nat

example : factorial 4 = 24 := rfl -- factorial

example (a : ℕ) : factorial a > 0 := factorial_pos a

example : dist 6 4 = 2 := rfl -- distance function

example (a b : ℕ) : a ≠ b → dist a b > 0 := dist_pos_of_ne

example (a b : ℕ) : gcd a b ∣ a ∧ gcd a b ∣ b := gcd_dvd a b

example : lcm 6 4 = 12 := rfl

example (a b : ℕ) : lcm a b = lcm b a := lcm_comm a b
example (a b : ℕ) : gcd a b * lcm a b = a * b := gcd_mul_lcm a b

-- type the congruence symbol with \==

example : 5 ≡ 8 [MOD 3] := rfl

-- nat.sqrt is integer square root (it rounds down).

#eval sqrt 1000047
-- returns 1000

example (a : ℕ) : sqrt (a * a) = a := sqrt_eq a

example (a b : ℕ) : sqrt a < b ↔ a < b * b := sqrt_lt

example : Nat.Prime 59 := by decide
```

```
-- (The default instance is `nat.decidable_prime`, which can't be
-- used by `dec_trivial`, because the kernel would need to unfold
-- irreducible proofs generated by well-founded recursion.)

-- The tactic `norm_num`, amongst other things, provides faster primality testing.

example : Nat.Prime 104729 := by
  norm_num

example (p : ℕ) : Nat.Prime p → p ≥ 2 := Prime.two_le

example (p : ℕ) : Nat.Prime p ↔ p ≥ 2 ∧ ∀ m, 2 ≤ m → m ≤ sqrt p → ¬ (m ∣ p) := prime_def_l

example (p : ℕ) : Nat.Prime p → (∀ m, Coprime p m ∨ p ∣ m) := coprime_or_dvd_of_prime

example : ∀ n, ∃ p, p ≥ n ∧ Nat.Prime p := exists_infinite_primes

-- minFac returns the smallest prime factor of n (or junk if it doesn't have one)

example : minFac 12 = 2 := rfl

-- `Nat.primeFactorsList n` is the prime factorization of `n`, listed in increasing order.
-- This doesn't seem to reduce, and apparently there has not been
-- an attempt to get the kernel to evaluate it sensibly.
-- But we can evaluate it in the virtual machine using #eval .

#eval primeFactorsList (2^32+1)
-- [641, 6700417]
```

Lean 中的数学：集合与类集合对象

列表（Lists）

Mathlib.Data.List.Basic

`List α` 是由类型 α 的元素构成的列表的类型。列表是有限且有序的，并且可以包含重复元素。列表只能包含同一类型的元素。列表通过 cons 函数构造，该函数将一个 α 类型的元素追加到列表的顶端。关于列表的更多讨论见 TPIL 第 7.5 章。

```
[1, 1, 2, 4] ≠ [1, 2, 1, 4]
```

```
[1, 2, 1, 4] ≠ [1, 2, 4]
```

多重集（Multisets）

Mathlib.Data.Multiset.Basic

`Multiset α` 是由类型 α 的元素构成的多重集的类型。多重集是有限的，可以包含重复元素，但不是有序的。它们被定义为列表对 `Perm` 等价关系的商。多重集只能包含同一类型的元素。

```
{1, 1, 2, 4} = {1, 2, 1, 4}
```

```
{1, 1, 2, 4} ≠ {1, 2, 4}
```

Finset（有限集）

Mathlib.Data.Finset.Basic

`Finset α` 是由类型 α 的互不相同的元素构成的无序列表的类型。一个 finset 由一个多重集以及该多重集不含重复元素的证明构造而成。finset 是有限的。finset 只能包含同一类型的元素。

```
{1, 1, 2, 4} = {1, 2, 1, 4}
```

```
{1, 1, 2, 4} = {1, 2, 4}
```

集合与子类型

Mathlib.Data.Set.Basic

`Set α`。集合被定义为一个谓词，即一个函数 `α → Prop`。所使用的记号如 `{n : ℕ | 4 ≤ n}` 表示大于或等于 4 的自然数构成的集合。集合可以是无限的，并且只能包含同一类型的元素。

子类型与集合类似，也是由一个谓词所定义。所使用的记号如 `{n : ℕ // 4 ≤ n}` 表示大于或等于 4 的自然数构成的类型。然而，子类型是一个类型而非集合，并且上述子类型的元素并不具有类型 `ℕ`，而是具有类型 `{n : ℕ // 4 ≤ n}`。这意味着加法在该类型上没有定义，自然数与该类型之间的相等关系也没有定义。不过，可以将该子类型的元素强制转换回自然数，正如自然数可以强制转换为整数那样，转换之后加法与相等便如常运作（关于强制转换的更多内容见 TPIL 第 6.7 章）。要构造 α 的某个子类型的元素，你需要一个 α 的元素以及它满足该谓词的证明，例如下例中的 `4` 与 `le_refl 4`。

```
def x : {n : ℕ // 4 ≤ n} := ⟨4, le_refl 4⟩
example : (x : ℕ) + 6 = 10 := rfl
```

在期望出现类型的地方都可以使用集合，此时集合会被强制转换为对应的子类型。

```
def S : Set ℕ := {n : ℕ | 4 ≤ n}
example : ∀ n : S, 4 ≤ (n : ℕ) := fun ⟨n, hn⟩ ↦ hn
```

当你需要在子类型上定义函数，或需要使用子类型的基数时，使用子类型而非集合是有益的。

有限类型

Mathlib.Data.Fintype.Basic

`Fintype α` 表示类型 α 是有限的。它由一个包含某类型全部元素的 finset 构造而成。

```
class Fintype (α : Type*) where
  /-- The `Finset` containing all elements of a `Fintype` -/
  elems : Finset α
  /-- A proof that `elems` contains every element of the type -/
  complete : ∀ x : α, x ∈ elems
```

`Fintype α` 不是一个命题，因为它包含数据，但它是一个子单例（subsingleton），意即任意两个 `Fintype α` 类型的元素都相等。

`Finset.univ` 是在给定 `Fintype α` 实例的情况下，包含某类型全部元素的 finset。

有限集合

Mathlib.Data.Set.Finite

有限集合的定义（不同于 finset）是其对应的类型与某个 `n : ℕ` 的 `Fin n` 之间存在双射。这意味着当该集合被强制转换为子类型时，类型 `Fintype s` 是非空的。利用 `Classical.choose`，你可以从 `Finite s` 的证明中产生一个 `Fintype s` 类型的对象。有一个函数 `Set.Finite.toFinset`，它从一个有限集合产生一个 finset。

基数

有三个函数 `Finset.card`、`Fintype.card` 和 `Multiset.card`，分别表示 finset、有限类型与多重集的大小。对于集合的有限基数，可以在给定该集合有限的证明的前提下使用 `Fintype.card`。

```
example : ∀ n : ℕ, Fintype.card (Fin n) = n := Fintype.card_fin
example : ∀ n : ℕ, Finset.card (Finset.range n) = n := Finset.card_range
```

这里，`Fin n` 是小于 n 的自然数的类型，而 `Finset.range n` 是小于 n 的自然数构成的 finset。

`Mathlib.SetTheory.Cardinal.Basic` 包含关于无限基数的理论。

Lean 中的数学：线性代数

半模、模与向量空间

Mathlib.Algebra.Module.Defs

此文件定义了类型类 `Module R M`，它在类型 `M` 上给出了一个 `R`-模结构。一个加法交换幺半群 `M` 是（半）环 `R` 上的模，如果存在一个标量乘法 `•`（`SMul`），它满足关于 `+`（在 `M` 和 `R` 中）以及 `*`（在 `R` 中）所期望的分配律公理。要定义一个 `Module R M` 实例，你首先需要 `Semiring R` 和 `AddCommMonoid M` 的实例。通过拆分这些依赖关系，我们避免了实例循环与菱形问题。

在一般的数学用法中，半环上的模也称为半模，域上的模也称为向量空间。我们没有单独的 `Semimodule` 或 `VectorSpace` 类型类，因为通过改变 `R`（和 `M`）上的类型类实例可以更容易地表达这些要求。在本文档中，我们将用“模”作为“半模、模或向量空间”的通称，用“环”作为“（交换）半环、环或域”的通称。

设 `m` 为任意类型，例如 `Fin n`，那么典型的例子有：`m → ℕ` 是一个 `ℕ`-半模，`m → ℤ` 是一个 `ℤ`-模，`m → ℚ` 是一个 `ℚ`-向量空间（在类型论之外，它们分别被称为 `ℕ^m`、`ℤ^m` 和 `ℚ^m`）。一个环是其自身上的模，其中 `•` 定义为 `*`（这一等式由 `simp` 引理 `smul_eq_mul` 给出）。每个加法幺半群都有一个由 `n • x = x + x + ... + x`（`n` 次）给出的标准 `ℕ`-模结构，每个加法群都有一个类似定义的标准 `ℤ`-模结构；这些也适用于（半）环。

文件 `Mathlib.LinearAlgebra.LinearIndependent` 定义了模中带索引族的线性无关性。要表达一个集合 `s : Set M` 是线性无关的，我们将其视为以自身为索引的族，写作 `LinearIndependent R ((↑) : s → M)`。

文件 `Mathlib.LinearAlgebra.Basis` 定义了模的基。

文件 `Mathlib.LinearAlgebra.Dimension.Basic` 将模的 `rank`（秩）定义为一个基数。我们也用 `rank` 表示向量空间的维数，因为其维数总是等于其秩。线性映射的 `rank` 定义为其像的维数。此文件中的大多数定义是不可计算的。

文件 `Mathlib.LinearAlgebra.Dimension.Finrank` 将模的 `finrank` 定义为一个自然数。按照约定，如果秩是无穷的，则 `finrank` 等于 0。

矩阵

Mathlib.Data.Matrix.Basic

类型 `Matrix m n α` 包含由类型 α 的元素构成的 m 行 n 列的矩形数组。它是类型 `m → n → α` 的别名。矩阵类型可以由任意类型索引。例如，一个图的邻接矩阵可以由该图中的节点索引。如果你想用自然数 `m n : ℕ` 来指定矩阵的维数，可以使用 `Fin m` 和 `Fin n` 作为索引类型。

通过给出从索引到元素的映射来构造矩阵：`(fun (i : m) (j : n) ↦ (_ : α)) : Matrix m n α`。然而，不建议使用形如 `fun i j ↦ _` 甚至 `(fun i j ↦ _ : Matrix m n α)` 的项来构造矩阵，因为 Lean 不会将它们识别为具有正确的类型。相反，应当使用 `Matrix.of`。对于由自然数索引的矩阵，你还可以使用 `Mathlib.Data.Matrix.Notation` 中定义的记号：`![![a, b, c], ![b, c, d]] : Matrix (Fin 2) (Fin 3) α`。要获取矩阵 `M : Matrix m n α` 中第 `i : m` 行、第 `j : n` 列的元素，你可以将 M 应用于这些索引：`M i j : α`。关于矩阵元素的引理通常以 `_apply` 结尾：`Matrix.add_apply M N i j : (M + N) i j = M i j + N i j`。

矩阵乘法与转置具有由命令 `open scoped Matrix` 提供的记号。矩阵的乘法照常用 * 表示。中缀运算符 `ᵥ*ᵥ` 表示 `Matrix.dotProduct`，后缀运算符 `ᵀ` 表示 `Matrix.transpose`。

在处理矩阵时，向量指的是对于任意 `Fintype m` 的一个函数 `m → α`。它们具有在 `algebra.module.pi` 中定义的模（或向量空间）结构，由逐点加法和乘法构成。行向量与列向量的区别仅由函数的选取来体现。例如，`Matrix.mulVec M v`（记作 `M *ᵥ v`）将一个矩阵与一个列向量 `v : m → α` 相乘，而 `Matrix.Vecmul v M`（记作 `v ᵥ* M`）将一个行向量 `v : m → α` 与一个矩阵相乘。如果你大量使用 `mulVec` 和 `Vecmul`，那么你可能需要考虑改用线性映射（见下文）。

置换矩阵定义在 `Mathlib.LinearAlgebra.Matrix.Permutation` 中。

矩阵的行列式定义在 `Mathlib.LinearAlgebra.Matrix.Determinant.Basic` 中。

伴随矩阵以及非奇异矩阵的逆定义在 `Mathlib.LinearAlgebra.Matrix.Adjugate` 和 `Mathlib.LinearAlgebra.Matrix.NonsingularInverse` 中。

类型 `Matrix.SpecialLinearGroup m R` 是行列式为 1 的 m 阶矩阵构成的群，定义在 `Mathlib.LinearAlgebra.Matrix.SpecialLinearGroup` 中。

线性映射与等价

Mathlib.Algebra.Module.LinearMap.Defs

类型 `M →[R]₁ M₂`（即 `LinearMap R M M₂`）表示从 R-模 M 到 R-模 M_2 的 R-线性映射。它们由其在 M 的元素上的作用来定义。类型 `M ≃[R]₁ M₂`（即 `LinearEquiv R M M₂`）是从 M 到 M_2 的可逆 R-线性映射的类型。

矩阵与线性映射之间的等价在 Matrix.toLin 中被形式化。Matrix.toLin' 表明 Matrix.mulVec 是 Matrix m n R 与 (n → R) →[R]₁ (m → R) 之间的一个线性等价。此外，LinearMap.toMatrix 接受 M_1 的一组基 ι 和 M_2 的一组基 κ，并给出 M_1 与 M_2 之间的 R-线性映射和 Matrix ι κ R 之间的等价。如果你的映射有一组明确的基，那么这一等价允许你进行诸如求行列式之类的计算。

矩阵与线性映射之间的区别在于：矩阵本质上是一个元素数组（这恰好允许诸如 Matrix.mulVec 之类的运算），而线性映射本质上是对向量的一种作用（如果我们有一组有限基，这恰好可以由一个矩阵来表示）。如果你想进行计算，矩阵是更好的选择。如果你想进行不涉及计算的证明，线性映射是更好的选择。

类型 Matrix.GeneralLinearGroup R M 是从 M 到其自身的可逆 R-线性映射构成的群。LinearMap.GeneralLinearGroup.generalLinearEquiv R M 是 GeneralLinearGroup 与 M ≃[R]₁ M 之间的等价。Matrix.SpecialLinearGroup.toGL 是从特殊线性群（矩阵的）到一般线性群（线性映射的）的嵌入。

对偶空间由线性映射 M →[R]₁ R 构成，定义在 Mathlib.LinearAlgebra.Dual 中。

双线性型、半双线性型与二次型

Mathlib.LinearAlgebra.BilinearMap

对于一个 R-模 M，类型 LinearMap.BilinForm R M 是对两个参数都线性的映射 M → M → R 的类型。LinearMap.BilinForm R M 与对两个参数都线性的映射 M →₁[R] M →₁[R] R 之间的等价称为 bilin_linear_map_equiv。一个矩阵 M 对应于一个双线性型，它将向量 v 和 w 映为 row v ⬝ M ⬝ col w。BilinForm R (n → R) 与 Matrix n n R 之间的等价称为 BilinForm.toMatrix。

Mathlib.LinearAlgebra.SesquilinearForm

对于一个 R-模 M 和 I : R →+* R，类型 M →₁ M →ₛ₁[I] R 是对第一个参数线性、而对第二个参数为 I-半线性的映射 M → M → R 的类型。f 关于环同态 I 的半线性意味着以下等式成立：f x (a • y) = I a * f x y。

Mathlib.LinearAlgebra.QuadraticForm.Basic

对于一个 R-模 M，类型 QuadraticForm R M 是满足 f (a • x) = a * a * f x 且 fun x y ↦ f (x + y) - f x - f y 是双线性映射的映射 f : M → R 的类型。

至多相差一个因子 2，二次型与双线性型的理论是等价的。LinearMap.BilinMap.toQuadraticMap f 是由 fun x ↦ f x x 给出的二次型。QuadraticMap.associatedHom f 是由 fun x y ↦ ⅟2

`* (f (x + y) - f x - f y)` 给出的双线性型（如果 2 有乘法逆元）。`QuadraticMap.toMatrix'` 和 `Matrix.toQuadraticMap'` 是二次型与矩阵之间的映射。

Lean 中的数学：拓扑空间、一致空间与度量空间

TopologicalSpace 类型类定义于 mathlib 中的 Mathlib.Topology.Defs.Basic。topology 中有大量代码，涵盖了拓扑空间、连续函数、拓扑群与拓扑环以及无穷求和的基础内容。本文档仅关注 Mathlib.Topology 文件夹中的内容。

基本类型类

TopologicalSpace 类型类是一个归纳类型，它以显然的方式定义为类型 α 上的一个结构：其中有一个 IsOpen 谓词，用以判断 U : Set α 何时为开集，随后是拓扑的诸公理（细致的说明：关于空集为开集的公理被省略了，因为它可由开集之并为开集这一事实应用于空并得出！）。

注意，将任意开集之并为开集这一公理形式化有两种方式：可以要求给定一族开集，其并为开集；也可以要求给定从某指标集 I 到开集集合的函数，该函数取值之并为开集。Mathlib 采用了前者，因此该公理为

```
isOpen_sUnion : ∀ (s : Set (set α)), (∀ t ∈ s, IsOpen t) → IsOpen (⋃₀ s)
```

而指标集版本则作为一条引理：

```
lemma isOpen_biUnion {f : ι → Set α} {s : Set ι} (h : ∀ i ∈ s, IsOpen (f i)) : IsOpen (⋃ i
```

注意 mathlib 中通行的命名约定：sUnion 是对一族集合取并，而 biUnion 是对一个指标集上的函数像取并。大写的 U 用以表示任意大小的并，与 union 相区别，后者表示两个集合之并：

```
lemma IsOpen.union (h₁ : is_open s₁) (h₂ : is_open s₂) : is_open (s₁ ∪ s₂)
```

文档中定义了谓词 IsClosed，以及函数 interior、closure 和 frontier（闭包减去内部，在数学中有时称为边界），并证明了它们的基本性质。例如

```
import Mathlib.Topology.Basic


open TopologicalSpace
variable {X : Type} [TopologicalSpace X] {U V C D Y Z : Set X}
```

```
example : IsClosed C → IsClosed D → IsClosed (C ∪ D) := IsClosed.union

example : IsOpen Cᶜ ↔ IsClosed C := isOpen_compl_iff

example : IsOpen U → IsClosed C → IsOpen (U \ C) := IsOpen.sdiff

example : interior Y = Y ↔ IsOpen Y := interior_eq_iff_isOpen

example : Y ⊆ Z → interior Y ⊆ interior Z := interior_mono

example : IsOpen Y ↔ ∀ x ∈ Y, ∃ U ⊆ Y, IsOpen U ∧ x ∈ U := isOpen_iff_forall_mem_open

example : closure Y = Y ↔ IsClosed Y := closure_eq_iff_isClosed

example : closure Y = (interior Yᶜ)ᶜ := closure_eq_compl_interior_compl
```

滤子

在 mathlib 中，不同于数学教科书中典型的处理方式，滤子被广泛用作拓扑空间理论中的工具。让我们简要回顾一下数学中滤子的概念。集合 X 上的滤子是 X 的子集所构成的一个非空族 F，满足以下两条公理：

1) 若 U ∈ F 且 U ⊆ V，则 V ∈ F；以及
2) 若 U, V ∈ F，则存在 W ∈ F 使得 W ⊆ U ∩ V。

非正式地说，可以将 F 看作 X 的”大”子集的集合。例如，若 X 是一个集合，F 是 X 的满足 X \ Y 有限的所有子集 Y 的集合，则 F 是一个滤子。它称为 X 上的 _ 有限余滤子 _（cofinite filter）。

注意，若滤子 F 包含空集，则由第一条公理它包含 X 的所有子集。这个滤子有时称为”底”（bottom，稍后我们会看到原因）。某些文献要求滤子中不允许含有空集——Lean 并无此限制。不含空集的滤子有时称为”真滤子”（proper filter）。

若 X 是拓扑空间，且 x ∈ X，则 x 的 _ 邻域滤子 _ 𝓝 x 是 X 的满足 x 属于 Y 内部的所有子集 Y 的集合。容易验证这是一个滤子（技术性说明：要看出这确实是 mathlib 中 𝓝 x 的定义，了解以下一点会有帮助：一个类型上的所有滤子构成一个完备格，其偏序为 F ≤ G 当且仅当 G ⊆ F，因此那个涉及下确界的定义实际上是一个并；此外，我这里给出的定义并非 mathlib 中字面上的定义，但 lemma mem_nhds_iff 表明它们的定义与此处的定义一致。还需注意，这正是为什么含集合最多的滤子被称为底！）。

我们为何对这些滤子感兴趣？这是因为，给定从 ℕ 到拓扑空间 X 的映射 f，可以验证所得序列 f 0、f 1、f 2……趋于 x ∈ X，当且仅当滤子 𝓝 x 中任一元素的原像属于 ℕ 上的有限余滤子——这不过是以

另一种方式表述：给定任意包含 x 的开集 U，存在 N 使得对所有 n ≥ N 有 f n ∈ U。因此滤子提供了一种思考极限的方式。

作为例子，下面用 Lean 表述了三个极限。该例使用了滤子 atTop 和 atBot，它们在配有序结构的类型中表示”趋于 ∞ “和”趋于 -∞ “。

```
open Filter Topology

-- The limit of `2 * x` as `x` tends to `3` is `6`
example : Tendsto (fun x : ℝ ↦ 2 * x) (𝓝 3) (𝓝 6) := sorry
-- The limit of `1 / x` as `x` tends to `∞` is `0`
example : Tendsto (fun x : ℝ ↦ 1 / x) atTop (𝓝 0) := sorry
-- The limit of `x ^ 2` as `x` tends to `-∞` is `∞`
example : Tendsto (fun x : ℝ ↦ x ^ 2) atBot atTop := sorry
```

附属于集合 X 的子集 Y 的 _ 主滤子 _ Filter.principal Y 是 X 的所有包含 Y 的子集所构成的族。因此不难说服自己，以下结果应当成立：

```
variable (X : Type) [TopologicalSpace X] (Y : Set X)

example : interior Y = {x | 𝓝 x ≤ Filter.principal Y} := interior_eq_nhds

example : IsOpen Y ↔ ∀ y ∈ Y, Y ∈ (𝓝 y).sets := isOpen_iff_eventually
```

用滤子刻画紧性

作为以滤子为核心的处理方式的一个后果，mathlib 中某些定义对于不习惯这一方式的数学家而言会显得相当奇怪。我们已经见过用滤子给出的关于序列趋于极限的定义。紧性的定义也以滤子论的术语写出：

```
/-- A set `s` is compact if for every nontrivial filter `f` that contains `s`,
    there exists `a ∈ s` such that every set of `f` meets every neighborhood of `a`. -/
def IsCompact (s : Set X) :=
  ∀ ⦃f⦄ [NeBot f], f ≤ 𝓟 s → ∃ x ∈ s, ClusterPt x f
```

翻译过来，这是说：拓扑空间 X 的子集 Y 是紧的，如果对 X 上的每个真滤子 F，若 Y 是 F 的元素，则存在 Y 中的元素 y，使得同时包含 F 与 y 的邻域滤子的最小滤子也不是 X 的所有子集所构成的滤子。这应当被看作 Bolzano-Weierstrass 定理（即在 ℝ^n 的紧子空间中任一序列都有收敛子列）的恰当的一般类比。

人们或许会问，为何选择这一紧性定义，而非关于开覆盖有有限子覆盖的标准定义。其原因在某种意义上是计算机科学的而非数学的——问题不应在于最终选择何种定义（事实上，开发者尽可以选择他们喜

欢的任意定义，只要它在逻辑上与通常的定义等价即可，且他们可能基于运行时间等非数学因素而有所考量），问题应在于如何证明所内置的定义与你在实践中想用的定义等价。所幸我们有

```
example : IsCompact Y ↔ ∀ {ι : Type} (U : ι → Set X),
      (∀ i, IsOpen (U i)) → (Y ⊆ ⋃ i, U i) → ∃ t : Finset ι, Y ⊆ ⋃ i ∈ t, U i :=
    isCompact_iff_finite_subcover
```

因此 Lean 中的定义与标准定义等价。

Hausdorff 空间

在 Lean 中，他们选用术语 `T2Space` 来表示 Hausdorff（也许是因为它更短!）。

```
class T2Space (X : Type u) [TopologicalSpace X] : Prop where
  /-- Every two points in a Hausdorff space admit disjoint open neighbourhoods. -/
  t2 : Pairwise fun x y => ∃ u v : Set X, IsOpen u ∧ IsOpen v ∧ x ∈ u ∧ y ∈ v ∧ Disjoint u
```

当然，Hausdorff 性正是确保极限唯一所需的条件，但由于极限是用滤子定义的，这一陈述最终读起来如下：

```
lemma tendsto_nhds_unique [T2Space X] {f : β → X} {l : Filter β} {x y : X}
  [l.NeBot] (hx : Tendsto f l (𝓝 x)) (hb : Tendsto f l (𝓝 y)) : x = y
```

注意，这一陈述实际上比经典陈述”在 Hausdorff 空间中若一序列趋于两个极限则这两个极限相同”更为一般，因为它适用于任意集合上的任意非平凡滤子，而不仅限于自然数上的有限余滤子。

拓扑的基。

若 `X` 是一个 _ 集合 _，`S` 是 `X` 的子集所构成的一个族，则可以考虑由 `S`“生成”的拓扑，它（如在此类情形中常见的那样）可以用两种方式定义：其一是 `X` 上所有包含 `S` 的拓扑之交（这里我们将拓扑等同于其底层的开集族），其二则更具构造性，即用拓扑空间的诸公理由 `S`“生成”的集合。不出所料，Lean 中采用的正是后一种定义，因为开集自然构成一个归纳类型；这些开集称为 `generate_open S`，而该拓扑为 `generate_from S`。

mathlib 中关于拓扑基的定义包含一条公理，即该拓扑按上述意义由该基生成，这可能使得终端用户难以直接证明某给定集合满足该定义。不过我们再次有一条定理，将问题归约为验证拓扑基通常的两条公理：

```
example (B : Set (Set X)) (h_open : ∀ V ∈ B, IsOpen V)
  (h_nhds : ∀ (x : X) (U : Set X), x ∈ U → IsOpen U → ∃ V ∈ B, x ∈ V ∧ V ⊆ U) :
IsTopologicalBasis B :=
isTopologicalBasis_of_isOpen_of_nhds h_open h_nhds
```

其他内容

还有其他涉及滤子的内容，有可分空间、第一可数空间和第二可数空间、积空间、子空间拓扑与商拓扑（以及更一般的拓扑的拉回与推前），还有诸如 t1 与 t3 空间之类的内容。

文件组织

以下”核心”模块构成一条线性的导入链。涉及这几个文件中所定义概念的定理，应当在此顺序中最后那个相关文件里找到。

- `Mathlib.Topology.Basic` 拓扑空间。开子集与闭子集、内部、闭包与边界（frontier）。邻域滤子。滤子的极限。局部有限族。连续性与某点处的连续性。
- `Mathlib.Topology.Order.Basic` 固定集合上的拓扑所构成的完备格结构。诱导拓扑与共诱导拓扑。
- `maps` 开映射与闭映射。“诱导”（inducing）映射。嵌入、开嵌入与闭嵌入。商映射。
- `Mathlib.Topology.Constructions` 由旧拓扑空间构造新拓扑空间：积、和、子空间与商。
- `Mathlib.Topology.Separation` 分离公理 T0 至 T6，分别也称为 Kolmogorov、Tychonoff 或 Fréchet、Hausdorff、正则与正规空间。

其余的一些目录与文件，排列不分先后：

- `Mathlib.Topology.Algebra` 配有相容的代数结构或序结构的拓扑空间。
- `Mathlib.Topology.Category` 拓扑空间、一致空间等所构成的范畴。
- `Mathlib.Topology.Instances` 具体的拓扑空间，如实数与复数。
- `Mathlib.Topology.MetricSpace` 度量空间理论；但其中某些人们可能预期会出现于此的概念，实际上被推广到了一致空间。
- `Mathlib.Topology.Sheaves` 拓扑空间上的预层。
- `Mathlib.Topology.UniformSpace` 一致空间理论，包括完备性、一致连续性与全有界集等概念。
- `Mathlib.Topology.Bases` 滤子与拓扑空间的基。可分空间、第一可数空间与第二可数空间。
- `Mathlib.Topology.CompactOpen` 两个拓扑空间之间连续映射所构成空间上的紧开拓扑。
- `Mathlib.Topology.ContinuousOn` 相对于某子集的邻域。某子集上的连续性，以及某子集内某点处的连续性。
- `Mathlib.Topology.DenseEmbedding` 嵌入及其他具有稠密像的函数。
- `Mathlib.Topology.Homeomorph` 拓扑空间之间的同胚。
- `Mathlib.Topology.List` 列表与向量上的拓扑。
- `Mathlib.Topology.Sequences` 序列闭包与序列空间。序列连续函数。
- `Mathlib.Topology.StoneCech` 拓扑空间的 Stone-Čech 紧化。

Lean 中的数学：范畴论

`Category` 类型类定义于 `Mathlib.CategoryTheory.Category.Basic`。它依赖于对象的类型, 因此例如当我们讨论一个其对象为类型（位于宇宙 `u` 中）的范畴时，可能会写作 `Category (Type u)`。

函子(它是一个结构,而非类型类)连同恒等函子与函子复合,定义于`Mathlib.CategoryTheory.Functor.Basic`。

自然变换及其复合定义于 `Mathlib.CategoryTheory.NatTrans`。

固定范畴 `C` 与 `D` 之间的函子与自然变换所构成的范畴定义于`Mathlib.CategoryTheory.Functor.Category`。

范畴、函子与自然变换的笛卡尔积出现于 `Mathlib.CategoryTheory.Products.Basic`。

类型的范畴以及 hom 配对函子定义于 `Mathlib.CategoryTheory.Types`。

记号

范畴

我们使用 ⟶（`\hom`）箭头来表示态射集，如 `X ⟶ Y`。这使得实际所涉及的范畴保持隐式；它由 `X` 与 `Y` 的类型通过类型类推断得出。

我们使用 𝟙（`\b1`）来表示恒等态射，如 `𝟙 X`。

我们使用 ≫（`\gg`）来表示态射的复合，如 `f ≫ g`，其含义为“先 `f` 后 `g`”。你也许更倾向于按照通常的约定来书写复合，即使用 ⊚（`\oo` 或 `\circledcirc`），如 `f ⊚ g` 表示“先 `g` 后 `f`”。为此，你需要通过如下方式在局部添加该记号：

```
local notation f ` ⊚ `:80 g:80 := category.comp g f
```

同构

我们使用 ≅ 来表示同构。

函子

我们使用 ⥤（\func）来表示函子，如 C ⥤ D 表示从 C 到 D 的函子的类型。

我们使用 F.obj X 来表示函子在对象上的作用。我们使用 F.map f 来表示函子在态射上的作用。

函子复合可写作 F ⋙ G。

自然变换

我们使用 τ.app X 来表示自然变换的各个分量。

除此之外，我们大多沿用任意范畴中态射的记号：

我们使用 F ⟶ G（\hom 或 -->）来表示函子 F 与 G 之间自然变换的类型。我们使用 F ≅ G（\iso）来表示自然同构的类型。

对于自然变换的纵向复合，我们直接使用 ≫。对于横向复合，则使用 hcomp。

如何使用 calc

`calc` 是一个环境——也就是一种类似于策略模式、项模式和 conv 模式 的「模式」。关于如何使用它的文档和基础示例，可参见 Theorem Proving In Lean 的 第 4 节。

基础用法示例：

```
example (a b c : ℕ) (H1 : a = b + 1) (H2 : b = c) : a = c + 1 :=
calc a = b + 1 := H1
     _ = c + 1 := by rw [H2]
```

`calc` 在策略模式中也可使用。你可以留下 `?_` 来创建一个新目标：

```
example (a b c : ℕ) (H1 : a = b + 1) (H2 : b = c) : a = c + 1 := by
  calc
    a = b + 1 := ?_
    _ = c + 1 := ?_
  · exact H1
  · rw [H2]
```

事实上，在策略模式中 `calc A = B := H ...` 的作用与调用 `refine (calc A = B := H ...)` 完全相同。

在使用 calc 时获取有效反馈

为了获得有帮助的错误信息，即便证明尚未完成，也应保持 calc 的结构。可以像上面的示例那样用 `_`，或用 `sorry` 来占据缺失的论证。`sorry` 会完全抑制错误信息，而 `_` 则会生成具有指引性的错误信息。

如果 calc 的结构不正确（例如缺少 `:=` 或其后的论证），你可能会看到晦涩难懂的错误信息，和/或最终落在某个随机 `_` 下方的红色波浪线。为避免这类情况，你可以先填入一个骨架证明，例如：

```
example (A B C D : ℝ ) : A = D :=
calc A = B := sorry
```

```
    _ = C := _
    _ = D := sorry
```

然后再逐步填入 sorry 和 _。

```
example (A B C D : ℝ ) : A = D := by
    calc A = B := sorry
         _ = C := ?_
         _ = D := sorry
    sorry
```

此外还有一个 Calc 小部件可以让这件事更简单，这里有一段视频演示了它。

使用等号以外的运算符

TPIL 中的许多基础示例对大部分或全部运算符都使用等号，但实际上 `calc` 可以与任何我们为之创建了 `Trans` 实例的关系一起使用

```
def r : ℕ → ℕ → Prop := sorry
variable (a b c: ℕ)
def r_trans {a b c} (h₁ : r a b) (h₂ : r b c) : r a c := sorry

instance : Trans r r r where
  trans := r_trans

infix:50 "***" => r

example (a b c : ℕ) (H1 : a *** b) (H2 : b *** c) : a *** c :=
calc a *** b := H1
    _ *** c := H2
```

使用多个运算符

这是可行的，例如：

```
theorem T2 (a b c d : ℕ)
  (h1 : a = b) (h2 : b ≤ c) (h3 : c + 1 < d) : a < d :=
  calc
    a = b     := h1
```

```
    _ < b + 1 := Nat.lt_succ_self b
    _ ≤ c + 1 := Nat.succ_le_succ h2
    _ < d     := h3
```

这里到底发生了什么？证明本身并不神秘，例如 `Nat.succ_le_succ h2` 是 `b + 1 ≤ c + 1` 的一个证明。巧妙之处在于，Lean 能够将所有这些组合起来，正确地推导出：若 `U = V < W ≤ X < Y`，则 `U < Y`。请注意以下的微妙之处：给定 `U op1 V` 和 `V op2 W`，Lean 必须为某个运算符得出 `U op3 W`，而这个运算符可能是 `op1` 或 `op2`（甚至，正如我们将看到的那样，是一个新的运算符）。Lean 是如何做到这一点的呢？最简单的情形是当 `op1` 和 `op2` 之一为 `=` 时。Lean 知道

```
#check trans_rel_right -- {α : Sort u} {a b c : α} (r : α → α → Prop) (h₁ : a = b) (h₂ : r
#check trans_rel_left -- {α : Sort u} {a b c : α} (r : α → α → Prop) (h₁ : r a b) (h₂ : b
```

并在其中一个运算符为等号运算符时使用它们。然而，如果两个运算符都不是等号运算符，Lean 就会查找 `Trans` 的各个实例并转而应用它们。

使用用户自定义的运算符

这就像创建相应的 `Trans` 实例一样简单。例如

```
def r : ℕ → ℕ → Prop := sorry
def s : ℕ → ℕ → Prop := sorry
def t : ℕ → ℕ → Prop := sorry

theorem rst_trans {a b c : ℕ} : r a b → s b c → t a c := sorry
infix:50 "****" => r
infix:50 "^^^^" => s
infix:50 "%%%%" => t

instance : Trans r s t  where
  trans := rst_trans

example (a b c : ℕ) (H1 : a **** b) (H2 : b ^^^^ c) : a %%%% c :=
calc a **** b := H1
     _ ^^^^ c := H2
```

这个示例向我们表明，第三个运算符 `op3` 可以与 `op1` 和 `op2` 都不同。

如何使用 congr()

`congr()` 是一个项繁释器（term elaborator），它利用同余性（congruence）来构造关于相等、`HEq` 与 `Iff` 的证明。最典型的同余引理称为 `congr`，它断言：当两个表达式（此处为两个函数应用）的各个组成部分相等（此处为函数本身及其参数）时，这两个表达式相等。

```
theorem congr {f₁ f₂ : α → β} {a₁ a₂ : α} (h₁ : f₁ = f₂) (h₂ : a₁ = a₂) : f₁ a₁ = f₂ a₂
```

同余引理使我们能够将多个相等证明串联起来，构成一个更大的证明。与其手动地使用 `congr` 及其特化版本 `congrFun` 和 `congrArg` 来组合证明，`congr()` 繁释器能以更少的语法噪声为你完成这一工作。

基本用法示例：

```
import Mathlib.Tactic.TermCongr

example (a b : Nat) (h : a = b) : 4 * (37 + a) = 4 * (37 + b) :=
  congr(4 * (37 + $h))
```

这里，传给 `congr()` 的表达式中，除了我们书写 `$h` 的位置之外，其余各部分都同时出现在等式的两侧。语法 `$h` 的含义是：`congr()` 将 `h` 的左端 `a` 插入到所得等式的左侧，将 `h` 的右端 `b` 插入到所得等式的右侧。`$h` 是反引用（antiquotation）的一个例子。在本文后面，我们将介绍 `congr()` 所使用的引用与反引用的完整语法。

`congr()` 支持 `Eq`、`Iff` 与 `HEq`，并能根据需要在它们之间相互转换：

```
example (a b : Nat) (h : a = b) : 4 * (37 + a) = 0 ↔ 4 * (37 + b) = 0 :=
  congr(4 * (37 + $h) = 0)

example (p q : Prop) [Decidable p] [Decidable q] (h : p ↔ q) :
    (if p then 1 else 0) = if q then 1 else 0 :=
  congr(if $h then 1 else 0)

example (a b : Nat) (h : a = b) [NeZero a] [NeZero b] : (37 : Fin a) ≍ (37 : Fin b) :=
  congr((37 : Fin $h))
```

语法

与其他繁释器和宏一样，括号总是紧贴 `congr()` 书写，中间不留空格。`congr(...)` 与 `congr (...)` 含义不同：前者是我们正在讨论的项繁释器，而后者则是库中的 `congr` 引理应用于参数 `(...)`。

括号内放置的是一个准引用（quasiquoted）表达式。除了普通表达式的语法之外，你还可以插入反引用，从而使其内容可以变化。反引用有两种：

- `$ident`，如上面例子中的 `$h`，用于引用单个变量。
- `$(term)`，可以在项的位置插入一整个表达式。

每当 `congr()` 遇到一个反引用时，它就将该反引用的内容当作一个表达式来处理。只要该表达式能够在当前上下文中被繁释，它可以任意复杂：

```
example (a b : Nat) (h : a = b) : 4 * (37 + a) = 4 * (37 + b) :=
  congr(4 * (37 + $h))

example (a b : Nat) (h : a = b) : 4 * (37 + a) = 4 * (37 + b) :=
  congr(4 * (37 + $(h)))

example (a b : Nat) (h : a = b) : 4 * (37 + a) = 4 * (37 + b) :=
  congr(4 * (37 + $(h.symm.symm)))

example (a b : Nat) (h : a = b) : 4 * (37 + a) = 4 * (37 + b) :=
  congr($(by rfl) * ($(by grind) + $(by simp [h])))
```

反引用内部的表达式可以引用局部变量，当你的证明尚未完全应用时这一点很有用：

```
example (f g : Nat → Nat) (h : ∀ i, f i = g i) (a : Nat) : f a = g a :=
  congr($(h a))

-- Make sure to apply the proof inside the antiquotation brackets.
-- The example below fails, because `h` is expected to be an equality of functions.
example (f g : Nat → Nat) (h : ∀ i, f i = g i) (a : Nat) : f a = g a :=
  congr($h a) -- Error: function expected.

-- The local variables can be bound inside the `congr` expression:
example (f g : Nat → Nat) (h : ∀ i, f i = g i) : (fun a => f a) = (fun a => g a) :=
  congr(fun a => $(h a))

-- Since `fun a => f a` is equal to `f`, this is a tricky proof of function extensionality
example (f g : Nat → Nat) (h : ∀ i, f i = g i) : f = g :=
```

```
  congr(fun a => $(h a))
```

证明

根据期望类型的不同，`congr()` 将产生一个 `Eq`、`HEq` 或 `Iff`。为简单起见，本节中我们假设 `congr()` 产生的是相等。我们将 `congr(t)` 中的项 `t` 称为模式（pattern），将 `congr(t)` 的类型称为结果（result）。若模式中不含反引用，则 `congr(t)` 等价于 `rfl : t = t`。对于任何一个反引用，结果的左侧将包含该反引用类型的左端，结果的右侧将包含该反引用类型的右端。

`congr()` 对模式进行两次处理：一次作为左侧，一次作为右侧。被引用的表达式按常规方式繁释，直到 `congr()` 遇到一个反引用为止：这些反引用首先在不带期望类型的情况下被繁释，以便确定其左端与右端。若此次繁释得到一个带有洞（hole）的类型，则这些洞会被推迟，留待之后通过与目标的合一（unification）来求解。

换言之，`congr()` 利用期望类型来填补模式中的洞：

```
example (a b : Nat) (h : a = b) : 1 + a = 1 + b := by
  have : 1 + _ = _ := congr(_ + $h)
  exact this
```

这意味着反引用可以按需推迟，例如仅在最后才调用 `assumption` 策略：

```
example (a b : Nat) (h : a = b) : 4 * (37 + a) = 4 * (37 + b) :=
  congr(4 * (37 + $(by assumption)))
```

但若缺少期望类型而反引用仍需被填补，则它会带着元变量（metavariable）运行，从而可能做出错误的猜测：

```
example (a b c : Nat) (h : a = b) (h2 : b = c) : 1 + a = 1 + b := by
  have : 1 + _ = _ := congr(_ + $(by assumption -- Goal: `?m.55 = ?m.56`, filled with `h2
  exact this -- Error: `this` has type `1 + b = 1 + c`
```

当模式被繁释两次后，它会与期望类型进行合一，以填补大多数剩余的洞。最后，将左侧与右侧相互匹配，并适当地插入同余引理或反引用内部的证明。`congr()` 对 `Subsingleton` 与命题外延性有专门的支持，使其能够处理依赖于诸如 `Decidable` 与 `Fintype` 等单元素类型类（subsingleton class）的函数。

根据上下文，`congr()` 能够在 `Eq`、`HEq` 或 `Iff` 的证明之间相互转换。默认情况下，`congr()` 会产生一个 `Eq` 的证明。当期望类型有所暗示时，它会使用 `HEq` 或 `Iff`：

```
example (p q : Prop) [Decidable p] [Decidable q] (h : p ↔ q) : (if p then 1 else 0) = (if
  have := congr($h)
  -- No expected type given above, so `this : p = q`.
```

```
  exact congr(if $this then 1 else 0)

example (p q : Prop) [Decidable p] [Decidable q] (h : p ↔ q) : (if p then 1 else 0) = (if
  have : _ ↔ _ := congr($h)
  -- `Iff` expected above, so `this : p ↔ q`.
  exact congr(if $this then 1 else 0)
```

`congr()` 能够深入绑定子（binder）内部。如果你的相等尚未完全应用，请务必使用正确的语法将其应用于绑定子内部：

```
example (f g : Nat → Nat) (s t : Finset Nat) (hfg : ∀ i, f i = g i) (hst : s = t) :
    ∑ i ∈ s, f i = ∑ i ∈ t, g i :=
  congr(∑ i ∈ $hst, $(hfg i))

-- Equivalent to:
example (f g : Nat → Nat) (s t : Finset Nat) (hfg : f = g) (hst : s = t) :
    ∑ i ∈ s, f i = ∑ i ∈ t, g i :=
  congr(∑ i ∈ $hst, $hfg i)
```

相关功能

有许多类似 `congr()` 的工具可用于构造涉及相等的证明。

- `congrm` 是 `congr()` 在策略模式下的等价物。除了 `congr()` 所支持的反引用之外，传给 `congrm` 的项还可以包含 `?_` 占位符，这些占位符将转化为新的证明目标。`congrm?` 是一个交互式小部件（widget），通过点击子表达式将其转化为洞来生成 `congrm` 调用。
- `congr` 与 `congr!` 策略利用同余规则来证明相等。`congr()` 繁释器从表达式各个小部分的证明出发，利用同余规则将它们组装成整个表达式的一个证明；而 `congr` 与 `congr!` 策略则反其道而行之，利用同余规则将整个表达式的相等目标拆解为若干小部分。`congr!` 比 `congr()` 更强大，因为它能够使用自定义的同余引理。
- 三角符号 `▸` 是一个用于在表达式类型内部进行重写的宏。若 `h : a = b` 且目标为 `4 * (37 + a) = 4 * (37 + b)`，则 `h ▸ _` 中的洞的期望类型为 `4 * (37 + a) = 4 * (37 + a)`。因此 `h ▸ rfl` 可以闭合这一目标，正如 `congr(4 * (37 + $h))` 一样。与 `congr()` 不同，三角符号是由外向内工作的：如果我们在没有期望类型的情况下写 `have := h ▸ rfl`，最终会得到一个假设 `this : a = b`。
- 诸如 `rw` 与 `simp` 之类的重写策略可以利用形如 `h : a = b` 的假设来证明形如 `4 * (37 + a) = 4 * (37 + b)` 的目标，办法是将等式两侧都重写为相同的形式，再用 `rfl` 闭合目标。与 `congr()` 不同，这些策略也是由外向内工作的：如果缺少目标类型，它们就会失败。`simp` 比 `congr()` 更强大，因为它使用了一个更强大的同余引理生成器。

局限性

`congr()` 无法处理所有可能的表达式：它只能组合你所传入的那些确切的相等。下面的例子之所以成功，是因为通过处处使用 `h : a = b` 进行重写，`(37 : Fin a)` 可以被转化为 `(37 : Fin b)`：

```
example (a b : Nat) (h : a = b) [NeZero a] [NeZero b] : (37 : Fin a) ≠ (37 : Fin b) :=
  congr((37 : Fin $h))
```

另一方面，下面的例子会失败，因为通过使用 `h : x ≠ y` 进行重写，无法将 `(37 + _ : Fin a)` 转化为 `(37 + _ : Fin b)`。

```
example (a b : Nat) [NeZero a] [NeZero b] (x : Fin a) (y : Fin b) (h : x ≠ y) :
    37 + x ≠ 37 + y :=
  congr(37 + $h) -- Error: could not generate congruence between `a` and `b`.
```

我们需要自行构造 `a = b` 的证明，并将其插入到表达式中正确的位置，才能让 `congr()` 工作：

```
-- import Mathlib.Data.Fin.Embedding to make this work
example (a b : Nat) [NeZero a] [NeZero b] (x : Fin a) (y : Fin b) (h : x ≠ y) :
    37 + x ≠ 37 + y :=
  have hab : a = b := Fin.equiv_iff_eq.mp ⟨type_eq_of_heq h ▸ Equiv.refl _⟩
  congr(Add.add (α := Fin $hab) 37 $h)
```

缺陷

在极少数情况下，内部元数据或可化简（reducible）包装会从所生成的项中泄漏出来，暴露于合一过程之中。用户看不到这些内容，但某些策略的模式匹配可能配置得过于严格，从而被它们绊倒。如果你注意到某个策略在使用 `congr()` 之后无法应用，请在 Lean 社区的 Zulip 聊天上报告该问题。

转换策略模式

在策略块内部，可以使用关键字 conv 进入转换模式（conversion mode）。该模式允许深入到假设和目标内部，甚至深入其中的 fun 绑定子内部，以施加重写或化简步骤。

这类似于其他定理证明器（如 HOL4、HOL Light 或 Isabelle）中的转换策略组合子（tactic combinators）。

基本导航与重写

作为第一个例子，让我们证明 example (a b c : ℕ) : a * (b * c) = a * (c * b)（本文中的例子多少有些刻意，因为 Tactic.Ring 中的 ring 策略可以立即完成它们）。最朴素的初次尝试是进入策略模式并尝试 rw [mul_comm]。但这会在交换项中出现的第一个乘法之后，把目标变成 b * c * a = a * (c * b)。有几种方法可以修正这一问题，其中一种是使用更精确的工具：转换模式。下面的代码块在每一行之后展示了当前的目标。注意目标以 | 为前缀，而普通模式中的目标以 ⊢ 为前缀（尽管如此，这些目标仍被称为”目标”）。

```
example (a b c : ℕ) : a * (b * c) = a * (c * b) := by
  conv =>            -- | a * (b * c) = a * (c * b)
    lhs              -- | a * (b * c)
    congr            -- | a and | b * c
    · skip            -- | a
    · rw [mul_comm] -- | c * b
```

上面的代码片段展示了三个导航命令：* lhs 导航到关系（此处为等式）的左侧，还有一个 rhs 导航到右侧。* congr 创建与当前头部函数（此处头部函数为乘法）的参数数量相同的多个目标 * skip 转到下一个目标

一旦到达相关的目标，我们便可像在普通模式中那样使用 rw。注意，若当前目标变为 x = x（在严格的句法意义下，定义等价是不够的：此时需要用 rfl 或 trivial 来收尾），Lean 会尝试求解它。

供参考，我们可以把“conv => lhs ..”写成“conv_lhs => ..”：

```
example (a b c : ℕ) : a * (b * c) = a * (c * b) := by
  conv_lhs =>
    congr
    · skip
    · rw [mul_comm]
```

使用转换模式的第二个主要理由是在绑定子下进行重写。假设我们想证明 `example (fun x : ℕ ↦ 0 + x) = (fun x ↦ x)`。最朴素的初次尝试是进入策略模式并尝试 `rw [zero_add]`。但这会令人沮丧地失败:

```
tactic 'rewrite' failed, did not find instance of the pattern in the target expression
  0 + ?a
⊢ (fun x ↦ 0 + x) = fun x ↦ x
```

解决方法是:

```
example : (fun x : ℕ ↦ 0 + x) = (fun x ↦ x) := by
  conv_lhs =>     -- | fun x ↦ 0 + x
    ext x         -- | 0 + x
    rw [zero_add] -- | x
```

其中 `ext` 是进入 `fun` 绑定子内部的导航命令。注意这个例子多少有些刻意，我们也可以这样做:

```
example : (fun x : ℕ ↦ 0 + x) = (fun x ↦ x) := by
  funext x; rw [zero_add]
```

以上所有这些也可用于重写局部上下文中的某个假设 `H`，方法是使用 `conv at H`。

模式匹配

使用上述命令进行导航可能很繁琐。我们可以利用模式匹配将其简化，如下所示:

```
example (a b c : ℕ) : a * (b * c) = a * (c * b) := by
  conv in (b * c) => -- | b * c
    rw [mul_comm]    -- | c * b
```

我们可以把这个证明写成一行:

```
example (a b c : ℕ) : a * (b * c) = a * (c * b) := by
  conv in (b * c) => rw [mul_comm]
```

当然，也允许使用通配符:

```
example (a b c : ℕ) : a * (b * c) = a * (c * b) := by
  conv in (_ * c) => rw [mul_comm]
```

在所有这些情况中，只有第一个匹配会受到影响。在转换模式内部，还可以使用 `for` 命令来进行更复杂的模式匹配。下面的代码仅对 `b * c` 的第二个和第三个出现位置执行重写：

```
example (a b c : ℕ) : (b * c) * (b * c) * (b * c) = (b * c) * (c * b) * (c * b) := by
  conv in (occs := 2 3) (b * c) =>
    · rw [mul_comm]
    · rw [mul_comm]
```

我们可以用“all_goals”把这个证明写成一行：

```
example (a b c : ℕ) : (b * c) * (b * c) * (b * c) = (b * c) * (c * b) * (c * b) := by
  conv in (occs := 2 3) (b * c) => all_goals rw [mul_comm]
```

转换模式中的其他策略

除了使用 `rw` 进行重写之外，还可以使用 `simp`、`dsimp`、`change` 和 `whnf`。`change` 是一个有用的工具——它允许将一个项更改为与之定义等价的某物，颇像策略模式中的 `show` 命令。`whnf` 命令意为”归约到弱头范式（weak head normal form）“，将在 Metaprogramming in Lean 4 第 4 节中加以解释。

mathlib 为 `conv` 提供的扩展，例如 `ring` 和 `norm_num`，可以通过 `Mathlib.Tactic.HelpCmd` 中的 `#help conv` 命令找到。

Simp

概述

在本文档中，我们将介绍 Lean 4 中化简器策略 `simp` 以及相关策略 `dsimp` 的基本用法。

我们将给出一些避免“非终结性 `simp`”的建议，并简要描述 `simp` 和 `dsimp` 的配置选项。

引言

Lean 拥有一个称为 `simp` 的“化简器”，它会查询一个称为 `simp` 引理的事实数据库，以（期望地）化简假设和目标。该化简器是所谓的 条件项重写系统：它所做的全部工作就是反复将形如 `A` 的子项替换（或 重写）为 `B`，对所有适用的形如 `A = B` 或 `A ↔ B` 的事实皆如此。化简器会机械地不断重写，直到再也无法重写为止。所有 `simp` 引理都是有方向的：左端始终被右端替换，而绝不会反过来。

理想情况下，这个事实数据库应当能将表达式化简为某种范式（normal form）。在实践中，这往往无法实现（范式可能不存在，或者可能不存在一组能产生范式的重写规则），但我们仍尽可能地力求逼近这一理想。更进一步，我们希望这个事实数据库是 合流的（confluent），即化简器考虑重写的顺序并不影响结果。同样，我们也尽可能地力求接近合流。

虽然这套系统能够完全自动地证明许多简单的命题，但证明所有简单命题并不在它的职责范围之内，尽管这或许令人失望。

下面是一个示例（使用了 mathlib）。

```
import Mathlib.Algebra.Group.Defs

variable (G : Type) [Group G] (a b c : G)

example : a * a⁻¹ * 1 * b = b * c * c⁻¹ := by
  simp
```

人类会如何求解这个目标呢？他们会注意到 `a * a⁻¹ = 1`、`1 * 1 = 1`，等等，直到将该示例化简为 `b = b`，而这显然成立。

化简器所做的也正是如此。事实上,如果你在该示例上方加上 `set_option trace.Meta.Tactic.simp.rewrite true`，那么 `simp` 下方就会出现一条蓝色波浪下划线（在 VS Code 中），点击它便会显示 `simp` 所执行的重写序列：

```
[Meta.Tactic.simp.rewrite] @mul_right_inv:1000, a * a⁻¹ ==> 1

[Meta.Tactic.simp.rewrite] @mul_one:1000, 1 * 1 ==> 1

[Meta.Tactic.simp.rewrite] @one_mul:1000, 1 * b ==> b

[Meta.Tactic.simp.rewrite] @mul_inv_cancel_right:1000, b * c * c⁻¹ ==> b

[Meta.Tactic.simp.rewrite] @eq_self:1000, b = b ==> True
```

`simp?` 策略是提取 `simp` 所应用的引理列表的一种有用方式。它会建议

```
simp only [mul_right_inv, mul_one, one_mul, mul_inv_cancel_right]
```

这是一次使用了这特定四条引理的 `simp` 调用。

若要查看化简过程中那些成功以及失败的重写,你可以使用更为详尽的选项 `set_option trace.Meta.Tactic.simp true`。

Simp 引理

那么 Lean 的化简器是如何知道 `a * a⁻¹ = 1` 的呢? 这是因为在 `Mathlib.Algebra.Group.Defs` 中有一条被标注了 `simp` 属性的引理：

```
@[simp] lemma mul_right_inv (a : G) : a * a⁻¹ = 1 := ...
```

我们将标注了 `simp` 属性的引理称为“`simp` 引理”。下面是 mathlib 中更多 `simp` 引理的例子：

```
@[simp] theorem Nat.dvd_one {n : ℕ} : n ∣ 1 ↔ n = 1 := ...
@[simp] theorem mul_eq_zero {a b : ℕ} : a * b = 0 ↔ a = 0 ∨ b = 0 := ...
@[simp] theorem List.mem_singleton {a b : α} : a ∈ [b] ↔ a = b := ...
@[simp] theorem Set.setOf_false : {a : α | False} = ∅ := ...
```

当化简器试图化简某个项 `T` 时，它会查阅系统在那一时刻已知的 `simp` 引理；若遇到一条适用的、形如 `A = B` 或 `A ↔ B` 的引理，且 `A` 作为子表达式出现在 `T` 中，它就会将 `T` 中 `A` 的那个实例重写为 `B`，然后再从头开始。注意 `simp` 从最内层的项开始，向外推进：它会先化简函数的参数，然后再化简函数本

身。此外，simp 内部具有一定的机巧，使其能够避免每次都考虑 所有 simp 引理（截至 2026 年 2 月，mathlib 中有四万余条这样的引理）。

化简器只在一个方向上应用 simp 引理：若 A = B 是一条 simp 引理，那么 simp 会将 A 替换为 B，但不会将 B 替换为 A。因此一条 simp 引理应当具有这样的性质：其右端比左端更简单。特别地，在这种情形下，= 和 ↔ 不应被视为对称的运算符。下面将是一条糟糕透顶的 simp 引理（假如它竟被允许的话）：

```
@[simp] lemma mul_right_inv_bad (a : G) : 1 = a * a⁻¹ := ...
```

将 1 替换为 a * a⁻¹ 并不是一个合理的默认推进方向。更糟糕的是那种会使表达式无限制增长的引理，它会导致 simp 永远循环下去：

```
@[simp] lemma even_worse_lemma: (1 : G) = 1 * 1⁻¹ := ...
```

在做出一个新定义时，通常也会同时引入若干 simp 引理，以将涉及该定义的表达式化为某种合理的形式。一个例子是 mathlib 中的 Data.Complex.Basic，其中有将近 100 条 simp 引理。尽管诸如

```
@[simp] lemma add_re (z w : ℂ) : (z + w).re = z.re + w.re := rfl
```

这样的定理依定义为真，它们仍被引入，因为它们赋予 simp 化简表达式并进而利用已有事实的能力。例如这一条就把复数加法转化为实数加法。如果你允许 simp 使用实数加法的交换律，那么它便能够经由 z.re + w.re = w.re + z.re 自动证明 (z + w).re = (w + z).re，而这正是复数加法可交换之证明的一半。

Lean 内核本身就是一个针对 lambda 演算的重写系统，它有着明确的推进进展之概念。有鉴于此，一类有用的 simp 引理是那些在此意义上让 simp 能够部分求值表达式的引理。例如，假设你有一个结构类型 Foo，并用该类型定义一个结构 myFoo，

```
structure Foo where n : ℕ

def myFoo : Foo where n := 37
```

那么如果你添加一条 simp 引理 myFoo.n = 37，就赋予了化简器对 myFoo 求值 Foo.n 投影的能力，从而省去展开 myFoo 定义的麻烦（默认情况下 simp 并不展开大多数定义）。创建这类 simp 引理是如此常见，以至于有 一个属性 能自动为你创建它们：

```
@[simps] def myFoo : Foo where n := 37
```

这会生成引理 @[simp] lemma myFoo_n : myFoo.n = 37。

基本用法

- simp 试图利用 Lean 在那一时刻已知的所有 simp 引理来化简目标。

- `simp [h1, h2]` 使用所有 simp 引理，外加 h1 和 h2（它们既可以是局部假设，也可以是出于某种原因未被标注为 simp 的其他引理）。
- `simp [← h]` 使用所有 simp 引理，并以反向形式 `B = A` 使用 `h : A = B`（即 simp 将 B 重写为 A）。
- `simp [-thm]` 阻止 simp 使用名为 thm 的 simp 引理。
- `simp [*]` 使用所有 simp 引理，外加当前所有局部假设，来尝试化简目标。
- `simp at h` 试图利用所有 simp 引理来化简 h。
- `simp [h1] at h2` ⊢ 试图利用 h1 和所有 simp 引理来化简 h2 和目标（注意：在 VS Code 中用 `\|-`、`\goal` 或 `\vdash` 输入⊢）。
- `simp [*] at *`：试图利用所有假设和所有 simp 引理来化简目标和所有假设。有时值得一试。
- `simp only [h1, h2, ..., hn]` 告诉 simp 只使用引理 h1、h2、……, 而非整套 simp 引理。（在证明中途使用 `simp only [...]` 是可以接受的，因为后续对 simp 集合的改动不会破坏该证明。）
- `simp [↓h1]` 在进入子项 之前使用 h1。通常情况下，simp 在进入子项 之后才使用引理。
- `simp [↑h1]` 在进入子项 之后使用 h1。此语法用于标注了 simp↓ 的引理。

注意，某些 simp 引理带有必须满足的附加假设。例如，一条关于在等式两端消去某因子的定理，只在该因子非零这一假设下方才成立。若 h 是假设 P 的一个证明，且 `P → A = B` 是一条 simp 引理，那么 `simp [h]` 会将目标中的 A 替换为 B。simp 会考虑附加假设这一事实，正是它被称为 条件项重写系统的原因。

Simp 范式

有时同一件事有若干种表述方式。例如，若 `n : ℕ`，则假设 `n ≠ 0`、`0 ≠ n`、`n > 0`、`0 < n`、`1 ≤ n` 和 `n ≥ 1` 在逻辑上都是等价的。这对于化简器这类重写系统可能造成困扰。原因在于化简器是通过 语法相等（syntactic equality）来寻找子项的。如果化简器正在处理项 T，且 `A = B` 是一条 simp 引理，那么除非 T 的某个子项 `A'` 与 A 在语法上完全相同（大致而言：它们具有字面上相同的文本表示），否则 simp 一般不会注意到该规则适用，因而不会将其重写为 B。类似地，若某条形如 `A = B` 的 simp 引理以 n 的非零性（以某一种方式表述）作为前提条件，而 h 是 n 非零性（以另一种方式表述）的一个证明，那么 `simp [h]` 可能不会将 A 替换为 B。

mathlib 中处理这个问题的方式，是一劳永逸地为某事物的表达方式固定一个 simp 范式（例如以 `0 < n` 表示非零性），然后在陈述引理时始终坚持采用这一变体。这样就免去了为每一种变体编写重复引理的麻烦。为帮助化简器，往往会有一些规范化引理，其唯一目的就是把表达式化为 simp 范式。

一般而言，如果你在写一条引理，你应当知道表达引理中各想法的“范式”方式。`#simp` 命令可以帮助你了解这一点：对表达式 `e` 写下 `#simp e`，便会用适用的 `simp` 引理化简该表达式。如果你在写一条关于自己所作定义的引理，请思考那些可以用多于一种方式表达的想法的范式。

`simp` 范式的一个例子，是表达类型的某个子集非空性的方式。若 `α : Type` 且 `s : Set α`，则 `s` 的非空性既可表示为 `s.Nonempty`，也可表示为 `s ≠ ∅`。在 mathlib 中，人们努力坚持以 `s.Nonempty` 作为范式形式。

另一个例子：每个有限集 `s : Finset α` 都可以被强制转换为 `Set α`，因此对 `a : α` 而言，`a ∈ s` 和 `a ∈ (s : Set α)` 都可表示同一含义。有限集中隶属关系的 simp 范式是 `a ∈ s`，而且还有一条规范化 `simp` 引理

```
@[simp] lemma mem_coe {a : α} {s : Finset α} : a ∈ (s : Set α) ↔ a ∈ s := ...
```

用于将 `a ∈ (s : Set α)` 的出现替换为正确的范式。

由于化简器是从内向外工作的——先化简函数的参数，再化简函数本身——因此一条 `simp` 引理在其左端，函数的参数应当处于 simp 范式。例如，若 `g 0` 可被化简，那么 `@[simp] lemma foo : f (g 0) = 0` 将永远不会被使用。Batteries 的 `simpNF` linter 会检查这一点（你可以通过在文件末尾加上 `#lint`，自行对某个模块运行 mathlib 的 linter）。

simpa

`simpa` 策略是 `simp` 的一种变体，用于完成证明——作为一个“收尾”策略，若它无法关闭目标，就会失败。其基本用法是

```
simpa [h1, h2] using e
```

其中 `[h1, h2]` 指可选的 `simp` 引理列表（语法与 `simp` 相同），而 `e` 是一个表达式。`e` 通常是某个假设的名字。`e` 的类型与目标都会被化简，若两者被化简为同一事物，`simpa` 即告成功。

下面是一个简单的 `simpa` 示例：

```
example (n : ℕ) (h : n + 1 - 1 ≠ 0) : n + 1 ≠ 1 := by
  simpa using h
```

若不用 `simpa`，我们或许会写 `simp at ⊢ h; exact h`。所谓“非终结性 `simp`”，即那些不关闭目标的 `simp` 用法，最好加以避免（参见下一节），而 `simpa` 正是避免它们的一种方式。

如果没有 `using` 子句，那么 `simpa` 转而执行以下三个步骤：

1. 化简目标。
2. 如果局部上下文中有一个名为 `this` 的假设，则化简其类型。
3. 应用 `assumption` 策略。

第 2 步是为了支持这样一种模式：`simpa` 紧跟在 `have : P` 或 `suffices : P` 之后，因为这两者都默认以 `this` 作为它们所引入假设的名字。

非终结性 `simp`

随着 `simp` 引理被添加进库（或从库中移除），`simp` 的行为也会随时间变化。这意味着使用 `simp` 的证明可能失效，而且，除非你了解 `simp` 引理集合是如何变化的，否则修复一个证明可能颇为困难。

例如，如果一个证明长这样

```
...
simp
rw [foo_eq_bar]
...
```

之后某人给 `foo_eq_bar` 加上了 `@[simp]` 属性，那么这次重写如今就会失败。

虽然在初始开发期间，于证明中途使用 `simp` 是没问题的，但经验法则是：当每个 `simp` 都完整地关闭一个目标时，Lean 代码更易于维护。当这样一个 `simp` 日后失效时，这能确保我们知道原本意图的目标是什么。

有几种在证明中途使用 `simp` 的“被认可”方式：

1) `simp only [h1, h2, ..., hn]` 将 `simp` 约束为只使用给定列表中的引理，因而它不受 `simp` 引理集合变化的影响。提示：用 `simp?` 来自动生成一个合适的 `simp only`。

2) 使用形如 `have h : P := by ...; simp` 的构造来引入一个由 `simp` 证明的假设。`have` 表达式可能处于证明中途，但其中的 `simp` 关闭的是它所引入的目标。

3) 如果 `simp` 将你的目标变为 `P`，那么你可以写

```
suffices : P by simpa
```

这会在当前目标之后添加一个新目标 `P`，引入一个新假设 `this : P`，同时化简目标和 `this`，然后试图用 `this` 关闭目标。`simpa` 策略 要求目标被关闭，这与 `simp` 不同，从而更容易知道它何时失效。源代码中显式写出的 `P` 有助于找到修复方法。

非终结性 `simp` 可能出现的一种情形，是在形如 `simp at ⊢ h; exact h` 的策略序列中。这些可以用 `simpa using h` 来替代。

dsimp

dsimp 是 simp 的一个变体，它只使用“定义性的”simp 引理。这些 simp 引理的证明是 rfl，也就是说，其两端依定义相等的引理。

与 simp 一样，建议你不要在证明中途使用它。不过，如果 dsimp 将你的目标变为 h，那么 change h 很可能会做同样的事。dsimp 的另一种常见用法是

```
dsimp only
```

它是 dsimp only [] 的简写，即一个带空 simp 引理集合的 dsimp。这可以安全地在证明中途使用，并且它可以是整理目标的一种有用方式：除其他作用外，它会对 lambda 表达式做 beta 归约（它会把 (fun x => f x) 37 变成 f 37），并且会归约结构投影（它会把 { toFun := f, ... }.toFun 变成 f）。

更高级的特性

析消器（Discharger）

Lean 有以下定理：

```
theorem Nat.max_eq_left {a b : ℕ} (h : b ≤ a) : max a b = a
```

然而，仅凭此定理，simp 无法证明以下目标。

```
example : max (1 : ℕ) 0 = 1 := by
  simp only [Nat.max_eq_left]
-- simp made no progress
```

这是因为 simp 未能解决附带条件 (0 : ℕ) ≤ 1;这可以通过命令 set_option trace.Meta.Tactic.simp.dischar
true 看到。

```
[Meta.Tactic.simp.discharge] @Nat.max_eq_left discharge □
      0 ≤ 1
```

这个附带条件可以简单地用 decide 解决：

```
example : (0 : ℕ) ≤ 1 := by
  decide
```

如何在 simp 中使用这个策略来解决附带条件呢？答案如下：

```
example : max (1 : ℕ) 0 = 1 := by
  simp (disch := decide) only [Nat.max_eq_left]
```

完整语法

以下是 dsimp 策略的完整语法：

> dsimp (?)? (!)? ((config := config))? ((disch := discharger))? (only)? ([引理列表])? (at locations)?

其中“(…)?”表示表达式中可选的部分。引理列表与 rw 的类似，但此外 -lemma_name 表示某条引理被排除在 simp 引理集合之外。配置选项将在后续小节中描述。

如果存在 !，它会向配置选项添加 autoUnfold := true。如果存在 ?，它会使 simp 建议一组足以完成任务的 simp 引理。

以下是 simp 策略的完整语法：

> simp (?)? (!)? ((config := config))? ((disch := discharger))? (only)? ([由 * 和引理构成的列表])? (at locations)?

以下是 simpa 策略的完整语法：

> simpa (?)? (!)? ((config := config))? ((disch := discharger))? (only)? ([由 * 和引理构成的列表])? (using expr)?

其含义与 simp 相同，但 using 可以接受任意表达式，而不仅限于 at 所要求的局部常量。

自定义 simp 属性

使用命令 register_simp_attr，你可以创建自己的类似 @[simp] 的属性，但有一个关键区别：被 @[new_attr] 标注的引理 不在默认的 simp 引理集合中。相反，它们应当被显式纳入：simp [new_attr]。这往往可以替代冗长的 simp only [...] 调用，并使代码更易读。一些常见用法的例子包括 enat_to_nat_top 和 coassoc_simps。

配置选项

simp 和 dsimp 都可以接受额外的配置选项。例如，simp +singlePass 在将 singlePass 配置选项设为 true 的情况下运行 simp；simp -singlePass 则会显式地将该选项设为 false。可以用 singlePass 来避免某些原本可能发生的循环。对于取值的选项，你可以使用具名参数语法，如 simp (maxSteps := 37)。这会将失败前所允许的最大步数设为 37。

Lean 核心文件 Init/MetaTypes.lean 在 Lean.Meta.DSimp.Config 和 Lean.Meta.Simp.Config 结构中揭示了其他配置选项。它们中的大多数对普通用户而言并不十分相关。下表复现了这些选项，其中某个配置选项对 simp 或 dsimp 的默认值分别给在相应的列中——若没有给出默认值，则表示该选项不可用。默认值“max”指 Lean.Meta.Simp.defaultMaxSteps，目前为 100000。

选项	simp	dsimp	描述
`maxSteps`	max		失败前所允许的最大步数
`maxDischargeDepth`	2		对附带条件递归应用化简时的最大递归深度
`contextual`	false		基于当前子表达式的上下文使用额外的 `simp` 引理（见下面的示例）
`memoize`	true		对子项的化简进行缓存
`singlePass`	false		每个子项至多访问一次
`zeta`	true	true	进行 zeta 归约：`let x := a; b` ⌧ `b[x := a]`
`beta`	true	true	进行 beta 归约：`(fun x => a) y` ⌧ `a[x := y]`
`eta`	true	true	允许 eta 等价：`(fun x => f x)` ⌧ `f`（目前尚未实现）
`etaStruct`	`.all`	`.all`	配置如何判定两个结构实例之间的定义相等性。参见 `Lean.Meta.EtaStructMode` 的文档
`iota`	true	true	归约递归子：`Nat.recOn (succ n) Z R` ⌧ `R n (Nat.recOn n Z R)`
`proj`	true	true	归约投影：`Prod.fst (a, b)` ⌧ `a`
`decide`	false	false	通过推断 `Decidable p` 实例并将其归约，把命题 `p` 重写为 `True` 或 `False`
`arith`	false		化简简单的算术表达式
`autoUnfold`	false	false	使用方程编译器生成的所有方程引理进行归约

选项	simp	dsimp	描述
dsimp	true		当为 true 时，若不存在能让 simp 访问依值参数的同余定理，则对这些参数切换为 dsimp。当 dsimp 为 false 时，则不访问该参数。
failIfUnchanged	true	true	若未应用任何化简则失败
ground	false		归约基项（ground term）。当一个项不含自由变量或元变量时，它是基项。
unfoldPartialApp	false	false	当我们请求展开 f 时，即使是 f 的部分应用也予以展开
zetaDelta	false	false	展开局部定义。即，给定包含条目 x : t := e 的局部上下文，自由变量 x 归约为 e。

autoUnfold 将方程/模式匹配编译器生成的方程引理添加到 simp 引理集合中。

contextual 选项赋予 simp 这样的能力：基于某个子表达式所处的上下文，将假设视为额外的 simp 引理。例如，当它化简某个蕴含式的后件时，会临时把前件添加为一条 simp 引理。下面这个示例就需要用到这一点：

```
example {x y : ℕ} : x = 0 → y = 0 → x = y := by
  simp +contextual
```

如何加速一个 Mathlib 文件

我们将说明如何让一个运行缓慢的 Mathlib 文件变得更快。这些笔记基于 Mathlib PR #12412 中进行的实验。

1. 第一步是找出代码中哪些部分较慢。为此，在 `import` 语句之后添加一行 `set_option profiler true`。这将在 infoview 中产生形如 `<blah> took <number>ms`（甚至 `<number>s`）的行，记录那些耗时至少 `100ms` 的步骤（这一以毫秒为单位的下界可通过 `set_option profiler.threshold <num>` 进行调整）。
2. 对于每一行这样的输出，尝试按照下面的说明加速相应的步骤。
3. 上一步可以在降低 `profiler.threshold` 设置后重复进行，以便发现并加速那些不太慢、但也不太快的部分。然而，这最终会带来递减的收益。
4. 再次移除 profiler 选项，然后提交 PR！

处理特定的缓慢步骤

这里我们说明如何尝试加速代码中导致 profiler 在 infoview 中产生消息的各个部分。

`typeclass inference of <name> took <a long time>`

1. 在引发该消息的声明之前紧接着添加 `set_option trace.Meta.synthInstance true in`。
2. 查看 infoview 中生成的实例综合（instance synthesis）追踪，找出 `<name>` 中较慢的那个（些）实例。
3. 在该声明之前使用 `#synth <name> <args>`（如有需要，可临时向上下文中添加 `variable`），以获得一个提供该实例的合适的项。
4. 在该声明之前（或在当前 section/namespace 的开头附近）添加一行 `@[local instance] lemma/def <some name> <possibly some args> : <name> <args> := <term>`。如果该实例需要证明内部的某些局部上下文，则改为在证明中合适的位置添加 `have/let <some name> <possibly some args> : <name> <args> := <term>`。
5. 移除该声明之前的 `set_option` 行。

有可能 `<term>` 又会触发一次缓慢的实例搜索，因此这一过程可能需要重复进行。

取舍： 在文件中到处散布局部实例并不美观，而且在某种程度上有违类型类系统的初衷。

当然，一个更好的解决方案是找出究竟是什么导致了所发现情形中类型类搜索变慢，然后为之找到修复方法。这很可能会使 Mathlib 中许多其他文件同样受益。

`simp took <a long time>`

将相关的 `simp/simpa` 调用替换为 `simp?/simpa?`，并点击 `Try this:` 建议，以将其替换为 `simp/simpa only` 调用。在某些情况下，还可以在一定程度上精简引理列表。

取舍： 证明可能会增长好几行密集的内容，并且现在会按名称提及许多引理，其中每一个都可能在将来被重命名，从而破坏你的证明。

`elaboration took <a long time>`

查找引发该消息的声明中的 `_`，弄清它们是由什么填充的，并将它们替换为相应的显式参数。

取舍： 如果显式参数很长，这会使陈述变得更长，并且可能更难阅读。

`compilation of <name> took <a long time>`

尝试在定义之前添加 `noncomputable`。

取舍： 定义将不再是内核可归约的（kernel-reducible），但在大多数情况下这应该不成问题。

`tactic execution of <tactic> took <a long time>`

尝试将缓慢的策略替换为对更简单策略的调用。

- 例如，一个缓慢的 `nontriviality ... using ...` 可以替换为

```
rcases subsingleton_or_nontrivial ... with H | H
· -- get `Subsingleton` case out of the way
  ...
-- now we have `Nontrivial ...`
```

- 一个缓慢的 `convert` 可以这样避免：先完成在它之后所做的那些重写，然后再使用 `refine` 或 `exact`。

取舍： 证明可能会变得稍长一些，也更繁琐一些。

方程编译器与 WellFoundedRelation

要递归地定义函数和证明，你可以使用方程编译器，前提是该类型上存在一个良基关系。

例如，自然数上 gcd 的定义就使用了良基递归

```
def gcd (m n : Nat) : Nat :=
  if m = 0 then
    n
  else
    gcd (n % m) m
  termination_by m
  decreasing_by simp_wf; apply mod_lt _ (zero_lt_of_ne_zero _); assumption
```

由于 < 是自然数上的良基关系，且 `¬m = 0 → n % m < m`，这个递归函数是良基的。

每当你使用方程编译器时，被递归的类型上都会存在一个默认的良基关系（由 `WellFoundedRelation` 实例给出），方程编译器将自动尝试证明该函数在此关系下是良基的。

当方程编译器失败时，主要有两种原因。

1. 它找不到一个递减的度量。
2. 它未能证明所需的不等式。

证明所需的不等式

如果我们修改上面的 gcd 示例，去掉 `termination_by` 和 `decreasing_by`，就会得到一个错误。

```
def gcd (m n : Nat) : Nat :=
  if m = 0 then
    n
  else
    gcd (n % m) m
```

```
fail to show termination for
  Nat.gcd
with errors
argument #1 was not used for structural recursion
  failed to eliminate recursive application
    (n % m).gcd m

argument #2 was not used for structural recursion
  failed to eliminate recursive application
    (n % m).gcd m

structural recursion cannot be used

Could not find a decreasing measure.
The arguments relate at each recursive call as follows:
(<, ≤, =: relation proved, ? all proofs failed, _: no proof attempted)
           m n
1) 93:4-18 ? ?
Please use `termination_by` to specify a decreasing measure.
```

错误信息提示我们使用 termination_by 来指定递减的度量，于是我们用 m 作为递减度量，但又得到了另一个错误。

```
def gcd (m n : Nat) : Nat :=
  if m = 0 then
    n
  else
    gcd (n % m) m
  termination_by m

failed to prove termination, possible solutions:
  - Use `have`-expressions to prove the remaining goals
  - Use `termination_by` to specify a different well-founded relation
  - Use `decreasing_by` to specify your own tactic for discharging this kind of goal
m n : ℕ
h□ : ¬m = 0
⊢ n % m < m
```

我们指定了正确的递减度量，因此我们的选择是使用 have 表达式或使用 decreasing_by。这里，我们使用 decreasing_by tactics 来证明该函数的可终止性。首先，我们用 sorry 作占位符以查看目标。

```
def gcd (m n : Nat) : Nat :=
  if m = 0 then
    n
  else
    gcd (n % m) m
  termination_by m
  decreasing_by sorry
```

```
m n : ℕ
h□ : ¬m = 0
⊢ (invImage (fun x ↦ PSigma.casesOn x fun m n ↦ m) instWellFoundedRelationOfSizeOf).1 ⟨n
```

这个目标是编译器生成的，因此难以阅读，所以我们用 `simp_wf` 来化简目标。

```
def gcd (m n : Nat) : Nat :=
  if m = 0 then
    n
  else
    gcd (n % m) m
  termination_by m
  decreasing_by simp_wf; sorry
```

```
m n : ℕ
h□ : ¬m = 0
⊢ n % m < m
```

顺便一提，假设 `h□ : ¬m = 0` 是从哪里来的? 实际上，在证明可终止性的过程中，`if p then t else f` 被重写为 `if h : p then t else f`，因此假设 `p` 就变得可用了。

剩下的任务只是证明这个目标:

```
def gcd (m n : Nat) : Nat :=
  if m = 0 then
    n
  else
    gcd (n % m) m
  termination_by m
  decreasing_by simp_wf; apply mod_lt _ (zero_lt_of_ne_zero _); assumption
```

供你参考，如果我们选择使用 `have` 表达式，函数将呈现如下形式:

```
def gcd (m n : Nat) : Nat :=
  if h : m = 0 then
    n
```

```
  else
    have := mod_lt n (zero_lt_of_ne_zero h)
    gcd (n % m) m
```

但当我们使用方程引理时，have 表达式可能会成为障碍。

另一个例子是 Data.Multiset.Basic 中的如下函数。

```
def strongInductionOn {p : Multiset α → Sort*} (s : Multiset α) (ih : ∀ s, (∀ t < s, p t)
    p s :=
    (ih s) fun t _h =>
      strongInductionOn t ih
termination_by card s
decreasing_by exact card_lt_card _h
```

这个例子使用 card s 而非参数本身作为递减度量。

WellFoundedRelation 实例

我们不仅可以指定 Nat 值，还可以指定任何带有 WellFoundedRelation 的类型作为递减度量，正如你在 Data.List.Defs 中所见：

```
def permutationsAux.rec {C : List α → List α → Sort v} (H0 : ∀ is, C [] is)
    (H1 : ∀ t ts is, C ts (t :: is) → C is [] → C (t :: ts) is) : ∀ l₁ l₂, C l₁ l₂
  | [], is => H0 is
  | t :: ts, is =>
      H1 t ts is (permutationsAux.rec H0 H1 ts (t :: is)) (permutationsAux.rec H0 H1 is []
  termination_by ts is => (length ts + length is, length ts)
  decreasing_by all_goals (simp_wf; omega)
```

这个例子使用 Nat × Nat 值作为递减度量。该类型上的 WellFoundedRelation 是自然数上的字典序，由 Init.WF 中的如下实例定义：

```
instance [ha : WellFoundedRelation α] [hb : WellFoundedRelation β] : WellFoundedRelation (
  lex ha hb
```

我们目前正在更新 Lean 社区网站，以介绍如何使用 Lean 4 进行工作，但你今天在这里看到的大部分信息仍然是关于 Lean 3 的。

警告：Lean 3 与 Lean 4 互不兼容。本页面中的所有示例在 Lean 4 中都无法运行。关于编写 Lean 4 策略的参考资料，例如可参见 Metaprogramming in Lean 4 以及 The Lean Language Reference。

非常欢迎提交更新本页面以适配 Lean 4 的拉取请求。本页面底部有相关链接。

在这一过渡时期，请访问 leanprover zulip，寻求你所需要的任何帮助！

Lean 3 的网站已被归档。如果你需要链接到 Lean 3 特有的资源，请链接到那里。

教程：在 Lean 3 中编写策略

本页面是关于 Lean 3 的：请在继续之前阅读上方的提示横幅。

这是一篇帮助你开始在 Lean 中编写自己的策略的入门教程。它面向的读者群体不一定有在函数式编程语言中使用单子（monad）的经验（例如大多数数学家）。

学习编写策略时其他有用的资源包括：* Rob Lewis 在 Lean for the Curious Mathematician 2020 上关于 Lean 元编程的 视频教程 * Hitchhiker’s Guide to Logical Verification 的第 7 章 * 关于 Lean 元编程的原始论文 A Metaprogramming Framework for Formal Verification

单子学（Monadology）

策略是作用于证明状态的程序。但 Lean 是一门函数式编程语言。这意味着你所能做的一切就是定义并求值函数。每个函数接受具有预定义类型的输入，并给出具有预定义类型的输出。这似乎妨碍了拥有全局状态（例如当前的假设和目标），也妨碍了输出类型依赖于输入值（例如一个策略可以成功或失败），或输出消息。这些问题通过三层技巧来解决（以下简短的描述有望在后文中变得更加清晰）：* 使用复杂的类型，将证明状态和策略运行状态携带传递 * 使用巧妙的记号，隐藏掉大部分复杂的类型管理与组合 * 使用交互式策略块，由 `by` 引入或由 `begin`/`end` 界定

前两点统称为单子式编程（monadic programming，当然有更精确的定义，但我们会尽量忽略它）。

一个能够检视证明状态、修改它，并可能返回某个类型为 `α` 的值（或失败）的函数，其类型称为 `tactic α`。特别地，`tactic unit` 只关乎操作证明状态，而不尝试返回任何东西（从技术上讲它会返回某个类型为 `unit` 的值，`unit` 是恰好只有一个项的类型，记作 `()`）。这样的函数要么被其他策略调用——这些通常位于 `tactic` 命名空间中——要么被用户在策略块内交互式地调用——这些必须位于 `tactic.interactive` 命名空间中（对于非常简单的策略这并非完全必要，但一般而言忽略此规则会发生奇怪的事情）。一个有时很方便的捷径是：可以使用 `run_cmd add_interactive [`my_tac1,`my_tac2, `my_tac3]` 将名为 `my_tac1`、`my_tac2`、`my_tac3` 的定义复制到 `tactic.interactive` 命名空间中。这些函数将用于生成 Lean 证明，但我们既不会对这些函数本身证明任何东西，常量 `my_tac1`、`my_tac2` 等也不会出现在它们所生成的证明中。通过在它们前面加上关键字 `meta`，我们告诉 Lean 它们仅用于“求值目的”，这会禁用非 `meta` 声明必须通过的某些检查。有了这些知识，就足以编写第一个策略了。

```
meta def my_first_tactic : tactic unit := tactic.trace "Hello, World."

example : true :=
begin
  my_first_tactic,
  trivial
end
```

在该示例中，my_first_tactic 以绿色下划线标出（在 VS Code 中），将光标移到那一行上会在 Lean 消息缓冲区中显示我们的消息。

接下来我们需要学习如何串联多个动作。归根结底，这完全是关于组合函数的，但单子记号将其隐藏，并模拟命令式编程。我们需要使用 and_then 组合子。第一种方式是使用中缀记号 >>，如下：

```
meta def my_second_tactic : tactic unit :=
tactic.trace "Hello," >> tactic.trace "World."
```

现在这会分两段打印我们的消息。或者，可以使用 do 语法，它还有其他好处。它引入了一个以逗号分隔的指令列表，按顺序依次执行。

```
meta def my_second_tactic' : tactic unit :=
do
  tactic.trace "Hello,",
  tactic.trace "World."
```

除了显示消息之外，策略接下来能做的事情是失败，并可能附带一些说明。

```
meta def my_failing_tactic  : tactic unit := tactic.failed

meta def my_failing_tactic' : tactic unit :=
tactic.fail "This tactic failed, we apologize for the inconvenience."
```

在串联指令时，第一次失败会中断整个过程。然而 orelse 组合子（以中缀 <|> 表示）允许在其左侧失败时尝试其右侧。下面的策略将成功地传达它的消息。

```
meta def my_orelse_tactic : tactic unit :=
my_failing_tactic <|> my_first_tactic
```

接下来要做的组合操作是使用某个函数，它在读取或改变证明状态之后，实际尝试返回某个值。例如内置的 tactic.target（尝试）返回当前目标。这个目标的类型是 expr（关于该类型后文会详述）。因此 tactic.target 的类型是 tactic expr。假设我们想追踪当前目标。一个朴素的尝试是：

```
meta def broken_trace_goal : tactic unit :=
tactic.trace tactic.target    -- WRONG!
```

这不可能正确，因为 tactic.target 可能失败（可能不再有目标），而 tactic.trace 不能把这种失败作为输入。我们需要绑定（bind）组合子，其中缀记号为 >>=，它在成功时把左侧的输出传给右侧，否则失败。

```
meta def trace_goal : tactic unit :=
 tactic.target >>= tactic.trace
```

或者，特别是当 tactic.target 的输出可能被多次使用时，可以在 do 块中使用 ← 进行赋值（与重写语法中的箭头相同）。当然，如果赋值右侧失败，这种对命令式变量赋值的模拟也会失败（就像上面那些失败的策略一样）。

```
meta def trace_goal' : tactic unit :=
do
 goal ← tactic.target,
 tactic.trace goal
```

请注意，这种赋值只是尝试从某个类型为 tactic α 的东西中提取类型为 α 的数据。它不能用于存储普通的东西。下面这段代码无法工作。

```
meta def broken_assignment : tactic unit :=
do
 message ← "Hello, World.",  -- WRONG!
 tactic.trace message
```

不过，可以在 do 块中使用 let，如下：

```
meta def let_example : tactic unit :=
do
 let message := "Hello, World.",
 tactic.trace message
```

接下来，我们想编写返回某个值的策略，正如 tactic.target 所做的那样。唯一额外的要素是 return 函数。下面的函数在没有更多目标时尝试返回 tt，否则返回 ff。接下来的那个可以交互式使用，并追踪其结果（注意，交互式地使用第一个函数不会产生任何可见效果，因为交互式使用会忽略返回值）。

```
meta def is_done : tactic bool :=
(tactic.target >> return ff) <|> return tt


meta def trace_is_done : tactic unit :=
is_done >>= tactic.trace
```

关于单子赋值，我们还需要了解的最后一件事是模式匹配赋值。下面的策略尝试将表达式 l 和 r 定义为当前目标的左侧和右侧。它还使用了 to_string 函数，该函数与 trace 结合使用对于调试策略非常方便，并且适用于任何作为 has_to_string 实例的类型。

```
meta def trace_goal_is_eq : tactic unit :=
do t ← tactic.target,
   match t with
   | `(%%l = %%r) := tactic.trace $ "Goal is equality between " ++ (to_string l) ++ " and
   | _ := tactic.trace "Goal is not an equality"
   end
```

Lean 还为带有单个模式和一个通配符的模式匹配提供了专门的语法，其中只有在模式匹配成功时执行才会继续到 do 块的下一行，否则执行 | 之后的表达式：

```
meta def trace_goal_is_eq : tactic unit :=
do `(%%l = %%r) ← tactic.target | tactic.trace "Goal is not an equality",
   tactic.trace $ "Goal is equality between " ++ (to_string l) ++ " and " ++ (to_string r)
```

如果省略 |，那么当模式不匹配时该策略会失败。我们可以使用前面提到的 orelse 组合子来捕获这种失败，但请注意，这样做会比前面捕获更多类型的失败：

```
meta def trace_goal_is_eq : tactic unit :=
(do  `(%%l = %%r) ← tactic.target,
     tactic.trace $ "Goal is equality between " ++ (to_string l) ++ " and " ++ (to_string
     some_other_tactic)
   <|> tactic.trace "Goal is not an equality, or `some_other_tactic` failed"
```

上面代码中的圆括号看起来不太美观。可以改用花括号，它允许界定一个 do 块，如下：

```
meta def trace_goal_is_eq : tactic unit :=
do { `(%%l = %%r) ← tactic.target,
     tactic.trace $ "Goal is equality between " ++ (to_string l) ++ " and " ++ (to_string
   <|> tactic.trace "Goal is not an equality"
```

第一个实际可用的策略

我们已经学习了足够的单子学，可以理解我们的第一个有用的策略了：assumption 策略，它在局部上下文中搜索一个能够关闭当前目标的假设。它还使用了另外几个内置策略，它们都在核心库的 init/meta/tactic.lean 中声明并有简要文档说明，但实际上是用 C++ 实现的。首先 `infer_type : expr → tactic expr` 尝试确定一个表达式的类型（由于它返回一个 `tactic expr`，它必须如上所述与 >>= 或 ← 串联起来）。其次是 `tactic.unify`，它在忽略几个可选参数的情况下，接受两个表达式，当且仅当它们在定义上相等时成功。assumption 策略的第一部分是一个辅助函数，它在一个表达式列表中搜索与某个表达式 e 共享类型的表达式，并返回第一个匹配项（如果没有匹配项则失败）。

```
meta def find_matching_type (e : expr) : list expr → tactic expr
| []        := tactic.failed
```

```
| (H :: Hs)  := do t ← tactic.infer_type H,
                   (tactic.unify e t >> return H) <|> find_matching_type Hs
```

请利用前一节的内容，确保你真正理解上面代码中的控制流。其基本模式是经典的列表递归查找。注意表达式 `e` 位于冒号左侧，因此它会原封不动地传递给递归调用 `find_matching_type Hs`。名称 `H` 取自 `hypothesis`（假设），而 `Hs` 遵循 Haskell 的命名约定，表示多个假设。对于非空列表所发生的事情，其命令式类比可以写成如下命令式伪代码

```
if unify(e, infer_type(H)) then return H else find_matching_type(e, HS)
```

现在我们可以把这个函数用于我们的交互式策略。我们首先需要使用 `local_context` 来获取局部上下文，它返回一个表达式列表，我们可以将其传给 `find_matching_type`。如果那个函数成功，它的输出会被传给内置策略 `tactic.exact`。这里我们需要使用完全限定名，因为可能会与 `exact` 的交互式版本混淆（后者接受不同的参数，因此它并不是非交互式版本的精确副本）。这是一个很好的机会来指出：本教程开头为清晰起见到处都使用了完全限定名，但当然实际工作流程中应该打开 `tactic` 命名空间。

```
meta def my_assumption : tactic unit :=
do { ctx ← tactic.local_context,
     t   ← tactic.target,
     find_matching_type t ctx >>= tactic.exact }
<|> tactic.fail "my_assumption tactic failed"
```

附加问题：如果我们去掉花括号会怎样？它是否仍然能通过类型检查？如果能，得到的策略是否相同？

单子式循环

命令式编程的一个关键工具是循环，因此单子必须模拟这一点。我们已经从通常的 Lean 中知道 `list.map` 和 `list.foldr`/`list.foldl` 允许对列表元素进行循环。但我们需要能与单子世界良好交互的版本（消费并返回类型为 `tactic stuff` 的项）。这些版本以“m”（表示 monad）作为前缀，例如 `list.mmap`、`list.mfoldr` 等。于是我们的策略是：

```
meta def list_types : tactic unit :=
do
  l ← tactic.local_context,
  l.mmap (λ h, tactic.infer_type h >>= tactic.trace),
  return ()
```

最后一行有点傻：它之所以存在，是因为我们从上一行得到的东西类型是 `list unit`，所以它不能是 do 块的最后一部分。因此我们加上 `return ()`，其中 `()` 是类型 `unit` 唯一的项。也可以使用策略 `skip` 来达到同样的目的。这种特殊情况如此常见，以至于我们实际上有一个 `list.mmap` 的变体 `list.mmap'`，它丢弃应用于列表元素的函数的结果，并在遍历完列表后返回 `()`。

操作局部上下文

我们接下来的目标是能够在局部上下文中使用和创建假设。我们将编写一个策略，通过把两个已知的等式相加来产生一个新的假设（如果这个操作没有意义则惨败）。起初这两个等式的名称会被愚蠢地硬编码在我们的策略中。所以我们想要一个策略来执行下面证明中的第一行。

```
example (a b j k : ℤ) (h₁ : a = b) (h₂ : j = k) :
  a + j = b + k :=
begin
  have := congr (congr_arg has_add.add h₁) h₂,
  exact this
end
```

我们需要的第一个新概念是名称（name）。为了支持命名空间管理，Lean 中的名称实际上被定义为一个归纳类型，在核心库 meta/name.lean 中。直接操作它的构造子并不方便，所以我们改用反引号记号（这是策略编写中众多反引号用法中的第一个）。实际上我们在最开始讨论 `add_interactive` 命令时就已经这样做过了。通过名称访问局部上下文中的某一项是通过 `tactic.get_local` 完成的。我们需要的下一个新部件是 `tactic.interactive.«have»`，它将创建我们新的上下文项。它那古怪的名称是为了绕开 `have` 是关键字、因而不是合法名称这一事实。它接受两个可选参数，我们现在先忽略它们，以及一个预表达式（pre-expression），它是我们新项的证明。这样的预表达式使用双反引号加圆括号记号来构造：``(...)。‘(...) ```. Inside such a construction, previously assigned expressions can be inserted 使用反引用前缀%%‘来访问。这种语法与我们上面看到的模式匹配语法非常相近（但不同）。

``(...). Inside such a construction, previously assigned expressions are accessed using the anti-quotation prefix %%. This syntax is very close to the pattern matching syntax we saw above (but different).

关于上述策略的最后一点说明：名称 `` `h₁ `` 和 `` `h₂ `` 是在
该策略执行时解析的。为了在解析该策略时触发名称解析，
应使用双反引号，如 ``` ``h₁ ```。当然在
上面的上下文中，这会触发一个错误，因为在策略解析时
没有任何名为 `h₁` 的东西。但它在其他情况下可能有用。

策略参数的解析

解析标识符

显然，如果假设名称是硬编码的，前面的策略就没什么用。所以我们将它替换为：

A last remark about the above tactic: the names `h₁ and `h₂ are resolved when the tactic is executed. In order to trigger name resolution when the tactic is parsed, one should use

double-backtick, as in ``` ``h₁ ```. Of course in the above context, that would trigger an error since nothing named `h₁` is in sight at tactic parsing time. But it can be useful in other cases.

Tactic arguments parsing

Parsing identifiers

Obviously the previous tactic is useless if the assumption names are hardwired. So we replace it by:

```
参数 `h1` 和 `h2` 告诉 Lean 去解析标识符。这里
发生了相当多的把戏。Lean 解析器看到冒号左侧的 `parse`，
所以它知道必须做一些参数解析，但随后得到的类型
不过是一个名称，如下所示。
```

The arguments `h1` and `h2` tell lean to parse identifiers. There is quite a bit of trickery going one here. The Lean parser sees `parse` left of colon, so it knows it must do some argument parsing, but then the resulting type is nothing but a name, as demonstrated below.

```
### 解析可选参数并使用记号（token）

对该策略的下一项改进提供了一个机会来给新的
局部假设命名（它当前命名为 `this`）。这样的名称
传统上由记号 `with` 引入，后跟所需的标识符。
这个“后跟”是用 `seq_right` 组合子表达的（这里又
潜伏着一个单子），其记号为 `*>`。解析一个记号是由
`lean.parser.tk` 后跟一个字符串引入的，该字符串必须取自一个
预先确定的列表（这个列表的初始值可以在
Lean 源代码中找到，位于 [frontends/lean/token_table.cpp](https://github.com/leanprover-community/lean/blob/master/src/frontends/lean/token_table.cpp)，
当字面量被用于 `notation`、`infix` 或 `precedence` 时会向此列表添加元素）。
然后整个组合被包裹进 `optional` 以使其成为可选的。我们在
下面得到的项 `h` 于是具有类型 `option name`，可以作为
`«have»` 的第一个参数传入，`«have»` 会在提供时使用它，
否则使用名称 `this`。
```

Parsing optional arguments and using tokens

The next improvement to this tactic offers the opportunity to name the new local assumption (which is currently named `this`). Such names are traditionally introduced by the token `with`, followed by the desired identifier. The "followed by" is expressed by the `seq_right` combinator (there is again a monad lurking here), with notation `*>`. Parsing a token is introduced by `lean.parser.tk` followed by a string which must be taken from a predetermined list (the initial value of this list can be found in Lean source code, in frontends/lean/token_table.cpp, elements are added to this list when literals are used in `notation`, `infix`, or `precedence`). And then the combination is wrapped into `optional` to make it optional. The term `h` we get below has then type `option name` and can be passed as the first argument of `«have»`, which will use it if provided, and otherwise use the name `this`.

解析位置（location）和表达式

我们的下一个策略将一个等式从左侧乘以一个给定的表达式
（如果这个操作没有意义则失败）。我们想把下面的证明机械化。

Parsing locations and expressions

Our next tactic multiplies from the left an equality by a given expression (or fails if this operation wouldn't make sense). We want to mechanize the following proof.

这里主要的新技能在于使用传统记号 `at` 来指明
我们想要作用的位置，以及向策略传递一个表达式。
位置在核心库 [meta/interactive_base.lean](https://github.com/leanprover-community/lean/blob/master/library/init/meta/interactive_base.lean) 中被定义为
一个具有两个构造子的归纳类型：`wildcard` 表示所有
位置，以及 `loc.ns`，它接受一个 `list (option name)`，其中 `option name` 中的
`none` 表示当前目标，而 `some n` 表示局部上下文中
名为 `n` 的东西。在我们的情形中，我们将对
解析出的位置进行模式匹配，并拒绝除了从局部上下文中指定单个名称之外的
一切。第二个新部件是如何解析用户提供的
表达式。相关的解析器是 `interactive.types.texpr`，其结果通过
`tactic.i_to_expr` 转换为实际的表达式。这也是
我们第一次认真使用模式匹配赋值的机会，
以及使用 `«have»` 的第二个可选参数（即期望类型）的机会
（否则我们会得到带有显式 lambda 的、未应用的乘法，试试看！）。

The main new skills here consist in indicating at what location we want to act, using the traditional token `at`, and passing an expression to the tactic. Locations are defined in the core library meta/interactive_base.lean as an inductive type having two constructors: `wildcard` which indicates all locations, and `loc.ns` which takes a `list (option name)`, where `none` in the `option name` means the current goal, whereas `some n` means the thing named `n` in the local context. In our case we will pattern-match on the parsed location and reject everything except specifying a single name from the local context. The second new piece is how to parse a user-provided expression. The relevant parser is `interactive.types.texpr`, whose result is converted to an actual expression using `tactic.i_to_expr`. This is also the opportunity for our first serious use of pattern matching assignment, and for using the second optional argument of «have» which is the expected type (otherwise we would get unapplied multiplication, with an explicit lambda, try it!).

作为最后一项改进，让我们做一个该策略的版本，它通过追加 `.mul` 来给相乘后的等式命名，并在策略名称后跟 `!` 时可选地删除原始等式。这是使用 `when` 的机会，它是 `ite` 的单子版本（else 分支什么也不做）。关于这一想法的其他变体，参见核心库中的 [control/combinators.lean](https://github.com/leanprover-community/lean/blob/master/library/init/control/combinators.lean)。

As a last refinement, let us make a version of this tactic which names the multiplied equality by appending `.mul`, and optionally removes the original one if the tactic name is followed by `!`. This is the opportunity to use `when` which is the monadic version of `ite` (with else branch doing nothing). See control/combinators.lean in core library for other variations on this idea.

现在该读些什么？

这就是本教程的结尾了（不过下面还有两份速查表）。如果你想了解更多，可以阅读核心库或 mathlib 中策略的定义，看看你能理解多少，并在 Zulip 上提出具体的问题。如需更多理论，特别是对单子的正确解释，你可以阅读 [Programming in Lean](https://lean-lang.org/programming_in_lean/)，但其中实际编写策略的部分已经过时。论文 [A Metaprogramming Framework for Formal Verification](https://lean-lang.org/papers/tactic.pdf)。

Mario 的反引号速查表

本节是 Mario 在 Zulip 上发的消息的直接汇编。

* `` `my.name `` 是引用一个名称的方式。它本质上是一种字符串引用形式；除了
  把点解析成带命名空间的名称之外，不做任何检查。

* ``` ``some ``` 在解析时进行名称解析，所以这个例子展开为 `` `option.some ``，
  如果给定的名称不存在则会报错。
* `` `(my expr) `` 在解析时构造一个表达式，在（该策略的）当前
  命名空间中尽可能地进行解析。
* ``` ``(my pexpr) ``` 在解析时构造一个预表达式，在（该策略的）当前
  命名空间中进行解析。
* ```` ```(my pexpr) ```` 构造一个 `pexpr`，但将解析推迟到（该策略的）运行时，
  这意味着任何引用都会在用户的 `begin` `end` 块的命名空间中解析，
  而不是在策略本身的命名空间中解析。
* `%%`：这称为反引用（anti-quotation），在所有 expr 和 pexpr 引用
  表达式 `` `(expr) ``、``` ``(pexpr) ```、```` ```(pexpr) ```` 以及 `` `[tacs] `` 中都受支持。
  在这些引用构造中任何期望表达式的地方，你都可以改用 `%%e`，
  其中 `e` 在策略的外层上下文中具有类型 `expr`，它会被拼接
  进所构造的 `expr`/`pexpr` 等之中。例如，如果 `a b : expr`，那么 `` `(%%a + %%b) `` 的
  类型是 `expr`。
* 如果 Lean 能推断出一个实例 `reflected t`，则 `reflect` 函数会把一个项 `t : T` 变成
  一个反射 `t` 的 `expr`。例如，这可以用于在引用内部
  使用 `%%(reflect n)` 来引用策略定义中的局部变量。举个例子，我们可以写

  ```lean
  meta def assert_ge_zero (n : ℕ) : tactic unit :=
  do v ← to_expr ``(nat.zero_le %%(reflect n)),
     t ← infer_type v,
     assertv `h t v,
     skip
  ```

  如果你在这里直接写 `n`，就会得到 “unexpected local in quotation expression”
  错误。
* `` `[tac...] `` 与 `begin tac... end` 完全相同，意思是它使用交互模式解析器
  解析 `tac...`，但与其求值该策略以产生一个项，不如说它只是
  把策略列表包裹成一个类型为 `tactic unit` 的单一策略。这对于编写
  “宏”或轻量级的策略编写很有用。

同样值得一提的是 `expr` 模式匹配，它的语法与
`` `(%%a + %%b) `` 相同。这些可以用在 match 的模式位置，或用在 do 记号中
`←` 的左侧，它们会解构一个表达式并
绑定被反引用的变量。

例如，如果 `e` 是一个表达式，那么 `` do `(%%a = %%b) ← return e, ... `` 会检查 `e` 是否是一个等式，并把左侧和右侧绑定到 `a` 和 `b`（类型为 `expr`），如果它不是等式，该策略就会失败。

（值得注意的是，这种模式匹配是在语法层面工作的。有时
使用合一（unification）会更灵活。）

Mario 的单子符号速查表

下面列表中的所有函数和记号都适用于比 `tactic` 更一般的单子，因此
它们以通用形式列出，但就本教程而言，
`m` 始终是 `tactic`（或 `lean.parser`）。尽管
一切都可以用本教程中介绍的符号完成，但更
深奥的符号可以压缩代码，理解它们对于
阅读现有的策略很有用。

* `return`：在单子中产生一个值（类型：`A → m A`）
* `ma >>= f`：从 `ma : m A` 中取出类型为 `A` 的值并把它传给 `f : A → m B`。替代语法：`do a ← ma, f a`
* `f <$> ma`：把函数 `f : A → B` 应用于 `ma : m A` 中的值，得到一个 `m B`。等同于 `do a ← ma, return (f a)`
* `ma >> mb`：等同于 `do a ← ma, mb`；这里 `ma` 的返回值被忽略，然后调用 `mb`。替代语法：`do ma, mb`
* `mf <*> ma`：等同于 `do f ← mf, f <$> ma`，或 `do f ← mf, a ← ma, return (f a)`
* `ma <* mb`：等同于 `do a ← ma, mb, return a`
* `ma *> mb`：等同于 `do ma, mb`，或 `ma >> mb`。为什么同一件事有两种记号？历史原因。
* `pure`：等同于 `return`。同样是历史原因。
* `failure`：失败值（具体的单子通常有更有用的形式，例如策略的 `fail` 和 `failed`）。
* `ma <|> ma'` 从失败中恢复：运行 `ma`，如果它失败则运行 `ma'`。
* `a $> mb`：等同于 `do mb, return a`
* `ma <$ b`：等同于 `do ma, return b`

<!-- source: templates/extras/pitfalls.md -->

常见的 Lean 陷阱

本文档列举了 Lean 用户经常犯的一些错误，以及 Lean 中可能导致错误的一些反直觉特性。

如果你在本文档中发现了错误，或者希望补充某些内容，请在 Zulip 上提出，或者在[本网站的代码仓
community/leanprover-community.github.io)中提交 issue 或拉取请求。

目录

- [自动隐式参数](#automatic-implicit-parameters)
- [忘记 Mathlib 缓存](#forgetting-the-mathlib-cache)
- [用 `have` 来表示数据](#using-have-for-data)
- [在绑定子下进行改写](#rewriting-under-binders)
- [指望策略展开定义](#trusting-tactics-to-unfold-definitions)
- [使用 `b > a` 而非 `a < b`](#using-b-gt-a-instead-of-a-lt-b)
- [混淆 `Prop` 与 `Bool`](#confusing-prop-and-bool)
- [未检查相异性](#not-checking-for-distinctness)
- [未考虑 0 的情形](#not-accounting-for-0)
- [除以 0](#division-by-0)
- [整数除法](#integer-division)
- [自然数减法](#natural-number-subtraction)
- [其他偏函数](#other-partial-functions)
- [`Fin` 中算术的回绕](#wrapping-arithmetic-in-fin)
- [实数幂](#real-power)
- [`Fin n → ℝ` 中的距离](#distance-in-fin-n--%E2%84%9D)
- [意外的双重 `iInf` 或 `iSup`](#accidental-double-iinf-or-isup)
- [试图从命题的证明中提取数据](#trying-to-extract-data-from-proofs-of-propositions)
- [处理类型的相等](#working-with-equality-of-types)
- [为已存在的实例引入参数](#parameters-for-instances-that-already-exist)
- [把 `Set` 当作类型使用](#using-sets-as-types)
- [Sort _](#sort-_)
- [试图证明关于 Float 的性质](#trying-to-prove-properties-about-float)
- [`native_decide`](#native_decide)
- [Panic 不会中止程序](#panic-does-not-abort)
- [Lean 3 代码](#lean-3-code)
- [非终结性 simp](#non-terminal-simp)
- [忽略警告](#ignoring-warnings)
- [易混淆的 Unicode 字符](#ambiguous-unicode-characters)
- [结构字段中的默认值](#default-values-in-structure-fields)

自动隐式参数

默认情况下，Lean 的[自动隐式参数](https://lean-lang.org/doc/reference/latest///Definitions
and-Signatures/#automatic-implicit-parameters)特性（简称 `autoImplicit`）会将未绑定的变量

```lean
theorem my_theorem : a + 1 = 1 + a := by omega
```

是

```
theorem my_theorem {a : Nat} : a + 1 = 1 + a := by omega
```

的简写。

这个特性可以使你的代码更加简洁，但它也意味着定理陈述中的任何笔误都可能使该定理陈述变得错误。例如，下面的陈述

```
theorem my_theorem (n : Nat) (h : 1 ≤ n) : 0 < m := sorry
```

是错误的，因为 `m` 是一个自动隐式参数。自动隐式特性使得人们很难察觉到本该输入 `n` 时却输入了 `m`。在较新的 Lean 版本中，你应该能在文本编辑器中 `my_theorem` 旁边看到一个内联的 `{m}`，这表明 `m` 正充当一个自动隐式参数；留意这些标注是值得的，因为它们能帮助你发现无意中使用了 `autoImplicit` 的情况。

问题也可能以难以预料的方式表现出来。例如，如果你尚未导入 `Mathlib.Data.Nat.Notation`，那么在

```
theorem my_theorem : ∃ (a b : ℕ), a ≠ b := sorry
```

中，`ℕ` 实际上是一个自动隐式变量，可以是任何类型，因此这个陈述实际上是错误的：

```
example : False := Exists.elim (my_theorem (ℕ := False)) (fun f _ ↦ f)
```

自动隐式参数默认在全局启用，但如果你在 Mathlib 上工作，它们会被禁用。如果你是 Lean 新手，我建议你禁用它们，转而使用 `variable` 命令。这可以通过在每个 Lean 文件顶部 `import` 语句之后添加

```
set_option autoImplicit false
```

来实现，或者在你的 `lakefile.toml` 或 `lakefile.lean` 中全局禁用它。

忘记 Mathlib 缓存

如果你正在 `Mathlib` 上工作，或者在一个依赖于 `Mathlib` 的项目上工作，那么首次打开一个依赖于 `Mathlib` 的文件，或者在终端中运行 `lake build`，可能需要一个多小时才能完成。当 Lean 文件首次被编译时，Lean 会将编译结果缓存到 `.olean` 文件中，从而使后续运行更快。

在终端中运行 `lake exe cache get`，或者在 VSCode 中调用“Project: Fetch Mathlib Build Cache”，会从中央服务器下载 `Mathlib` 的 `.olean` 文件，从而使你无需自行编译它们，可以节省一个多小时的计算时间。（如果 `lake exe cache get` 不起作用，你可能想试试 `lake exe cache get!`。）如果你发现你的项目在 VSCode 中编译或启动花费了异常长的时间，可能是因为你忘记运行这个命令了。

请注意，解压后编译版本的 Mathlib 超过 5 GB，因此在下载缓存之前，请确保你有足够的存储空间，并且连接到了合适的网络（而不是例如有每月流量限制的手机热点）。

用 have 来表示数据

有时在证明中引入一个新变量来表示一个较为复杂的表达式会很方便。例如，假设你有一个复杂的谓词 `MyPredicate`，并希望在证明中设 `x := 37 * n + 42 * m + 76`。如果你用 `have` 来做这件事，然后证明 `h : MyPredicate x`，你会发现没有办法用 `h` 来完成目标。

```
example (n m : Nat) : MyPredicate (37 * n + 42 * m + 76) := by
  have x : Nat := 37 * n + 42 * m + 76
  have h : MyPredicate x := ...
  /- Tactic state is now:
  n m x : Nat
  h : MyPredicate x
  ⊢ MyPredicate (37 * n + 42 * m + 76)
  Now what? -/
```

此外，如果你的证明依赖于 `x` 的具体取值，你可能根本无法证明 `MyPredicate x`。

这里的问题在于，当你用 `have` 在证明中创建一个新变量时，Lean 会忘记你赋给 `have` 的值。因此，在执行 `have x : Nat := 37 * n + 42 * m + 76` 之后，命题 `x = 37 * n + 42 * m + 76` 现在就无法证明了。

解决办法是，每当你引入一个并非证明的新变量时，使用 `let` 而非 `have`。注意在下面的策略状态中，`x` 是如何附带一个值的。

```
example (n m : Nat) : MyPredicate (37 * n + 42 * m + 76) := by
  let x : Nat := 37 * n + 42 * m + 76
  have h : MyPredicate x := ...
  /- Tactic state is now:
  n m : Nat
  x : Nat := 37 * n + 42 * m + 76
  h : MyPredicate x
  ⊢ MyPredicate (37 * n + 42 * m + 76)
  -/
```

这意味着在证明的其余部分中，`x` 在定义上等于 `37 * n + 42 * m + 76`。因此，`h` 的类型在定义上等于目标，所以我们可以用 `exact h` 完成证明，而无需进行任何手动转换。如果你确实想要一个 `x = 37 * n + 42 * m + 76` 的证明，你可以用 `have hx : x = 37 * n + 42 * m + 76 := rfl` 来获得它，其中 `rfl` 之所以是这一事实的有效证明，正是由于定义上的相等。

你可能还会对 set 策略感兴趣，它类似于 let，但还会自动替换证明状态中该表达式的所有出现之处。

在绑定子下进行改写

人们有理由期望 rw 在下面的例子中把 0 + i ^ 2 改写为 i ^ 2，但不幸的是 rw 失败了：

```
import Mathlib.Algebra.BigOperators.Group.Finset.Defs

example (s : Finset Nat) : ∑ i ∈ s, (0 + i ^ 2) = ∑ i ∈ s, i ^ 2 := by
  rw [Nat.zero_add]
  /-
  tactic 'rewrite' failed, did not find instance of the pattern in the target expression
    0 + ?n
  s : Finset ℕ
  ⊢ ∑ i ∈ s, (0 + i ^ 2) = ∑ i ∈ s, i ^ 2
  -/
```

这是因为 ∑ i ∈ s, (0 + i ^ 2) 是 Finset.sum s (fun i ↦ 0 + i ^ 2) 的简写，而 rw 有时无法在 fun 表达式内部进行改写，因为它无法改写像 0 + i ^ 2 这样含有绑定变量的子表达式。

在这种情况下，你可以使用 simp_rw [Nat.zero_add] 来执行改写，但在某些情况下，你可能需要使用转换模式进行精确的改写：

```
example (s : Finset Nat) : ∑ i ∈ s, (0 + i ^ 2) = ∑ i ∈ s, i ^ 2 := by
  conv =>
    lhs
    congr
    · rfl
    · intro i
      rw [Nat.zero_add]
```

指望策略展开定义

Lean 中一个常见的错误是试图使用诸如 rw 和 simp 之类的策略，期望它们能“看穿”def 和 let 语句。例如，在下面的例子中，你可能希望 rw 能意识到 x 只不过是 0 + n 的简写，并将其改写为 n，但这并不会发生：

```
theorem mythm (n : Nat) : True := by
  let x := 0 + n
```

```
  have : x = n := by
    rw [Nat.zero_add]
    /-
    tactic 'rewrite' failed, did not find instance of the pattern in the target expressior
      0 + ?n
    n : Nat
    x : Nat := 0 + n
    ⊢ x = n
    -/
```

类似地，`simp` 在这里也会失败，并给出错误信息“simp made no progress”。

这里的解决办法是，先 `unfold x` 或使用 `change 0 + n = n`，然后再调用 `rw [Nat.zero_add]` 或 `simp`。你也可以用 `simp [x]` 代替 `simp`。最后，如果 `x` 是一个 `def` 而非 `let`，你可以执行 `rw [x]`，这等价于 `rw [show x = 0 + n from rfl]`。请注意，对 `x` 使用 `let` 而非 `have` 是很重要的；参见上文关于用 `have` 来表示数据的章节。

话虽如此，理解为什么这从一开始就是个问题是值得的。回想一下，Lean 中主要有三种相等的概念：命题相等、定义相等和句法相等。命题相等是传统数学中通常意义上的相等概念，Lean 中的命题 `a = b` 指的就是命题相等。相比之下，定义相等和句法相等不是你能够证明或否证的东西；两个表达式要么在定义上相等，要么不相等，它们要么在句法上相等，要么不相等。

一种粗略的思考方式是：如果两个项在 Lean 中以相同的方式书写，那么它们就是句法相等的；如果对两个项调用 `#reduce` 会得到相同的表达式，那么它们就是定义相等的。例如，当 `n` 是一个未知的自然数时：- `n + 0` 与 `n + 0` 既句法相等，又定义相等，也命题相等。- `n + 0` 与 `n` 定义相等且命题相等，但不句法相等。这是因为 `Nat.add` 是按第二个参数递归定义的，所以即便 Lean 不知道 `n` 是什么，它也能把 `n + 0` 归约为 `n`。- `0 + n` 与 `n` 命题相等，但既不定义相等也不句法相等。这是因为当 `n` 未知时，Lean 不知道如何归约 `0 + n`，所以这个陈述必须用归纳法来证明。

更多信息,请参见 https://b-mehta.github.io/formalising-mathematics-notes/Part_1/equality.html

不幸的是，在 Lean 中，句法相等与定义相等之间的界限往往是模糊的。需要记住的重要一点是，某些策略（例如 `exact`）在定义相等的意义下工作，而另一些策略（例如 `rw` 和 `simp`）则在句法相等的意义下工作。尽管 Lean 的核心类型检查器只关心定义相等，但策略可以自由地利用关于项的句法的额外信息来辅助其运作。

使用 `b > a` 而非 `a < b`

大多数情况下，`a < b` 与 `b > a` 在 Lean 中是定义相等的。但鉴于上面关于不要指望策略展开定义的讨论，即便两个项在定义上相等，它们仍然可能并非在所有情形下都可以互换。例如，下面这个改写会失败，因为 `Nat.mod_eq_iff_lt` 的右端是 `m < n`，而非 `n > m`：

```
example {n m : Nat} (h₁ : m % n = m) (h₂ : n ≠ 0) : n > m := by
  rw [←Nat.mod_eq_iff_lt h₂]
```

在 Mathlib 中，`a < b` 优先于 `b > a`。事实上，`>` 在 Mathlib 中几乎从不出现。这意味着，如果你想避免一直使用 `gt_iff_lt`，你应该在代码中处处优先使用 `a < b` 而非 `b > a`。类似地，你应该优先使用 `a ≤ b` 而非 `b ≥ a`。除了像 `∀ ε > 0, ∃ δ > 0, _` 这样 `>` 与 `∀` 或 `∃` 相连的表达式之外，通常最好完全不使用 `>` 或 `≥`。

混淆 Prop 与 Bool

类型 `Prop` 和 `Bool` 都刻画了某物为真或为假的概念，但它们在 Lean 中的行为大相径庭。`Prop` 是命题的全集，命题是具有真值的数学陈述，而 `Bool` 是布尔值的类型；它恰好由两个元素 `true` 和 `false` 组成。请注意，`True` 和 `False` 是命题，而 `true` 和 `false` 是布尔值。

在期望命题的地方使用布尔值，或反之，可能导致一些反直觉的错误。此外，由于布尔值可以被隐式强制转换为命题，由此产生的问题可能并不会立即显现。例如，`a = b` 是一个命题，而 `a == b` 是一个布尔值（参见 `BEq` 类型类），混淆它们会引发问题。确保你充分掌握了 `Prop` 与 `Bool` 之间的区别是值得的。

在 Lean 中，`Prop` 和 `Bool` 都是类型，但 `Prop` 同时也是一个全集，而 `Bool` 只是一个包含两个元素的普通类型。这意味着，如果 `p : Prop` 且 `q : Bool`，那么 `h : p` 可能是合法的，但 `h : q` 永远不合法。另一方面，你可以在 `match` 语句中使用 `Bool` 类型的项，但你不能对 `Prop` 类型的项进行 `match`（在 `Prop` 上的等价操作是 `by_cases hp : p` 策略，或者“依值”的 if-then-else 语句 `if hp : p then ... else ...`）。`Bool` 在数学中很少使用，但当 Lean 被用作编程语言时，它的使用要频繁得多。

直观地说，命题是一个可能被证明或被否证的数学陈述。如果 `p` 是一个命题，那么说 `h : p` 是有意义的，它意味着 `h` 是 `p` 的一个证明。另一方面，如果 `q` 是一个布尔值，那么 `q` 实实在在地等于 `true` 或 `false` 之一；“证明”`q` 是没有意义的，因为 `q` 是一个值，而非一个陈述。

如果你有一个布尔值 `q`，你可以通过写 `q = true` 将它转换为一个命题。反之，如果 `p` 是一个命题，并且 Lean 能合成出 `Decidable p` 的一个实例，那么你可以通过写 `decide p` 将 `p` 转换为一个布尔值。（这里，项 `decide` 不应与策略 `decide` 相混淆。）由于公理 `Classical.choice` 可以为任意命题 `p` 产生一个 `Decidable p` 实例，因此任何命题都可以被转换为布尔值，从而 `Prop` 与 `Bool` 在经典意义下是等价的。

尽管如此，在 `Prop` 与 `Bool` 之间保持区分仍然是有意义的。在 mathlib 中，`Prop` 与 `Bool` 在类型类上可能被赋予不同的实例；例如，`Prop` 被赋予 Sierpiński 空间（以 `{True}` 为开集）作为其拓扑，而 `Bool` 被赋予离散拓扑。

未检查相异性

考虑下面这个鸽笼原理的陈述，它说的是：如果 $f : A \to B$ 是有限集之间的一个函数且 $|A| > |B|$，那么在 A 中存在两个相异的元素被 f 映射到 B 中同一个元素：

```
import Mathlib.Data.Fintype.Card

variable {A B : Type*} [Fintype A] [Fintype B]

theorem pigeonhole_principle (f : A → B) (h : Fintype.card B < Fintype.card A) : ∃ (x y :
  let a : A := Classical.choice <| Fintype.card_pos_iff.mp (by omega)
  exact ⟨a, a, rfl⟩
```

如果你看一下这个证明，你会注意到它太过简单了。注意，该定理的陈述忘记要求 `x` 与 `y` 相异，所以我们要做的只是证明存在某个元素 `a : α`，然后根据定义就有 `f(a) = f(a)`。这并非 Lean 特有的问题，但在 Lean 中它比在非形式化数学中更容易在日后引发麻烦。

未考虑 0 的情形

考虑下面这个费马大定理的陈述：

```
theorem flt (a b c n : Nat) (h : 3 ≤ n) : a ^ n + b ^ n ≠ c ^ n := sorry
```

不幸的是，这个陈述是错误的：

```
example : False := flt 0 0 0 3 le_rfl rfl
```

记住，在 Lean 中，`0 : ℕ`，你应该在你的定理陈述中考虑到这一情形。（这也是一个使用 Mathlib 定义本可以提供帮助的例子：Mathlib 已经为费马大定理的陈述提供了一个定义 `FermatLastTheorem`，它包含错误的可能性要小得多。）

除以 0

在 Lean 中，`n / 0 = 0`。这与其他编程语言不同，在那些语言中除以零通常会导致异常。

这意味着像下面这样的陈述

```
import Mathlib.Data.Real.Basic

theorem my_theorem : ∃! (x : ℝ), x / (1 + x) = 0 := sorry
```

实际上是错误的，因为在 Lean 中，方程 `x / (1 + x) = 0` 在实数上有两个解：

```
example : False := by
  suffices h : (0 : ℝ) = -1 by aesop
  apply my_theorem.unique <;> simp
```

除法之所以这样表现，是因为 Lean 是一门纯函数式语言，所以所有函数都必须是全函数——函数不能抛出异常。在 Lean 中，处理偏函数 $f : A \rightharpoonup B$ 的推荐方式是改为定义一个函数 $g : A \to B$，使得只要 $x \in \mathrm{dom}(f)$，就有 $g(x) = f(x)$。g 在 $A \setminus \mathrm{dom}(f)$ 上的取值是任意的，通常被选取为能给出最良好的代数性质的值（这些值常被称为“垃圾值”）。

也可以改为像这样定义除法

```
def div (a b : ℝ) (h : b ≠ 0) : ℝ := ...
```

但在实践中，每次想要对两个数做除法时都提供分母非零的证明会很快变得繁琐。相比之下，涉及除法的定理通常会带有一个表明分母非零的假设。建议你为自己的定理添加确保分母非零的假设，除非你已经仔细考虑过分母为零的情形。更多信息，请参见这篇 Xena 项目博文。

整数除法

在 Lean 中对两个 `Int` 或 `Nat` 做除法时，结果总是向下取整（这与大多数语言略有不同，那些语言是向零取整的）。这是为了使用除法不会导致所处理的数的类型发生改变。例如，下面的陈述是错误的：

```
import Mathlib.Algebra.Field.Rat
import Mathlib.Algebra.Order.Ring.Rat

def thm₁ (n : ℤ) (h : 2 < n) : (1 / n) < (1 / 2) := by
  /-
  ====================
  Found a counter-example!
  n := 3
  guard: 2 < 3
  issue: 0 < 0 does not hold
  (0 shrinks)
  --------------------
  -/
  plausible
```

但与此同时，你可能并不想依赖这种行为。例如，下面这个定理实际上是成立的：

```
def thm₂ (n : ℚ) (h : 2 < n) : (1 / n) < (1 / 2) := by
  simp [inv_strictAntiₒ rfl h]
```

在这个例子中，< 的左端是一个有理数，所以 Lean 将右端也解释为有理数，于是整数除法就不再适用了。这与下面发生的情况类似：

```
#eval (1 / 2) + 0 -- 0
#eval (1 / 2) + (0 : ℚ) -- (1 : Rat)/2
```

如果你习惯了像 Java 这样的语言，你可能会对此感到惊讶——在 Java 中 `(1 / 2) + 0.0` 求值为 `0.0`，因为在 Java 中括号外的类型信息通常不会影响括号内的运算。

一般来说，判断整数除法是否存在是很困难的。此外，Lean 在最近这个例子中的行为在未来也有可能改变。

如果你对此有所顾虑，你可能想考虑在文件顶部附近添加 `set_option pp.numericTypes true`，这样数值字面量在 infoview 中显示时就会被标注上其相应的类型（例如 ℕ、ℤ、ℚ、ℝ、ℂ）。

自然数减法

自然数的减法在 `0` 处截断，因为我们希望自然数减法的返回类型是 `Nat`。也就是说，我们有

```
#eval 5 - 3 -- 2
#eval 5 - 7 -- 0
#eval 10 - 100 + 100 -- 100
#eval 5 - 13 + 11 + 0 -- 11
```

正如上面整数除法一节中所解释的，类型标注会影响所执行的运算：

```
#eval 10 - 100 + (100 : Int) -- 10
#eval 5 - 13 + 11 + (-0) -- 3
```

截断减法也被称为 monus。

自然数减法可能出现的一个场合是：如果你有一个谓词 `P : Nat → Prop`，并想证明对所有 `n ≥ 1` 都有 `P n` 成立。如果你试图证明

```
theorem mythm {n : Nat} (hn : 1 ≤ n) : P n := ...
```

你可能会发现 `mythm` 的证明需要你频繁地用到 `n - 1`。尽管这里并未发生截断，但这个陈述证明起来仍可能很繁琐，因为自动化往往不能很好地处理自然数减法。在许多情况下，改为证明

```
theorem mythm (n : Nat) : P (n + 1) := ...
```

是更明智的，因为这完全避免了自然数减法。更一般地，你常常可以把一个涉及自然数减法的陈述重述为一个不涉及它的陈述。此外，在很多情况下，mythm 的后一种表述比前一种表述更易于使用。

其他偏函数

你还应该意识到, Mathlib 中许多其他函数也使用了垃圾值。例如, - 当 f 在 s 内于 x 处不可微时, deriv f s x = 0。类似地，当 f 在给定点处不可微时，Fréchet 导数 fderiv 返回零线性映射。你可能需要在定理的假设中加入 HasDerivAt 或 HasFDerivAt。- “拓扑和”tsum f，也记作 ∑' i, f i，计算一个级数的和，或者在该和不收敛时返回 0。考虑为涉及 ∑' i, f i 的定理加入 Summable f 假设。- 注意，由于 tsum 不假设定义域上有序结构，只有无条件收敛的级数才被视为 Summable。直观地说，如果级数求和的次序无关紧要，那么它就是无条件收敛的。例如，交错调和级数不是 Summable 的，因为它只是条件收敛的。（由 Riemann 级数定理可知，一个实值级数无条件收敛当且仅当它绝对收敛。一般而言，无条件收敛比绝对收敛更弱：如果 $(e_n)_{n\in\mathbb{N}}$ 是一个可数无穷维 Hilbert 空间的一组标准正交基，那么 $\sum(e_n)/n$ 无条件收敛，但不绝对收敛。）- tprod f（记作 ∏' i, f i）与 tsum 类似，但其垃圾值是 1 而非 0。- Nat.sqrt x 取 $\sqrt{x}$ 的下取整。- Real.sqrt x 对负数输入返回 0。- Real.log x 在 $x \neq 0$ 时实际上表示 $\log_e |x|$，并在 $x = 0$ 时为 0。这比将其对所有负的 x 都设为 0 给出了更好的代数性质。- 当集合为空或无上界时，Real.sSup 和 Real.iSup 为 0；同样地，当集合为空或无下界时，Real.sInf 和 Real.iInf 为 0。这与 Real.sqrt 配合得很好：它意味着对所有 $x \in \mathbb{R}$ 都有 $\sqrt{x} = \sup\{y \mid y^2 < x\}$，因为当 $x \leq 0$ 时两端都为 0。

Fin 中算术的回绕

Fin 中的算术在上溢或下溢时会回绕:

```
#eval (7 : Fin 10) + 6 -- 3
#eval (7 : Fin 10) * 6 -- 2
#eval (2 : Fin 10) - 8 -- 4
```

这对字面量也适用:

```
#eval (15 : Fin 10) -- 5
#eval (10 : Fin 10) -- 0
```

请注意，尽管对于正的 n，Fin n 使用与 ZMod n 相同的加法、减法和乘法概念，但它们并不完全相同，因为 Fin n 是有序的而 ZMod n 是无序的，并且 Fin n 使用截断的整数除法，而 ZMod n 在 n 为素数时使用 $\mathbb{Z}/n\mathbb{Z}$ 上“数学上正确”的除法概念。此外，Fin 0 是空的，而 ZMod 0 = Int（这是为了使 ZMod n 始终是一个特征为 n 的环，因为空类型不能成为环）。

Fin 之所以使用回绕算术而非诸如饱和算术之类的东西，原因之一是它被用来表示像 UInt32 这样的原生定宽整数类型，而 Fin 上的运算需要与机器原生算术的上溢和下溢相一致。

实数幂

`Real.rpow x y` 对负的 `x` 的定义有些任意。具体来说，它被定义为复指数函数的实部，而复指数函数本身也有些任意，因为它依赖于复对数。这赋予了该函数良好的解析性质，但也意味着对负数取根会有些反直觉。例如，`(-125 : ℝ) ^ (1/3 : ℝ)` 的值是 $5\cos(\pi/3)$，而非你可能预期的 -5。

`Fin n → ℝ` 中的距离

在许多情况下，`Fin n → ℝ` 是在 Lean 中表示 $\mathbb{R}^n$ 的推荐方式。这种类型的向量可以用 `![x,y,z,...]` 记法书写。但请注意，在 Lean 中，如果 $(S_i)_{i\in I}$ 是一族有限多个（伪）度量空间，那么 $\prod_{i\in I} S_i$ 使用 L^∞ 度量：

$$\operatorname{dist}(\mathbf{x},\mathbf{y}) = \sup\{\operatorname{dist}(x_i,y_i) \mid i\in I\}$$

而 `Fin n → ℝ` 是这种情形的一个特例。

这意味着 $(1,0)$ 与 $(0,1)$ 之间的距离实际上是 1：

```
import Mathlib.Analysis.InnerProductSpace.PiL2

example : dist ![(1 : ℝ),0] ![0,1] = 1 := by
  rw [dist_pi_eq_iff (by positivity)]
  constructor
  · use 0
    simp
  · intro b
    fin_cases b <;> simp
```

要改用标准的欧几里得度量（也称为 L^2 度量），你必须使用 `EuclideanSpace ℝ (Fin n)`，它是 `WithLp 2 (Fin n → ℝ)` 的缩写。这种类型的向量可以用 `!₂[x,y,z,...]` 记法书写。

```
example : dist !₂[(1 : ℝ),0] !₂[0,1] = √2 := by
  norm_num [EuclideanSpace.dist_eq]
```

这样做的一个后果是，`Fin n → ℝ` 并未被注册为内积空间，因为与该内积相对应的范数会与已有的 L^∞ 范数相冲突。如果你需要把 $\mathbb{R}^n$ 作为内积空间，请使用 `EuclideanSpace ℝ (Fin n)`。

意外的双重 `iInf` 或 `iSup`

你可能期望 `⨅ x > 2, (x : ℝ) ^ 2` 等于 `4`，因为 `(2,∞)` 在 `fun x : ℝ ↦ x ^ 2` 下的像是 `(4,∞)`。但在 Lean 中，这个表达式实际上等于 `0`：

```
import Mathlib.Data.Real.Archimedean
import Mathlib.Order.ConditionallyCompleteLattice.Indexed

-- The infimum of x^2 on (2,∞) is ... 0?
example : ⨅ x > 2, (x : ℝ) ^ 2 = 0 := by
  set f := fun x : ℝ ↦ ⨅ (h : x > 2), x ^ 2
  have hf₁ (x : ℝ) : f x = if _ : x > 2 then x ^ 2 else sInf ∅ := ciInf_eq_ite
  have hf₂ : f 0 = 0 := by simp [hf₁]
  have hf₃ (x : ℝ) : 0 ≤ f x := by
    rw [hf₁]
    split_ifs <;> simp [sq_nonneg]
  apply le_antisymm
  · rw [←hf₂]
    refine ciInf_le ?_ 0
    rw [bddBelow_iff_exists_le 0]
    use 0
    simp [hf₃]
  · simp [le_ciInf, hf₃]
```

其原因在于，`⨅ x > 2, (x : ℝ) ^ 2` 是 `⨅ (x : ℝ) (h : x > 2), x ^ 2` 的简写，而后者又是 `⨅ (x : ℝ), ⨅ (h : x > 2), x ^ 2` 的简写。注意：- 当 `x > 2` 时，`⨅ (h : x > 2), x ^ 2` 表示 `⨅ (h : True), x ^ 2`，它等于 `x ^ 2`。- 当 `x ≤ 2` 时，`⨅ (h : x > 2), x ^ 2` 表示 `⨅ (h : False), x ^ 2`，它等于 `sInf (∅ : Set ℝ)`。在传统数学中，实数上的 $\inf \emptyset$ 通常是无定义的，或者被定义为等于 $+\infty$。但在 Lean 中，`sInf (∅ : Set ℝ) = 0`（解释参见上面关于偏函数的章节）。

所以，总结来说，`⨅ x > 2, (x : ℝ) ^ 2` 等于 `⨅ (x : ℝ), f x`，其中

$$f(x) = \begin{cases} x^2 & \text{for } x > 2 \\ 0 & \text{for } x \leq 2 \end{cases}$$

而这个函数的下确界是 `0`。

集合也会出现类似的情况。`⨅ x ∈ s, f x` 是 `⨅ x, ⨅ (_ : x ∈ s), f x` 的简写，它可能与 `⨅ x : ↑s, f ↑x`（其中索引类型是被强制转换为类型的 `s`）不同。此外，本节中的一切对 `iSup` 的适用程度与对 `iInf` 的一样。

`⨅` 和 `⨆` 之所以这样表现，是为了与诸如 `∀` 和 `∃` 之类的其他绑定子保持一致：`∀ x ∈ s, p x` 是 `∀ x, ∀ h : x ∈ s, p x` 的简写，`∃ x ∈ s, p x` 是 `∃ x, ∃ h : x ∈ s, p x` 的简写。然而，关于未来是否可能改变这种行为，已经有过一些讨论：参见 Zulip 上的这则讨论。

试图从命题的证明中提取数据

给定一个证明 `h : ∃ n : Nat, p n`，你可能希望提取出使得 `p n` 成立的那个特定的 `n`。或者给定一个项 `q : Nonempty Nat`，你可能希望获得用于构造 `q` 的那个特定的 `Nat`。在上述每种情况下，从 `h` 或 `q` 中提取一个任意的 `n : Nat` 都是可能的，但获得 `h` 或 `p` 的构造中所用的那个特定的 `n` 则是不可能的。

要理解其原因，首先回想一下 Lean 有一个单一的全集层级 `Sort 0`、`Sort 1`、`Sort 2`、`Sort 3`、……，并且 `Prop` 是 `Sort 0` 的简写，`Type` 是 `Sort 1` 的简写，`Type 1` 是 `Sort 2` 的简写，如此等等。之所以要区分 `Prop` 和 `Type u`，是因为 `Prop` 具有一些特殊行为，有时需要我们把它与其余的全集区别对待。`Prop` 最重要的两个特殊性质是非直谓性和证明无关性。在本节中，我将主要讨论证明无关性的后果。

本质上，证明无关性表明每个命题至多有一个证明；也就是说，同一命题的任意两个证明都相等。另一种表述方式是，每个命题都是一个子单元集——一个具有零个或一个元素的类型。证明无关性在 Lean 中的实现方式是：如果 `P : Prop` 且 `p : P` 且 `q : P`，那么 `p` 与 `q` 在定义上相等，因此 `rfl : p = q`。这是 Lean 类型论中的一条内置规则，不应与 `propext` 相混淆，后者是一条公理，表明如果 `P Q : Prop` 且 `P ↔ Q`，那么 `P = Q`。

要明白为什么需要这条规则，回想一下子类型 `{x : X // p x}` 的一个元素形如 `{ val := x, property := h }`，其中 `x` 是 `X` 的一个元素，`h` 是 `p x` 的一个证明。如果我们定义

```
def x : {n : Nat // 4 < n} := { val := 8, property := Nat.lt_of_sub_eq_succ rfl }
def y : {n : Nat // 4 < n} := { val := 8, property := by omega }
```

那么 `x` 与 `y` 具有相同的值，但具有不同的证明。我们希望 `x` 与 `y` 尽管证明不同却仍然相等，而且如果它们甚至定义相等的话就非常方便了，正是证明无关性使这成为可能：

```
example : x = y := rfl
```

由于证明无关性，当一个证明 `h : ∃ n : Nat, p n` 被构造出来时，它“遗忘”了用来证明它的是哪个 `n`。如果存在某个函数 `f : (∃ n : Nat, p n) → Nat` 能提取出 `h` 的构造中所用的自然数，那么我们就可以构造两个不同的证明 `h₁ h₂ : ∃ n : Nat, p n`，使得 `f h₁ ≠ f h₂`，但由证明无关性又有 `h₁ = h₂`。这会产生矛盾，所以不存在这样的 `f`。

更一般地，当你定义一个属于 `Prop` 的归纳类型 `T` 时，`T.rec` 的返回类型只被允许属于 `Prop`。实际上，这意味着虽然函数

```
def swap {P Q : Prop} (h : P ∨ Q) : Q ∨ P :=
  match h with
  | Or.inl hp => Or.inr hp
  | Or.inr hq => Or.inl hq
```

是被允许的，因为该 match 表达式的返回类型属于 `Prop`，但函数

```
def bad {P Q : Prop} (h : P ∨ Q) : Nat :=
  match h with
```

```
  | Or.inl hp => 1
  | Or.inr hq => 2
```

则是不被允许的，因为该 match 表达式的返回类型属于 Type。请注意，某些归纳命题——称为句法子单元集——通过一种称为子单元集消去的机制而豁免于这一限制。

如果你确实必须从命题中提取数据，有几种方法可供选择：- 如果你是在某个命题的证明过程中提取数据（而不是在一个属于 Type 或更高层级的 def 中），那么诸如 cases 和 obtain 之类的策略会像往常一样工作。你也可以应用诸如 Exists.elim 和 Nonempty.elim 之类的消去子，它们只允许你消去到命题。- 如果你只是需要从一个证明 h : Nonempty α 中提取一个任意的 α，你可以使用公理 Classical.choice 来做到这一点。请注意，除了 Classical.choice 的输出属于 α 这一事实之外，关于该输出无法证明任何性质。例如，如果你对一个证明 h : Nonempty Nat 使用 Classical.choice 以产生一个元素 n : Nat，那么诸如 n = 37 之类的陈述在 Lean 中既不可证也不可否。此外，如果你编写的代码的行为依赖于 n 的值，那么你将被迫把这样的代码标记为 noncomputable。- 如果你拥有的是一个元素 h : ∃ n : Nat, p n，那么你可以使用 Classical.choose 来获得一个元素 n : Nat，并使用 Classical.choose_spec 来获得 p n 的一个证明。Classical.choose 和 Classical.choose_spec 在内部是通过使用 Classical.choice 产生 {n : Nat // p n} 的一个元素来定义的，所以关于可计算性的相同限制也适用。- 如果你需要依赖于 ∃ n : Nat, p n 的证明中所用的那个特定的值，你或许应该完全避免使用 Exists 类型，而是给出 {n : Nat // p n} 的一个元素。∃ n : Nat, p n 与 {n : Nat // p n} 都由一个 n : Nat 和一个 p n 的证明组成；它们唯一的区别在于该类型属于哪个全集。- 如果你是在 Nat 上工作，并且你有一个 h : ∃ n : Nat, p n，其中 p 是一个 DecidablePred，那么你可以使用 Nat.find 和 Nat.find_spec，其用法类似于 Classical.choose 和 Classical.choose_spec。Nat.find 是可计算的，尽管它可能并不特别高效，因为求值 Nat.find h 会从 0 开始按顺序检查每一个自然数，直到找到满足 p 的最小者。

处理类型的相等

类型的相等在 Lean 的类型论中表现得很糟糕。如果 A 与 B 是 Type u 中两个不同的归纳类型或结构，并且 A 与 B 具有相同的基数，那么陈述 A = B 在 Lean 中既不可证也不可否。例如，Int = Nat 既不可证也不可否。更多信息，请阅读 Jason Rute 和 Andrej Bauer 对这个问题的回答：https://proofassistants.stackexchange.com/q/4046

粗略地说，你唯一能够证明两个类型相等的情形，是它们由相同的不可约类型定义而来。例如，当 n = m 时 Fin n = Fin m 是可证的，但 Fin 2 = Bool 则不可证。你唯一能够证明两个类型不相等的情形，是它们具有不同的基数。所以，Fin 3 ≠ Bool 是可证的，但 Fin 2 ≠ Bool 则不可证。例外是 Prop，对它而言，公理 propext 可以被理解为表明：如果两个 Prop 具有相同的基数，那么它们相等。

如果你确实能够证明 A = B，那么你可以使用 cast 把 A 的一个元素转换为 B 的一个元素。但对 cast 的依赖往往会在日后引发问题，并可能导致表现糟糕的 HEq。通常，与其处理类型的相等，处理某种合适的同构类型（如果你不涉及任何代数或拓扑结构，则用 Equiv）要好得多。另外，如果你处理的是 α

的若干 Subtype 之间的相等，你或许应该改为处理 Set α 的元素。Set α 中的相等表现良好。

为已存在的实例引入参数

一个常见的错误是为已经存在的实例引入 variable 或参数。例如，在下面的定理陈述中，注意 [Ring ℤ] 是多余的，因为 ℤ 已经有一个可用的 Ring 实例。

```
import Mathlib.Algebra.Equiv.TransferInstance
import Mathlib.Algebra.Field.ZMod
import Mathlib.Algebra.Polynomial.Cardinal

theorem my_theorem [Ring ℤ] : CharZero ℤ := sorry
```

事实上，按照所陈述的，这个陈述说的是 $\mathbb{Z}$ 上每一个环的特征都为 0，而这是错误的，因为存在特征非零的可数无穷环（例如 $(\mathbb{Z}/2\mathbb{Z})[X]$）。下面的证明由 Bhavik Mehta 给出：

```
open Cardinal in
theorem bad : False := by
  have e1 : #(Polynomial (ZMod 2)) = ℵ₀ := by simpa using (Cardinal.nat_lt_aleph0 2).le
  have e2 : #ℤ = ℵ₀ := by simp
  obtain ⟨e⟩ := Cardinal.eq.1 (e2.trans e1.symm)
  let i : Ring ℤ := e.ring
  cases @Nat.cast_injective _ _ my_theorem 2 0 rfl
```

这种情况的一个症状是，你可能会遇到在 infoview 中看起来完全相同但实际上并不相等的表达式，导致诸如 rfl、apply 和 rw 之类的策略带着晦涩的信息失败。例如，下面的策略状态显示目标是证明 dist 0 1 = dist 0 1，而你必须深入点击右侧 dist 函数的三层，才能发现它使用了错误的实例。

```
import Mathlib.Topology.MetricSpace.Basic

example [inst : MetricSpace ℝ] : dist (0 : ℝ) 1 = inst.dist (0 : ℝ) 1 := by
  /-
  Tactic state:
  1 goal
  inst : MetricSpace ℝ
  ⊢ dist 0 1 = dist 0 1
  -/
  sorry
```

在这个特定的例子中，尝试 rfl 会给你如下错误信息

```
tactic 'rfl' failed, the left-hand side
```

```
  @dist ℝ (@PseudoMetricSpace.toDist ℝ Real.pseudoMetricSpace) 0 1
is not definitionally equal to the right-hand side
  @dist ℝ (@PseudoMetricSpace.toDist ℝ MetricSpace.toPseudoMetricSpace) 0 1
inst : MetricSpace ℝ
⊢ dist 0 1 = dist 0 1
```

由此你可以断定这两个类型类并不匹配。当诸如 `congr` 或 `convert` 之类的策略生成子目标时，往往值得在那些目标上运行 `rfl`，以查看它们是否是由不匹配的类型类实例引起的。

在实际代码中，你大概不会无意间写出 `inst.dist`，但在更复杂的例子中，错误的实例可能会以难以察觉的方式表现出来。

感谢 Edward van de Meent 建议我纳入这个主题并找出了一个误导性的陈述，也感谢 Bhavik Mehta 完成了第一个例子为 `False` 的证明细节。

把 Set 当作类型使用

在 Lean 中，`Set X` 被定义为 `X → Prop`，所以集合是谓词，而非类型。然而，存在一个从 `Set X` 到 `Type u` 的隐式强制转换，因此如果你有参数 `(X : Type) (s : Set X) (a : s)`，那么 `(a : s)` 实际上表示 `(a : {x : X // x ∈ s})`。这里，`{x : X // x ∈ s}` 是 `Subtype (fun x ↦ x ∈ s)` 的记法。

也就是说，`a` 实际上是一个对 `⟨x, hx⟩`，其中 `x : X` 且 `hx` 是 `x ∈ s` 的一个证明。所以，`a` 实际上并不是 `X` 的一个元素，但你可能没意识到这一点，因为还存在另一个从 `{x : X // x ∈ s}` 到 `X` 的隐式强制转换，它使得 `a` 在某些（但并非所有）情形下表现得像 `X` 的一个元素。

这会导致各种各样的问题。例如，

```
import Mathlib.Data.Set.Basic

example (s : Set ℕ) (n : s) : 0 + n = n := by
  sorry
```

无法编译，因为 Lean 不知道 `0 + n` 是什么意思，因为从技术上讲 `n` 并不是一个自然数。你必须写 `(n : ℕ)` 才能使这个加法成立。

此外，如果你试图对 `n` 进行归纳来证明这个，你可能会发现你的策略并没有做你所期望的事。

```
import Mathlib.Data.Set.Basic
import Mathlib.Tactic

example (s : Set ℕ) (n : s) : 0 + (n : ℕ) = n := by
  /- Tactic state before:
```

```
  s : Set ℕ
  n : ↑s
  ⊢ 0 + ↑n = ↑n
  -/
  induction' n with d hd
  /- Tactic state after:
  s : Set ℕ
  d : ℕ
  hd : d ∈ s
  ⊢ 0 + ↑⟨d, hd⟩ = ↑⟨d, hd⟩
  -/
```

这里，该策略并没有把 n 当作自然数来进行归纳；相反，它拆解了 Subtype 的两个分量。如果你改用未加撇号的 induction 策略，你会得到一条“invalid alternative name ‘zero’, expected ‘mk’”的错误信息。

注意策略状态中 ↑ 的出现。它表示一个强制转换，这是 n 的类型可能并非你所想的那样的一条线索。

由于这些问题以及其他原因，如果你有一个参数 (s : Set X) 并且想假设 a 是 s 的一个元素，那么添加两个参数 (a : X) (ha : a ∈ S) 往往比写 (a : s) 更好。类似地，如果你想让 t 成为 s 的一个子集，你应该声明 (t : Set X) (h : t ⊆ s)，而非 (t : Set s)。这种从 Set 到类型的强制转换通常应当只保留给那些你需要把一个 Set 传递给另一个要求类型作为输入的函数的情形。

Mathlib 的代数库正是基于这一考虑设计的。例如，Subgroup 是这样定义的

```
structure Subgroup (M : Type*) [Group M] where
  carrier : Set M
  mul_mem {a b} : a ∈ carrier → b ∈ carrier → a * b ∈ carrier
  one_mem : (1 : M) ∈ carrier
  inv_mem {x} : x ∈ carrier → x⁻¹ ∈ carrier
```

这是为了让你在拥有一个 H : Subgroup G 时，可以通过声明 x : G 和 hx : x ∈ H 来处理 H 的元素，而不必写 x : H 并应对 x 实际上并不具有类型 G 的问题。

Sort _

写 Sort _ 和 Type _ 会让 Lean 自动填入全集层级参数。有时，这会导致 Lean 把全集层级特化得比你预期的更窄。例如，在

```
import Mathlib.Data.Countable.Defs
```

```
example (α : Sort _) (x y : α) (_ : Countable α := inferInstance) : x = y := by
  rfl
```

中，`inferInstance` 默认参数找到了 `Prop.countable`，这导致 `Sort _` 被限制为 `Prop`。于是，`α : Prop`，这意味着由证明无关性，`x : α` 与 `y : α` 在定义上相等。

在较旧的 Lean 版本中，这种行为甚至更容易被触发，因为 `:=` 之后的代码可能会影响 `Sort _` 是什么。例如，下面的代码可以在 Lean 4.8.0 中编译：

```
example {α : Sort _} (x y : α) : x = y := by
  have := α ∧ α -- Force α to have type Prop
  rfl
```

由于这种行为，建议你要么使用 `Sort*` 或 `Type*`（它们是 `Sort _` 和 `Type _` 的更严格版本，会迫使 Lean 每次使用时都生成一个新的全集参数），要么显式地指定全集参数。

试图证明关于 Float 的性质

默认情况下，输入像 `5.42` 这样的数会创建一个类型为 `Float` 的项，而非 `Rat` 或 `Real`。证明关于 `Float` 的性质是非常困难的。这是因为在 Lean 的数学部分中，`Float` 被定义为一个不透明类型，而 `Float = Unit` 与 Lean 的核心类型论是相容的（只要不使用 `native_decide`）。但在计算上，`Float` 遵循 IEEE 754 binary64 格式，这类似于 C 中的 `double` 或 Rust 中的 `f64`。（关于 32 位浮点数，参见 `Float32`。）这意味着要证明关于 `Float` 的任何有意义的内容，你都必须使用 `native_decide`，而这是一个有风险的策略（参见本文档中关于 `native_decide` 的章节）。

此外，由于浮点数的行为，`0.1 + 0.2 == 0.3` 求值为 `false`。（解释参见 https://0.30000000000000004.com/。）

所以，如果你对把 Lean 用作编程语言不感兴趣，或者想要证明关于你所处理的数的复杂性质，你应该避免使用 `Float`，转而使用诸如 `Rat` 或 `Real` 之类的其他数值类型。

`native_decide`

`native_decide`（也写作 `decide +native`）类似于 `decide` 策略，区别在于它使用 `#eval` 来运行判定过程，而非在内核中对其进行归约。这有可能比 `decide` 快得多，而且它是证明关于 `Float` 及其他一些不透明类型的性质的唯一途径，但它是有风险的，因为它信任 Lean 编译器，而编译器比 Lean 内核复杂得多，也更有可能包含错误。

具体来说，使用 `native_decide` 会使你的定理依赖于 `Lean.ofReduceBool` 公理，该公理表明 Lean 编译器是可信的。你可以通过使用 `#print axioms` 来检查一个证明是否依赖于这条公理。除了信任编

译器之外，Lean.ofReduceBool 还信任所有的 @[extern] 和 @[implemented_by] 注解以及对编译器的其他扩展。由于在 Lean 核心中有数百个这样的注解，目前 native_decide 几乎可以肯定有能力证明 False。它还会信任你自己的 @[extern] 和 @[implemented_by] 注解，而不会对它们加以追踪：

```
def m := 37

@[implemented_by m]
def n := 22

#reduce n -- 22
#eval n -- 37

theorem bad : False := by
  have : n = 22 := by decide
  have : n = 37 := by native_decide
  contradiction

#print axioms bad -- 'bad' depends on axioms: [Lean.ofReduceBool]
```

贡献给 Mathlib 的代码不允许使用 native_decide。

Panic 不会中止程序

如果你把 Lean 用作编程语言，请注意 panic! 宏默认并不会触发崩溃；相反，它只是打印一条错误信息并让代码继续运行。这同样适用于像 Option.get! 这样的函数。panic 之所以这样表现，是因为它是一个安全函数，并且在形式上等价于 default。

如果你用 panic 来防护潜在危险的 IO 操作或防止数据损坏，这可能会很危险。你可能想考虑把环境变量 LEAN_ABORT_ON_PANIC 设为 1，尽管对于环境变量难以控制的面向用户的应用而言，这可能并不足够。

Lean 3 代码

互联网上能找到大量的 Lean 3 代码，而 LLM 也经常生成 Lean 3 代码而非 Lean 4 代码。如果你在阅读旧的 Lean 代码，或者在使用 LLM，并且你看到诸如以下的细节 - 类型名以小写字母开头，例如 nat 和 rat - 不以库名开头的小写 import，例如 import data.real.basic（其 Lean 4 等价形式是 import Mathlib.Data.Real.Basic）。- 使用 begin 和 end 来包围策略模式，而非使用 by - 在引入 2 个或更多变量或全集时使用复数命令，例如 variables 和 universes - 不带方括号的 rw 策略 - refl 策略（在 Lean 4 中拼作 rfl）

那么你所阅读的代码很可能是 Lean 3 代码（或者，如果你在使用 LLM，它可能是 Lean 3 与 Lean 4 不连贯的混合体）。Lean 3 和 Lean 4 是非常不同的语言，所以 Lean 3 代码在 Lean 4 中无法工作。

非终结性 simp

这是你初次编写证明时无需担心的事情,但如果你想整理一个证明,或许是为了能将其添加到像 mathlib 这样的库中，那么请注意，在策略块的中间使用 `simp` 被认为是不良实践。在下面的例子中

```
example : ... := by
  ...
  simp
  rw [...]
  exact h
```

对 `simp` 的使用被认为是非终结性的。

非终结性 `simp` 会带来可维护性问题。一般来说，随着越来越多的引理被添加到 mathlib，`simp` 会随时间变得越来越强大，这意味着当库中其他部分的代码发生改变时，`simp` 之后的目标状态可能随之改变，从而有可能破坏 `simp` 之后的任何策略，例如本例中的 `rw`。在较少见的从 mathlib 中移除引理的情况下，终结性 `simp` 和非终结性 `simp` 都可能被破坏，但修复一个终结性 `simp` 调用通常要容易得多。

为避免非终结性 `simp`，你可以使用诸如 `simpa` 或 `simp_rw` 之类的 simp 变体，把 `simp` 与其后的策略合并起来；或者你可以通过使用 `simp?` 来“压缩”你的 simp 调用。请注意，只要 `simp` 能完整地关闭一个目标，它出现在证明中间是没有问题的。例如，尽管下面例子中的 `simp`

```
  induction n with
  | zero =>
    simp
  | succ d hd =>
    ...
```

位于证明中间，但它不被视为非终结性 simp，因为它完整地关闭了一个目标。

更多信息，请参见这些关于非终结性 simp 的笔记。

忽略警告

请密切关注你的 Lean 文件中的警告。一个已完成的定理中出现的未使用变量警告，可能意味着并非所有假设都被用到了，而这通常表明该定理陈述是错误的。类似地，如果你自己并没有输入 `sorry`，却出现一条声明使用了 sorry 的警告，这可能表明某个策略失败了，并悄悄地用一个合成的 sorry 关闭了证明。

如果你确信某条警告不适用，你可以通过适当地使用 `set_option` 来禁用它。这比忽略警告、把判断哪些警告是预期的、哪些是你应当关注的留给他人（或未来的你自己!）去猜测，是更好的编程实践。

请注意，`#lint` 命令（定义于 `Batteries` 中）能检测一些常见问题，例如句法上的恒真式。它只会检查 `#lint` 命令之上的文件部分，因此应当在文件末尾运行它。

易混淆的 Unicode 字符

Lean 大量使用非 ASCII 的 Unicode 字符，但这样做的一个副作用是，许多字符看起来相似，但在 Lean 中却具有非常不同的功能。如果你使用带有 Lean 4 扩展的 VSCode，你可以将鼠标悬停在不熟悉的字符上，以查看如何输入它们的说明。

一些经常被混淆的字符包括：

字符	Unicode 名称	输入方式	在 Lean 中的用途
`|`	Vertical Line (U+007C)	ASCII 字符	集合构造记法；模式匹配，绝对值
∣	Divides (U+2223)	`\\|`、`\dvd`、`\mid`、`\shortmid`	整除
Π	Greek Capital Letter Pi (U+03A0)	`\p`、`\P`、`\Pi`	依值函数类型：`Π x : α, β x` 是 `(x : α) → β x` 的另一种写法
∏	N-Ary Product (U+220F)	`\prod`	带索引的乘积：参见 `BigOperators.bigprod`
⊓	Square Cap (U+2293)	`\inf`、`\meet`、`\glb`、`\sqcap`	两个元素的下确界：`a ⊓ b` 表示 `min a b`。参见 `Min` 类型类，即便序不是线性的也会用到它
⊓	N-Ary Square Intersection Operator (U+2A05)	`\infi`、`\Glb`、`\Inf`、`\Meet`、`\Sqcap`、`\bigglb`、`\biginf`、`\bigmeet`、`\bigsqcap`	带索引的下确界运算，需要 `InfSet` 类型类。
Σ	Greek Capital Letter Sigma (U+03A3)	`\S`、`\GS`、`\Sigma`	`Sigma`（即依值对／依值和）类型
∑	N-Ary Summation (U+2211)	`\sum`	带索引的求和：参见 `BigOperators.bigsum`

字符	Unicode 名称	输入方式	在 Lean 中的用途
×	Multiplication Sign (U+00D7)	\x、\times、\multiplication	类型的笛卡尔积
x	Latin Small Letter X (U+0078)	ASCII 字符	无特殊功能
X	Latin Capital Letter X (U+0058)	ASCII 字符	通常无特殊功能；当 Polynomial 命名空间打开时，表示多项式变量／不定元
*	Asterisk (U+002A)	ASCII 字符	乘法
∨	Logical Or (U+2228)	\v、\or、\vee	或符号：a ∨ b 表示 Or a b
\/		ASCII 字符	或符号 ∨ 的 ASCII 替代写法
v	Latin Small Letter V (U+0076)	ASCII 字符	无特殊功能
·	Middle Dot (U+00B7)	\.、\centerdot	在策略模式中缩进并聚焦于子目标；匿名函数语法，例如 (· + 2)，它是 fun x ↦ x + 2 的简写
•	Bullet (U+2022)	\smul、\bub、\bu	标量乘法或左作用（参见 HSMul 和 SMul 类型类）
□	Black Very Small Square (U+2B1D)	\cdot、\dot、\tr、\con	仅用于 u □ᵥ w = dotProduct u w（用 _v 输入 ᵥ）
→	Rightwards Arrow (U+2192)	\r、\imp、\->、\to、\r-、\rightarrow	函数类型 A → B
->		ASCII 字符	函数类型中 → 的 ASCII 替代写法
↦	Rightwards Arrow From Bar (U+21A6)	\mapsto、\\|->、\r-|	用于 Lambda 绑定子，例如 fun x : Nat ↦ x + 2
=>		ASCII 字符	模式匹配；↦ 的 ASCII 替代写法
⟶	Long Rightwards Arrow (U+27F6)	\hom、\-->、\longrightarrow、\r--	a ⟶ b 是范畴或箭图中从 a 到 b 的态射的类型：参见 Quiver.Hom。

字符	Unicode 名称	输入方式	在 Lean 中的用途
↪	Rightwards Arrow With Hook (U+21AA)	`\hookrightarrow`	`α ↪ β` 是从 `α` 到 `β` 的单射函数的类型：参见 `Function.Embedding`
=	Equals Sign (U+003D)	ASCII 字符	对象之间的相等：参见 `Eq`
==		ASCII 字符	布尔相等：参见 `BEq`。布尔相等是类型相关的，但如果存在 `LawfulBEq` 实例，则 `BEq` 与 `Eq` 一致。
≃	Asymptotically Equal To (U+2243)	`\equiv`、`\~-`、`\simeq`	双射：`α ≃ β = Equiv α β` 是配有双侧逆的从 `α → β` 的函数的类型；`≃o`、`≃r` 等则指特定种类的同构。
≅	Approximately Equal To (U+2245)	`\iso`、`\~=`、`\cong`	范畴中的同构；当 `open scoped Congruent` 生效时，转而指 `Congruent`。
≈	Almost Equal To (U+2248)	`\~~`、`\approx`、`\thickapprox`	类型相关的等价记法：参见 `HasEquiv`。通常指一个由类型类推断出的 `Setoid` 实例
~	Tilde (U+007E)	ASCII 字符	`List`、`Array` 和 `Vector` 的置换等价关系；对其他类型有某些特殊含义
≡	Identical To (U+2261)	`\==`	`a ≡ b [MOD n]` 表示 `Nat.ModEq n a b`，`a ≡ b [ZMOD n]` 表示 `Int.ModEq n a b`。对其他类型有某些特殊含义

结构字段中的默认值

用户常常以为结构字段中 := 的使用意味着“此字段被定义为具有此值”。但事实并非如此！相反，这是为结构字段提供默认值的方式。具体来说，这些值可以被覆盖。

```
structure foo : Type where
  n : Nat := 37 -- I want n to always be 37...

def X : foo where
  n := 42 -- ...but that's not the way to do it

#eval X.n -- 42

def Y : foo where -- only if `n` is not supplied does the default value kick in.

#eval Y.n -- 37
```

如果你确实想要类似这样的效果，请改试下面的做法：

```
structure foo : Type where

def foo.n : Nat := 37

def X : foo where

#eval X.n -- 37
```

为 mathlib 做贡献

我们很高兴你有兴趣为 mathlib 做贡献。这个项目需要大量的帮助。另一方面，它也相当庞大：要做出有用的贡献，你需要遵循若干原则。

本页解释为 mathlib 贡献什么以及为什么贡献，涵盖：* 哪些类型的贡献是受欢迎的 * 特别是仅涉及风格的贡献 * 负责任地使用 AI：只有特定类型的 AI 贡献才会被 mathlib 接受

为 mathlib 贡献什么

小的修复（例如对文档字符串的修复）以及在已有理论中添加单个引理，作为对 mathlib 的贡献几乎总是受欢迎的。扩展已有理论的较长 PR 也几乎总是受欢迎的。

但是，向 `mathlib` 添加全新的理论又如何呢？在这里，情况会更加微妙。你需要考虑的第一个问题是，你想要贡献的内容是否适合 `mathlib`。虽然目前对于 mathlib 的职责范围究竟为何还没有正式的描述，但以下是一些你可以就所提议的贡献加以思考的问题。

- 这些内容通常会在数学系教授或研习吗？它会自然地成为本科或研究生数学课程的一部分，或研究层次数学研讨小组的一部分吗？如果不是，那么这些内容可能不在 `mathlib` 的范围之内。
- 这些内容的主题是否处于 `mathlib` 维护者的数学兴趣 之内?如果不是,那么随着 lean 和 `mathlib` 随时间演进，维护者可能会发现你的代码难以维护，这同样可能使其不适合 `mathlib`。

特别地，mathlib 的职责范围 不应被理解为「全部数学及相关领域」。随着未关闭 PR 数量的增加，维护者有时需要做出一些艰难的决定。

如果你不确定你所提议的主题是否适合 mathlib，那么请随时在 Lean Zulip 的 `#mathlib` 频道 开启一场讨论。

与维护者的专业知识可能无法覆盖全部数学这一事实相关的一个问题是：你或许需要思考 谁将会评审你潜在的 PR。我们鼓励贡献者为自己的 PR 寻找评审者。PR 评审者 不必是维护者！这似乎是社区中一个常见的误解。对 PR 的评审，尤其是来自新评审者的评审，本质上总是受欢迎的。

也请考虑创建一个独立仓库、并将 `mathlib` 作为依赖项的可能性。github 上有许多 Lean 仓库，由 reservoir 编入索引。而 这里 列出了那些以 `mathlib` 作为依赖项的项目。建立一个依赖于 `mathlib`

的新项目这一方案，特别适合那些与 mathlib 维护者专业知识不相吻合的领域中的项目。这样一个仓库的例子是 组合博弈论仓库。这一方案也适合那些希望快速推进的项目；在撰写本文时（2026 年中），mathlib 有超过 2600 个未关闭的 PR，对 mathlib 贡献的评审和合并可能需要一段时间。

风格更改

`mathlib` 有一份 风格指南，修复该指南所记录的风格违例的 PR 是受欢迎的。其他未经受影响文件作者明确批准的风格类 PR 可能会被关闭。我们邀请作者改为在 Zulip 上讨论所提议的更改，并在评审者之间达成显著共识后，向风格指南提交 PR。

AI 的使用

使用人工智能工具来生成代码正变得越来越普遍。尽管这可能很实用，但它们的使用也带来了伦理、生态、法律和社会方面的关切。我们认识到，在这一话题上存在着强烈的意见分歧。话虽如此，虽然单凭个人行动无法解决这些关切，但我们请你考虑你使用 AI 所带来的影响。在评审 PR 时，我们尤为担忧的是：如果没有一位人类贡献者在积极学习，那么评审者工作的教学价值就被白白浪费了。

在 GitHub 或 Zulip 上撰写评论时不允许使用 LLM：请使用你自己的语言。

Mathlib 有意保持非常高的标准（在普遍性、与库其余部分的整合以及可维护性方面，包括代码风格）。截至 2026 年中，在没有 Lean 领域专家监督下由 AI 编写的代码，远远未能达到这一标准。评审团队的成员将不加评论地径直关闭任何使用 LLM 生成的低质量 PR，尤其当作者在提交 PR 之前几乎没有付出努力直接参与社区中关于其价值的讨论时。如果我们注意到你提交了多个 PR 却未付出这种学习努力，或未遵守我们社区的伦理标准，我们将暂停（或永久封禁）你提交新 PR 以及使用 Zulip 聊天的权限。

让代码达到 mathlib 的标准，需要亲手理解和编写 Lean 代码。如果你只是想帮忙而不愿付出学习的努力，那么向 mathlib 提交 PR 是适得其反的：mathlib 维护者所需付出的努力大于所得的收益，因为用于提升代码质量的时间并不会在未来的 PR 中带来更高的质量。

如果你使用了人工智能（例如，使用 GitHub 的 copilot 模式、向 ChatGPT 这样的 LLM 提问，或使用 Codex、Claude、Gemini 这样的智能体，乃至 Aristotle 这样专门面向 Lean 的智能体），你必须在 PR 描述中说明这一点。请说明你使用了哪些工具以及你是如何使用的。这为评审者提供了有用的背景信息：工具所犯的错误与人类不同，因此了解这一点能更容易地发现常见错误。如果你的 PR 包含大量由 LLM 生成的代码，请通过添加 `LLM-generated` 评论来添加 `LLM-generated` 标签。

你务必理解 AI 所写的全部内容。这包括理解为形式化所做的任何设计决策，并能够在不借助 AI 的情况下向评审者论证每一项决策的合理性。如果你做不到这一点，那么这个 PR 很可能实际上对社区具有负价值。

如何贡献

如果你已经阅读并理解了上述贡献准则，那么你就可以 学习如何贡献 了。

Mathlib 的价值观

我们的价值观

为了最大化 Mathlib 库的效用并应对其规模，我们追求以下若干价值观。

可信赖性

纸面上的数学概念并不总是能够直接地、简单明了地转写为 Lean 代码。事实上，关于一般性的微妙之处以及关于代码中实际实现的问题，可能引出一些难以回答的问题——如果我们要求 Lean 代码忠实地反映现代研究数学的重要思想，就必须回答这些问题。

Mathlib 中的代码会由该数学领域的专家以及 Lean 开发的专家以审慎的眼光进行审阅，从而使代码的使用者能够信赖：其中的定义和定理陈述正是数学家所期望的样子。

开源

Mathlib 是免费的开源软件。

可维护性

当新代码被加入 Mathlib 时，Mathlib 维护者同意对该代码进行无限期的“维护”。这意味着，如果出于任何原因需要调整代码（例如为了使其与新版本的 Lean 语言兼容，或由于库中其他地方的改动），维护者接受确保该调整得以完成的责任。因此，维护者要求新代码以减少此类调整必要性的方式编写。需要牢记的一句口号是：“零 `sorry` 是起点，而非终点”。

一般化的定义

传统数学文献往往比 Mathlib 具有更为聚焦的范围，并且其定义是为这一范围量身定制的。例如，一本微积分教材可能采用在 p-进数上无效的定义来展开理论，一篇代数几何论文可能假定系数为复数，或者一部关于流形的专著可能假定所有流形都是有限维的。然而，在 Mathlib 中，我们希望在尽可能多的合理情形下为用户提供支持，因此我们往往会极为审慎地确保定义具有非常广泛的适用性。（事实上，Mathlib 的微积分理论确实支持 p-进数，其代数几何允许系数为任意交换环，而其流形可以是无限维的。）

弱假设

在 Mathlib 中证明定理时，我们竭力使假设尽可能弱。

这除了具有使定理更具广泛适用性的明显好处之外，还能减少任何调用该定理者的工作量。例如，如果某个定理在去掉紧致性假设后仍然成立，那么即便对于在紧致性成立的情形中应用该定理的用户而言，在调用该定理时无需证明紧致性，也可能为他们省去大量工作。鉴于单个定理可能被调用许许多多次，付出额外努力将其假设尽可能削弱，往往是值得的。

强结论

与“弱假设”这一价值观类似，我们力求使定理的结论尽可能强。例如，如果一个存在性论证可以被加强为证明某个点集具有正测度，那么我们便会以这种形式陈述定理。即便这可能需要付出相当多的额外努力，我们有时仍会力求给出更强的结论。

经典而非构造

Mathlib 不做任何努力去避免使用排中律，也几乎不做任何努力去给出可计算的定义。在这里，我们将这两种做法都称为“构造性的”。

Mathlib 不以构造性为目标，主要有三个原因：1. 大多数当代研究并非构造性的 1. 许多结果在构造意义下并不成立 1. 构造性证明往往更长、更难

我们并不声称这是一种更优越的数学实践方式，而仅仅是出于实用主义的考虑而坚持这一价值观。

尽管如此，Mathlib 的某些角落实际上是构造性的。这部分是因为库中一些重要的早期内容本就如此。然而，我们并不要求新材料是构造性的，甚至当非构造性数学能够带来更短的证明或为用户提供更符合人体工学的体验时，我们会更倾向于采用它（用技术术语来说，例如人们可能希望避免“可判定性菱形（decidability diamonds）”）。

下游项目

越来越多的项目依赖于 Mathlib。这类项目对我们在 Mathlib 中所做的任何破坏性改动（例如略微改变某个定义或引理陈述）都很敏感。我们竭力以高度一般化的方式编写定义和引理，原因之一便是减少此类改动成为必要的可能性。我们还会插入迁移提示（例如 `deprecated` 注解）以帮助下游项目应对破坏性改动。

对非形式化工作者的可及性

我们希望 Mathlib 对那些除 LaTeX 之外没有计算机代码经验的人也是可及的。此外，我们希望在实现这种可及性的同时不损害上述价值观。这是一项艰巨的任务，仍有大量工作有待完成。

部分出于可及性的考虑，我们要求所有定义都附带人类可读的注释（即“文档字符串”），并且我们总体上鼓励撰写经过深思熟虑的代码注释，尤其是在较长的证明中。这类代码注释在使用自然语言或 LaTeX 而非 Lean 代码时，往往最为有用。我们还相信，可及性的关键在于创建独立的产物（教程、术语对照手册、教学课程、期刊论文……），这些产物建立在 Mathlib 之上，并对重要概念给出并列对照的演示。

对 PR 审阅的意义

Mathlib 通过合并来自贡献者的 PR（拉取请求）而演进。正是在审阅这类 PR 的过程中，使命的一致性得到评估，价值观得以践行。我们在下文中就这一代码审阅流程给出一些说明。在许多情形下，风格指南和 PR 审阅指南中提供了更多细节。

请保持礼貌

Mathlib 有一份行为准则。在贡献和审阅时，请铭记这一点。

主题是否合适?

对每一项贡献提出的第一个问题是：其主题是否在 Mathlib 的范围之内。这通常很容易回答。更多说明请参见贡献指南。

作者是人类吗?

除其他功能之外，代码审阅还是一种教育工具。Mathlib 有数百名贡献者，其中大多数人很可能是在审阅过程中习得其技艺的。对于新人而言，审阅的教育意义往往远比其对库的实际贡献更有价值，这并不

罕见。由于审阅带宽长期处于饱和状态，许多审阅者不愿提供其服务，除非作者是人类。

定义是否“正确”?

这或许是审阅最重要的功能，因为 Lean 只能告诉你你定义了某个东西，却无法告诉你你定义的正是你所意图的东西。非平凡的定义在审阅中会得到最多的关注。

此外，给予定义特别关注还有非数学方面的原因。同一概念在 Lean 中往往存在许多数学上正确的实现，其中某些实现相对于其他实现可能具有明显的优势。

代码是否可维护?

对可维护性作出审慎的评估是审阅的关键部分。

定理是否合适?

在定义之后，我们研究新的定理，检查其假设是否在合理范围内尽可能弱，以及其结论是否足够强。此外，在审阅过程中，我们常常要求贡献者将定理“模块化”，其程度远超非形式化文献，即把一个定理拆解为许多个小引理。

新定义是否配有 API?

新定义应当附带使用它们的引理。在理想情形下，一个定义可以被一组引理完全刻画；即便无法做到这一点，部分刻画仍然非常可取。我们将这样一组引理称为 API，在审阅过程中，我们鼓励贡献者添加此类引理。添加 API 还有助于增强信心：定义确实表达了其所意图的含义，并且符合人体工学。

各项内容是否放在了正确的文件中?

将定义、引理和定理放置在正确的文件中是很重要的。通过保持 Mathlib 的导入树宽而扁平，我们减少了那些只导入库的某一片段的人的内存占用，并通过使编译更易于并行化来缩短挂钟编译时间。正确放置的另一个原因是可发现性：我们希望让人们能够轻松猜出某个定义或引理可能位于何处。

证明是否人类可读?

较长的证明脚本（50 行以上）应当努力勾勒出论证的轮廓，必要时使用代码注释。这一点的重要性低于其他各项，但仍然是可取的。

命名是否正确?

Mathlib 遵循命名约定中所描述的命名方案。我们力求在审阅过程中贯彻这一方案。

代码性能是否良好?

我们竭力避免添加在繁释（elaborate）或类型检查时缓慢的代码。如果代码使用 `set_option` 来更改诸如 `maxHeartbeats` 或 `synthInstance.maxHeartbeats` 这类值的默认设置，那么通常意味着某处出了问题。此外，我们希望代码具有足够的鲁棒性，以便我们可以预期未来在其之上开发的代码不会遇到性能问题。

Lean 包含一套用于分析代码性能的工具。此外，在 PR 上用 `!bench` 进行评论可以提供有价值的性能信息。

携带数据的类型类实例是否会造成菱形?

任何携带数据的新类型类 `instance` 在审阅过程中都会受到研究，以增强对其不会造成菱形的信心。这些菱形可能难以发现，需要审慎的思考。

自动化注解是否就位?

确保为自动化添加注解是审阅的一个重要部分，例如 simp、gcongr、fun_prop 和 grind。类似地，对诸如 to_additive、to_dual 这类代码生成器的使用也应在审阅过程中加以检查。

代码风格是否良好?

在审阅过程中，我们还会检查一些次要的风格要点，例如：* 在适当之处使用 `variable`。* 在适当之处使用 `section` 和 `namespace`。* 是否正确放置了 `public` 和 `private` 修饰符?（新文件应避免对所有内容使用 `@[expose] public section`。）* 证明压缩（golfing），但不过度压缩。请注意，减少字符数／行数有时是更优证明的副产品，但它并非目标，而且一个更好的证明往往会增加这些数值。* 空白字符。* 文档字符串的格式。

如何为 mathlib 做贡献

本页说明在为 mathlib 做贡献时应当如何操作以及会遇到什么情况。即使你是经验丰富的 Git 用户，也请查看下面的步骤，因为我们会解释 mathlib 拉取请求所特有的约定。

- 在你着手贡献之前以及贡献过程中，使用 Zulip 来讨论你的贡献。
- 创建一个 GitHub 账号，并通过个人设置面板将你的 GitHub 用户名添加到你的 Zulip 个人资料中。我们也强烈建议将你在 Zulip 上的显示名称设置为你的真实姓名。
- 遵守以下准则：
  - 面向贡献者的风格指南。
  - 关于命名约定的说明。
  - 文档准则。

在 mathlib 上工作

我们使用 `git` 来管理 `mathlib` 并进行版本控制。

如果你以前没有使用 git 为开源项目做过贡献，请参阅 Mathlib4 贡献者 Git 指南以获取详细说明。

`master` 分支是 mathlib 的“生产”版本。master 分支中的所有内容都必须能够无错误地编译，并且不能有任何 `sorry`，这一点至关重要。为确保这一点，我们只将通过了自动化持续集成（“CI”）测试、并经过 mathlib 维护者批准的改动提交到 `master`。

当你在为 `mathlib` 编写新的贡献时，应当在另一个分支上进行。你应当在你自己的 `mathlib` 仓库分叉（fork）中进行这项工作。

典型的工作流程:* 要开始工作,你需要一份 mathlib 的本地副本。* 首先,你需要前往 https://github.com/leanprover-community/mathlib4 并点击右上角的“Fork” ，以创建你自己的仓库分叉。你的分叉位于 https://github.com/USER/mathlib4。* 现在为你的分叉创建一份本地克隆并对其进行正确配置。关于如何正确设置你的分叉的详细分步说明，请参阅 Mathlib4 贡献者 Git 指南。`git clone https://github.com/YOUR_USERNAME/mathlib4.git  cd mathlib4  lake exe cache get` * 上述步骤只需做一次（而非每次贡献都做一次）。* 现在，每当你想为 mathlib 进行一项新的改动时，创建一个新分支：`git switch -c my_new_branch  # This creates a new branch`

`and switches to it` * 进行本地改动，例如使用带有 Lean 扩展的 Visual Studio Code。* 使用 `git commit -a`（或通过 VS Code 界面）提交你的改动。* 如果你想在本地编译所有内容以检查自己没有破坏任何东西，运行 `lake build`。如果你修改了导入层级中靠下的文件，这可能会花费很长时间。你也可以在向主仓库开启 PR 后推送你的改动，让我们的中央 CI 服务器为你完成这项工作。* 如果你创建了新文件，运行 `lake exe mk_all`。这会更新 `Mathlib.lean`，以确保所有文件都在其中被导入。* 为了将你的改动推送回 github 上你的仓库，使用 `git push` 如果它抱怨远程未配置，请按照 `git` 输出中的建议操作，运行 `git push --set-upstream origin my_new_branch` * 一旦你向主 `mathlib` 仓库开启了一个 PR（见下文），此时持续集成将自动启动。你可以在 GitHub 上你的 PR 页面查看 CI 状态（如果一切正常会有一个绿色的对勾，否则如果 CI 仍在运行会是一个黄色的圆圈，或者如果出了问题会是一个红色的叉）。你也可以使用 GitHub CLI 检查 CI 状态：`gh pr status`。* CI 完成后，你可以运行 `lake exe cache get` 来下载编译好的 oleans。

发起拉取请求（PR）

一旦你对本地改动满意，就该向主 `leanprover-community/mathlib4` 仓库的 `master` 分支发起一个拉取请求了：注意不要将此 PR 针对你自己分叉上的 `master` 分支发起。

- 如果你还没有这样做，请前往 https://leanprover.zulipchat.com/ ，做个自我介绍，并提及你的新 PR。
- 如果你做了大量的改动/新增，请尽量分成许多个包含小而自包含部分的 PR；一般来说，越小越好！这有助于你在进行过程中获得反馈，而且审阅起来也容易得多。这对新贡献者尤为重要，因为它能避免做无用功。
- PR 的标题和描述应当遵循我们的提交约定。
- 如果你正在移动或删除声明，请在提交信息的底部（即在 `---` 之前）使用以下格式包含这些行：

Moves: - Vector.* -> Mathlib.Vector.* - …

Deletions: - Nat.bit1_add_bit1 - …

任何你想从 PR 提交中排除的其他评论应当放在 `---` 之下。

一个 PR 的生命周期

许多审阅者使用审阅队列来识别已准备好接受审阅的 PR。下面的说明将确保你的 PR 出现在该队列中；如果它没有出现在那里，可能就不会受到太多关注。我们也欢迎所有人定期查看该队列（在 Zulip 上被链接为 `#queueboard`），并对其专长范围内的 PR 撰写审阅意见。你可以检查你的 PR 是否在队列上，如果不在，还需要做些什么才能让它进入队列。

审阅队列由 GitHub 的“标签”（labels）控制。在某个 PR 的主页面上，右侧应当有一个带有“reviewers”“assignees”“labels”等面板的侧边栏。点击“labels”标题以为当前项目添加或移除标签。标签只能由“GitHub 协作者”（即经验丰富的贡献者）直接编辑。不过，任何人都可以通过在 PR 评论中写下以下命令（每条单独一行）来添加/移除下列标签：- `awaiting-author` 会添加 **“awaiting-author”** 标签 - `-awaiting-author` 会移除 **“awaiting-author”** 标签 - `awaiting-zulip` 会添加 **“awaiting-zulip”** 标签 - `-awaiting-zulip` 会移除 **“awaiting-zulip”** 标签。在做出决定并已实施后使用此命令。- `WIP` 会添加 **“WIP”** 标签 - `-WIP` 会移除 **“WIP”** 标签 - `easy` 会添加 **“easy”** 标签 - `-easy` 会移除 **“easy”** 标签 - `help-wanted`、`-help-wanted`、`please-adopt`、`-please-adopt` 可用于标记一个需要外部投入的 PR。- `LLM-generated`、`-LLM-generated` 应当用于标记包含大量 LLM 生成代码的 PR。（请记住，如果你使用了 AI，你必须在 PR 描述中说明具体情况。）- 供贡献者从下游项目 **brownian**、**carleson**、**CFT**、**FLT**、**infinity-cosmos**、**sphere-packing** 和 **toric** 上行（upstream）工作时使用的标签，也可以用同样的方式添加和移除。- 任何形如 `t-*` 的主题标签（例如 `t-topology`），以及标签 `CI` 和 `IMO`，也可以用同样的方式添加和移除。PR 会根据其内容被自动打标签，但有时自动打标签不正确或不完整，因此这让你能够手动覆盖它。参见可用的主题标签。

此列表是完整的。如果你想添加一个不同的标签，请在 Zulip 上提出来！

如果你的 PR 能够构建（有一个绿色的对勾），会有人在几周内（取决于 PR 的大小；较小的 PR 会得到更快的回应）“审阅”它。他们很可能会留下评论并添加 **“awaiting-author”** 标签。你应当处理每一条评论，在问题解决后点击“resolve conversation”按钮。理想情况下，每个问题都用一个新的提交来解决，但这里没有硬性规定。一旦所有要求的改动都已实施，你应当移除 **“awaiting-author”** 标签，以重新开始这个流程。

有不同的人群可以审阅你的 PR：任何人、审阅者和维护者。任何有有用见解的人都可以审阅你的 PR。如果他们认为你的 PR 已准备好进入下一阶段，他们可能会在 GitHub 上留下一个“approving”（批准）审阅。这些审阅会被审阅者纳入考量。如果一位审阅者认为你的 PR 已准备好被合并，他们会为你的 PR 添加 **“maintainer-merge”** 标签。这些标签供维护者用来确定其审阅的优先级。维护者始终是给出最终批准的人。维护者拥有审阅者的权限，但还有更多的权力（例如合并 PR）。取决于人员的可用情况，第一位查看你 PR 的审阅者可能就是一位维护者：在这种情况下，你的 PR 可能无需先被“maintainer merge”就被合并。审阅时间可能会因我们志愿者的可用情况而有所不同。为了加快这个流程，你可以查看审阅准则并尽量确保你的 PR 遵循它们。如果你想明确地请求审阅，请在 Zulip 的 PR reviews 频道中创建一个话题。

如果一位维护者批准了你的 PR，一个 **“ready-to-merge”** 标签会被自动应用到该 PR 上。一个名为 `bors` 的机器人会从这里接手。（关于 bors 的更多细节请参见此处。）该 PR 会被加入到“合并队列”中。合并队列会被自动处理，但这需要一定的时间，因为它需要构建 mathlib 的各个分支。

在某些情况下，维护者会“委派”（delegate）该 PR。你会看到你的 PR 现在带有一个 **“delegated”** 标签。这要么意味着还有一些最终的改动被要求，但维护者信任你会做出这些改动并自己将 PR 发送给 bors，要么意味着维护者想给你最后一次机会在 PR 被合并前检查一遍。无论是哪种情况，当你准备就绪时，写一条包含“bors merge”这一行的评论将会使该 PR 被合并。

以下是一些其他常用的标签：

- 一个 **“WIP”**（= work in progress，进行中）PR 在接受审阅之前仍需要一些基础性的工作（例如，也许它仍然包含 `sorry`）。如果你想宣布你正在做某件你预期很快会完成的事情，可以发布一个 WIP。
- 一个 **“RFC”**（= request for comment，征求意见）是关于某项可能有争议、或需要专家就是否应当进行而做出决定的改动的 PR。
- 如果你不确定 CI 是否会成功，可以添加 **“awaiting-CI”**。这会暂时在主审阅队列中隐藏该 PR。当 CI 完成时，该标签会被自动移除。
- 考虑添加 **“help wanted”** 标签以直接征求贡献。
- **“blocked-by-other-PR”** 标签意味着在处理本 PR 之前，应当先解决某些特定的其他 PR。要为你的 PR 添加“blocked-by-other-PR”标签，请在 PR 评论中包含所依赖的 PR 编号（遵循那里评论中隐藏的示例），以便他人能够一目了然地看出应当先审阅哪些 PR。该标签会由一个机器人自动添加，并且会在那些其他 PR 被合并后自动移除。带有此标签的 PR 不会出现在审阅队列中。
- **easy** 标签应当用于标记那些可以立即批准的 PR。维护者和审阅者经常先看 easy 的 PR，以保持队列的流动。easy 的 PR 通常添加单个引理、修正文档中的拼写错误，或类似的内容。如果你对自己的 PR 是否琐碎有任何疑问，就不应当添加此标签。特别地，如果一个 PR 的差异（diff）超过 25 行、改动了任何现有的定义或定理陈述、添加了任何定义或新文件，或添加了任何并非与现有 `simp` 引理或实例直接类似的 `simp` 引理或实例，那么它通常不是 easy 的。（如果它们是直接类似的，你无论如何都应当在 PR 描述中给出这样的背景信息。）请注意，差异小并不会自动使一个 PR 成为 easy!
- **delegated** 标签意味着一位维护者已发出“bors delegate”（或“bors d+”）命令。此时该 PR 的作者应当在做出任何最终要求的改动且 CI 成功后，自己合并该 PR。他们可以使用“bors merge”来做到这一点。

处理合并冲突

由于多人并行地在 mathlib 上工作，可能有人在 `master` 上引入了一项与你在 PR 中所提议的改动相冲突的改动。如果这发生在你的 PR 上，一个机器人会自动添加 **“merge-conflict”** 标签，并且你的 PR 不会出现在审阅队列中。关于如何使用 GitHub 的在线工具解决合并冲突，请查看这篇 GitHub 教程。一旦冲突被解决，**“merge-conflict”** 标签会被自动移除，并且你的 PR 会回到审阅队列。

用合并，而非变基

（如果你不知道变基（rebasing）是什么，可以跳过这一节。）Mathlib 的 PR 在合并时会被压缩（squash）：特别地，中间提交的历史除了在 PR 中以外不会被记录。一旦一个 PR 已经经历了一些审阅活动，强烈建议不要再进行变基或压缩，因为这会混淆审阅历史，使得审阅新的改动更加困难。

面向 Mathlib4 贡献者的 Git 指南

本指南面向初次接触 git 但希望为 mathlib4 库做出贡献的数学工作者。贡献通过拉取请求来完成。我们将一步步介绍必要的工作流程。请注意，网络上还有许多其他指南介绍如何通过拉取请求为开源项目做出贡献。

本指南分为三个主要部分：

1. **一次性设置**（在你首次开始贡献时执行一次）
2. **日常工作流程**（处理贡献时的常用操作）
3. **补充信息**

前置条件

在开始之前，请确保你已具备：

- 计算机上已安装 Git
- 一个 GitHub 账户
- （可选但推荐）已安装 GitHub CLI 工具（`gh`）

第一部分：一次性设置

以下步骤只需在你首次开始为 mathlib4 做贡献时执行一次。

使用 GitHub CLI 直接跳到步骤 4

以下命令会为你完成步骤 1 到 3：它将 mathlib4 仓库 fork 到你的 GitHub 账户，将仓库克隆到你当前的目录中，并按推荐方式设置远程仓库。

```
gh repo fork leanprover-community/mathlib4 --default-branch-only --clone
```

步骤 1：在 GitHub 上 Fork 仓库

首先，你需要创建自己的 mathlib4 仓库副本（fork）：

1. 前往 https://github.com/leanprover-community/mathlib4
2. 点击右上角的“Fork”按钮
3. 选择你的 GitHub 账户作为目标。建议保持勾选“copy the master branch only”。
4. 等待 GitHub 创建你的 fork

此步骤你只需执行一次。你可以将你的 fork 复用于许多不同的分支和拉取请求。

步骤 2：获取仓库的本地副本

你在上一步创建的 fork 是 mathlib4 的一个”远程”副本，它位于 GitHub 的服务器上。现在你需要在自己的计算机上设置 mathlib4 的本地副本（也称为”克隆”）。

根据你是否已经拥有 mathlib4 的克隆，你有两种选择：

选项 A：如果你尚未克隆 mathlib4

方法 1：使用 GitHub CLI（推荐）

在下面的 shell 命令中，将 `YOUR_USERNAME` 替换为你的 GitHub 用户名：

```
gh repo clone YOUR_USERNAME/mathlib4
cd mathlib4
```

方法 2：手动克隆

在下面的 shell 命令中，将 `YOUR_USERNAME` 替换为你的 GitHub 用户名，以便将你的 fork（而非原始仓库）克隆到当前工作目录下一个名为 `mathlib4` 的目录中，然后进入该目录：

```
git clone https://github.com/YOUR_USERNAME/mathlib4.git
cd mathlib4
```

这会自动将你的 fork 设置为 `origin` 远程仓库，这正是我们想要的。

选项 B：如果你已经克隆了 mathlib4

如果你已经从原始仓库克隆了一份（例如你通过 Lean 4 VS Code 扩展创建了一份），你可以复用它。只需进入你现有的 mathlib4 目录：

```
cd path/to/your/existing/mathlib4
```

配置 GitHub CLI（如已安装）

如果你已安装 GitHub CLI，请将默认仓库设置为上游 mathlib4：

```
gh repo set-default leanprover-community/mathlib4
```

这可确保诸如 `gh pr checkout` 之类的 GitHub CLI 命令作用于主 mathlib4 仓库，而非你的 fork。

步骤 3：正确设置远程仓库

远程仓库的设置取决于你在上面选择了哪个选项：

如果你使用 GitHub CLI 克隆了你的 fork（选项 A，方法 1）

gh 已经为你处理好了这一步。

如果你未使用 GitHub CLI 克隆你的 fork（选项 A，方法 2）

你需要将原始仓库添加为 upstream：

```
git remote add upstream https://github.com/leanprover-community/mathlib4.git
```

如果你使用了已有的克隆（选项 B）

你需要重命名现有的远程仓库并添加你的 fork。将 YOUR_USERNAME 替换为你的 GitHub 用户名：

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR_USERNAME/mathlib4.git
```

验证你的远程仓库

无论你选择了哪个选项，都要验证你的远程仓库设置是否正确：

```
git remote -v
```

你应当看到：

```
origin    https://github.com/YOUR_USERNAME/mathlib4.git (fetch)
origin    https://github.com/YOUR_USERNAME/mathlib4.git (push)
upstream  https://github.com/leanprover-community/mathlib4.git (fetch)
upstream  https://github.com/leanprover-community/mathlib4.git (push)
```

步骤 4：配置 Master 分支

首先，从上游获取分支：

```
git fetch upstream
```

然后确保你的 master 分支跟踪 upstream/master：

```
git branch --set-upstream-to=upstream/master master
```

步骤 5：为你的工作流程配置 Git

设置默认推送行为

将 git 配置为默认将新分支推送到 `origin`：

```
git config push.default current
git config push.autoSetupRemote true
```

防止意外提交到 Master（可选但推荐）

为避免意外地直接提交到 `master`，你可以设置一个 pre-commit 钩子。首先，创建钩子文件：

```
mkdir -p .git/hooks
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/sh
branch="$(git rev-parse --abbrev-ref HEAD)"
if [ "$branch" = "master" ]; then
  echo "You can't commit directly to master branch"
  exit 1
fi
EOF
```

然后使其可执行：

```
chmod +x .git/hooks/pre-commit
```

第二部分：日常工作流程

以下是你在处理贡献时会经常用到的操作。

创建并在新分支上工作

保持你的 Master 分支为最新

在创建新分支之前执行此操作，以确保你基于最新的更改进行工作：

```
git switch master
git pull
```

创建新分支

然后创建并切换到一个新分支：

```
git switch -c my-feature-branch
```

当你首次推送该分支时，它会自动跟踪 `origin/my-feature-branch`。

基于另一个 PR 进行工作

如果你打算让你的工作依赖于另一个 PR：

1. 检出相关的 PR 分支：如果这是你自己的 PR，运行 `git switch <pr-branch-name>`；如果你打算在他人的 PR 之上进行工作，请按照下面*与他人的 PR 协作*一节中的说明操作。
2. 运行 `git pull` 以确保你与该拉取请求保持同步。
3. 运行 `git switch -c my-feature-branch` 以在当前分支之上创建一个新分支。

推送你的分支并开启 PR

推送你的分支

在完成你的更改和提交之后：

```
git push
```

开启拉取请求

1. 前往你在 GitHub 上的 fork：`https://github.com/YOUR_USERNAME/mathlib4`
2. 你应当看到一个横幅，建议为你最近的推送开启一个 PR
3. 点击“Compare & pull request”
4. 填写 PR 标题和描述
5. 点击“Create pull request”

或者,如果你没有看到该横幅,你也可以前往 https://github.com/leanprover-community/mathlib4/compare, 然后点击 `compare across forks`。你需要在“head repository”下拉菜单中选择你的 fork，并在“compare”下拉菜单中选择你想要合并的分支。

与他人的 PR 协作

请注意，即使只是用 VS Code 打开来自某个不可信来源分支的 Lean 代码，也可能在你的计算机上执行代码！请查阅补充信息一节中的安全警告。

方法 1：使用 GitHub CLI（推荐）

这比手动方法简单得多。要检出 PR #1234：

```
gh pr checkout 1234
```

这会自动处理远程设置和分支检出。

要切回你自己的分支：

```
git switch my-feature-branch
```

方法 2：手动检出

要手动检出他人的 PR，首先将他们的 fork 添加为远程仓库（将 `USERNAME` 替换为他们的 GitHub 用户名）：

```
git remote add contributor-name https://github.com/USERNAME/mathlib4.git
```

然后获取他们的分支：

```
git fetch contributor-name
```

最后，检出他们的分支：

```
git checkout contributor-name/their-branch-name
```

（可以用 `git remote remove <contributor-name>` 移除远程仓库。）

要切回你自己的分支：

```
git switch my-feature-branch
```

授予协作者访问权限

如果你想允许他人直接推送到你的 PR 分支：

1. 前往你的 fork：`https://github.com/YOUR_USERNAME/mathlib4`
2. 点击“Settings”标签页
3. 点击“Collaborators”（你可能需要重新认证）
4. 输入他们的 GitHub 用户名
5. 选择“Write”权限级别
6. 发送邀请

一旦他们接受，他们就可以在按照《基于另一个 PR 进行工作》 中的某种方法操作后，使用 `git push` 直接推送到你的 PR 分支。

补充信息

⚠ 安全警告

重要：当你授予某人对你 fork 的协作者访问权限，或者当你检出并运行他人的代码时，你可能在自己的计算机上运行了未经审查的代码。请只与你信任的人协作，因为他们有可能植入在构建过程中运行的恶意代码。

获取帮助

如果你遇到问题或对 git 工作流程有疑问，请在 Lean Zulip 聊天 的 `#new users` 频道中提问。社区非常乐于助人，欢迎大家提问！

快速参考

下面是你将会用到的最常见命令的汇总。

更新 master：

```
git switch master
git pull
```

在当前分支之上创建一个新分支：

```
git switch -c new-branch-name
```

推送你的分支并设置跟踪：

```
git push origin new-branch-name
```

如果你已按照上面指南设置了默认推送选项，那么以下命令即可：

```
git push
```

检出他人的 PR：

```
gh pr checkout PR_NUMBER
```

检查远程仓库配置：

```
git remote -v
```

检查你当前所在的分支：

```
git branch
```

切换到另一个分支进行工作：

```
git switch your-branch-name
```

常见故障排查

问题：“Your branch is behind ‘upstream/master’ ”**解决方法**：

```
git switch master
git pull
```

问题：“fatal: The current branch has no upstream branch”**解决方法**：

```
git push --set-upstream origin branch-name
```

问题：意外地提交到了你的 master 分支副本 **解决方法**：将这些提交移到一个新分支：

```
git branch new-branch-name
git switch master
git reset --hard upstream/master
git switch new-branch-name
```

其他资源

- git 术语表
- 日常 git 命令
- git 用户手册

Mathlib 命名约定

本指南针对 Lean 4 编写。

文件名

mathlib 中的 `.lean` 文件一般应采用 `UpperCamelCased` 命名。一个（极为罕见的）例外是以某个特定小写对象命名的文件，例如 `lp.lean` 用于专门讨论空间 ℓ_p（而非 L^p）的文件。此类例外应先在 Zulip 上讨论。

一般约定

大小写

与 Lean 3 不同（在 Lean 3 中约定所有声明都使用 `snake_case`），在 Lean 4 下的 mathlib 中，我们根据以下命名方案，混合使用 `snake_case`、`lowerCamelCase` 和 `UpperCamelCase`。

1. `Prop` 的项（例如证明、定理名）使用 `snake_case`。
2. `Prop` 与 `Type`（或 `Sort`）（归纳类型、结构、类）使用 `UpperCamelCase`。存在一些罕见的例外：某些结构的字段目前被错误地小写化了（见下面关于类 `LT` 的示例）。
3. 函数的命名方式与其返回值相同（例如，类型为 `A → B → C` 的函数，其命名方式如同它是类型 `C` 的一个项）。
4. 所有其他 `Type` 的项（基本上就是其余的一切）使用 `lowerCamelCase`。
5. 当一个以 `UpperCamelCase` 命名的事物作为某个以 `snake_case` 命名的事物的一部分时，它以 `lowerCamelCase` 引用。
6. 像 `LE` 这样的首字母缩写词作为一个整体写成全大写或全小写，取决于其首字符本应是什么。
7. 规则 1-6 以同样的方式适用于结构的字段或归纳类型的构造子。

为保持局部命名的对称性,存在一些罕见的例外:例如,我们使用 `Ne` 而非 `NE`,以遵循 `Eq` 的范例;`outParam` 的输出是 `Sort` 但并不采用 `UpperCamelCase`。其他一些例外包括区间（`Set.Icc`、`Set.Iic` 等），其

中 I 被大写，尽管按照约定它本应是 lowerCamelCase。任何此类例外都应在 Zulip 上讨论。

示例

```
-- follows rule 2
structure OneHom (M : Type _) (N : Type _) [One M] [One N] where
  toFun : M → N -- follows rule 4 via rule 3 and rule 7
  map_one' : toFun 1 = 1 -- follows rule 1 via rule 7

-- follows rule 2 via rule 3
class CoeIsOneHom [One M] [One N] : Prop where
  coe_one : (↑(1 : M) : N) = 1 -- follows rule 1 via rule 6

-- follows rule 1 via rule 3
theorem map_one [OneHomClass F M N] (f : F) : f 1 = 1 := sorry

-- follows rules 1 and 5
theorem MonoidHom.toOneHom_injective [MulOneClass M] [MulOneClass N] :
  Function.Injective (MonoidHom.toOneHom : (M →* N) → OneHom M N) := sorry

-- follows rule 2
class HPow (α : Type u) (β : Type v) (γ : Type w) where
  hPow : α → β → γ -- follows rule 3 via rule 6; note that rule 5 does not apply

-- follows rules 2 and 6
class LT (α : Type u) where
  lt : α → α → Prop -- this is an exception to rule 2

-- follows rules 2 (for `Semifield`) and 4 (for `toIsField`)
theorem Semifield.toIsField (R : Type u) [Semifield R] :
    IsField R -- follows rule 2

-- follows rules 1 and 6
theorem gt_iff_lt [LT α] {a b : α} : a > b ↔ b < a := sorry

-- follows rule 2; `Ne` is an exception to rule 6
class NeZero : Prop := sorry

-- follows rules 1 and 5
```

```
theorem neZero_iff {R : Type _} [Zero R] {n : R} : NeZero n ↔ n ≠ 0 := sorry
```

拼写

声明名使用美式英语拼写。因此，例如我们使用 `factorization`、`Localization` 和 `FiberBundle`，而不使用 `factorisation`、`Localisation` 或 `FibreBundle`。这与文档的规则形成对比，文档允许使用其他常见的英语拼写。

符号的名称

在将定理的陈述翻译成文字时，常使用下面的对照表。

逻辑

symbol	shortcut	name	notes
∨	\or	or	
∧	\and	and	
→	\r	of / imp	结论先陈述，假设常常省略
↔	\iff	iff	有时连同 iff 的右侧一并省略
¬	\n	not	
∃	\ex	exists / bex	bex 表示“bounded exists”（有界存在）
∀	\fo	all / forall / ball	ball 表示“bounded forall”（有界全称）
=		eq	常常省略
≠	\ne	ne	
∘	\o	comp	

`ball` 与 `bex` 仍在 Lean core 中使用，但不应在 mathlib 中使用。

集合

symbol	shortcut	name	notes
∈	\in	mem	
∉	\notin	notMem	
∪	\cup	union	
∩	\cap	inter	

symbol	shortcut	name	notes
⋃	\bigcup	iUnion / biUnion	i 表示“indexed”（带索引），bi 表示“bounded indexed”（有界带索引）
⋂	\bigcap	iInter / biInter	i 表示“indexed”（带索引），bi 表示“bounded indexed”（有界带索引）
$⋃_0$	\bigcup\0	sUnion	s 表示“set”（集合）
$⋂_0$	\bigcap\0	sInter	s 表示“set”（集合）
\	\\	sdiff	
ᶜ	\^c	compl	
{x | p x}		setOf	
{x}		singleton	
{x, y}		pair	

代数

symbol	shortcut	name	notes
0		zero	
+		add	
-		neg / sub	neg 用于一元函数，sub 用于二元函数
1		one	
*		mul	
^		pow	
/		div	
•	\bu	smul	
$^{-1}$	\-1	inv	
⅟	\frac1	invOf	
∣	\\|	dvd	
∑	\sum	sum	
∏	\prod	prod	

格

symbol	shortcut	name	notes
<		lt / gt	
≤	\le	le / ge	

symbol	shortcut	name	notes
⊔	\sup	sup	二元运算符
⊓	\inf	inf	二元运算符
⨆	\supr	iSup / biSup / ciSup	c 表示“conditionally complete”（条件完备）
⨅	\infi	iInf / biInf / ciInf	c 表示“conditionally complete”（条件完备）
⊥	\bot	bot	
⊤	\top	top	

符号 ≤ 与 < 有一个特殊的命名约定。在 mathlib 中，我们几乎总是使用 ≤ 与 < 而非 ≥ 与 >，因此我们可以用 le/lt 和 ge/gt 来为 ≤ 与 < 命名。使用 ge/gt 有几个理由：

1. 当 ≤ 或 < 的参数以不同顺序出现时，我们使用 ge/gt。对于定理名中第一次出现的 ≤/<，我们使用 le/lt，随后用 ge/gt 来表示参数被交换了。
2. 我们使用 ge/gt 以匹配另一个关系（如 = 或 ≠）的参数顺序。
3. 我们使用 ge/gt 来描述参数被交换后的 ≤ 或 < 关系。
4. 当 ≤ 或 < 的第二个参数“更具变动性”时，我们使用 ge/gt。

```
-- follows rule 1
theorem lt_iff_le_not_ge [Preorder α] {a b : α} : a < b ↔ a ≤ b ∧ ¬b ≤ a := sorry
theorem not_le_of_gt [Preorder α] {a b : α} (h : a < b) : ¬b ≤ a := sorry
theorem LT.lt.not_ge [Preorder α] {a b : α} (h : a < b) : ¬b ≤ a := sorry

-- follows rule 2
theorem Eq.ge [Preorder α] {a b : α} (h : a = b) : b ≤ a := sorry
theorem ne_of_gt [Preorder α] {a b : α} (h : b < a) : a ≠ b := sorry

-- follows rule 3
theorem ge_trans [Preorder α] {a b : α} : b ≤ a → c ≤ b → c ≤ a := sorry

-- follows rule 4
theorem le_of_forall_gt [LinearOrder α] {a b : α} (H : ∀ (c : α), a < c → b < c) : b ≤ a :
```

强制转换

强制转换（coercion）以其底层函数命名。

```
-- Named after `Subtype.val`
theorem Subtype.val_injective {p : α → Prop} : ((↑) : {a : α // p a} → α).Injective := sor

-- Named after `ENNReal.ofNNReal`
theorem ENNReal.ofNNReal_injective : ((↑) : ℝ≥0 → ℝ≥0∞).Injective := sorry

-- Named after `DFunLike.coe`
theorem DFunLike.coe_injective {F α : Sort*} {β : α → Sort*} [DFunLike F α β] :
    ((↑) : F → ∀ a, β a).Injective := sorry

-- Named after `SetLike.coe`
theorem SetLike.coe_injective {α β : Type*} [SetLike α β] :
    ((↑) : α → Set β).Injective := sorry
```

当类型支持多种自然的强制转换时，这有助于消歧义，其动机在于强制转换是可约的（reducible）。在 Lean 3 中并非如此，因此许多名称仍然是错误的。

点号

点号用于命名空间，也用于自动生成的名称，例如递归子（recursor）、消去子（eliminator）和结构投影。它们也可以手动引入，例如在投影记法很有用的场合。因此，它们用于以下所有情形。

注意：由于 `And` 是一个（值为 `Prop` 的二元函数），按照命名约定它采用 `UpperCamelCase`，因此其命名空间为 `And.*`。这看起来似乎与对照表中 `∧ –> and` 相矛盾，但因为大驼峰式的类型在定理名中出现时会被转为小驼峰式，所以该对照表整体上仍然有效。同样的情形适用于 `Or`、`Iff`、`Not`、`Eq`、`HEq`、`Ne` 等。

逻辑联结词的引入（intro）、消去（elim）和析构（destruct）规则，无论它们是否自动生成：

- `And.intro`
- `And.elim`
- `And.left`
- `And.right`
- `Or.inl`
- `Or.inr`
- `Or.intro_left`
- `Or.intro_right`
- `Iff.intro`
- `Iff.elim`
- `Iff.mp`
- `Iff.mpr`

- Not.intro
- Not.elim
- Eq.refl
- Eq.rec
- Eq.subst
- HEq.refl
- HEq.rec
- HEq.subst
- Exists.intro
- Exists.elim
- True.intro
- False.elim

投影记法有用的场合，例如：

- And.symm
- Or.symm
- Or.resolve_left
- Or.resolve_right
- Eq.symm
- Eq.trans
- HEq.symm
- HEq.trans
- Iff.symm
- Iff.refl

即使对于并非归纳类型的类型，使用点号记法也是有益的。例如，我们使用：

- LE.trans
- LT.trans_le
- LE.trans_lt

公理化描述

某些定理使用公理化名称来描述，而非描述其结论。

- def（用于展开定义）
- refl
- irrefl
- symm
- trans

- `antisymm`
- `asymm`
- `congr`
- `comm`
- `assoc`
- `left_comm`
- `right_comm`
- `mul_left_cancel`
- `mul_right_cancel`
- `inj`（injective，单射）

变量约定

- `u`、`v`、`w`、……用于宇宙
- `α`、`β`、`γ`、……用于一般类型
- `a`、`b`、`c`、……用于命题
- `x`、`y`、`z`、……用于一般类型的元素
- `h`、h_1、……用于假设
- `p`、`q`、`r`、……用于谓词和关系
- `s`、`t`、……用于列表
- `s`、`t`、……用于集合
- `m`、`n`、`k`、……用于自然数
- `i`、`j`、`k`、……用于整数

具有数学内涵的类型用通常的数学记法表示，常常使用大写字母（`G` 表示群，`R` 表示环，`K` 或 `𝕜` 表示域，`E` 表示向量空间，……）。在较旧的文件中并未遵循这一约定，那里所有类型都使用希腊字母。欢迎提交重命名这些文件中类型变量的拉取请求。

标识符与定理名

我们采用以下命名准则，以便用户更容易猜出定理的名称，或借助 tab 补全找到它。诸如合取或析取这样的运算的常见“公理化”性质，被放在以该运算名称开头的命名空间中：

```
import Mathlib.Logic.Basic

#check And.comm
#check Or.comm
```

特别地，这包括逻辑联结词的 `intro` 和 `elim` 运算，以及关系的性质：

```
import Mathlib.Logic.Basic

#check And.intro
#check And.elim
#check Or.intro_left
#check Or.intro_right
#check Or.elim

#check Eq.refl
#check Eq.symm
#check Eq.trans
```

然而请注意，对于公理化的逻辑和算术运算，我们并不这样做。

```
import Mathlib.Algebra.Group.Basic

#check and_assoc
#check mul_comm
#check mul_assoc
#check @mul_left_cancel  -- multiplication is left cancelative
```

不过在大多数情况下，我们依赖描述性名称。定理的名称往往直接描述其结论：

```
import Mathlib.Algebra.Ring.Basic
open Nat
#check succ_ne_zero
#check mul_zero
#check mul_one
#check @sub_add_eq_add_sub
#check @le_iff_lt_or_eq
```

如果描述的一个前缀就足以传达含义，名称可以变得更短：

```
import Mathlib.Algebra.Ring.Basic

#check @neg_neg
#check Nat.pred_succ
```

当一个运算以中缀形式书写时，定理名也随之而行。例如，我们写 `neg_mul_neg` 而非 `mul_neg_neg`，以描述模式 `-a * -b`。

有时，为消除定理名称的歧义或更好地传达其意图，有必要描述某些假设。词“of”用于分隔这些假设：

```
import Mathlib.Algebra.Order.Monoid.Lemmas
```

```
open Nat

#check lt_of_succ_le
#check lt_of_not_ge
#check lt_of_le_of_ne
#check add_lt_add_of_lt_of_le
```

这些假设按它们出现的顺序列出，而非逆序。例如，定理 `A → B → C` 将被命名为 `C_of_A_of_B`。

有时缩写或替代描述更便于使用。例如, 我们使用 pos、neg、nonpos、nonneg 而非 `zero_lt`、`lt_zero`、`le_zero` 和 `zero_le`。

```
import Mathlib.Algebra.Order.Monoid.Lemmas
import Mathlib.Algebra.Order.Ring.Lemmas

open Nat

#check mul_pos
#check mul_nonpos_of_nonneg_of_nonpos
#check add_lt_of_lt_of_nonpos
#check add_lt_of_nonpos_of_lt
```

这些约定并不完美。它们无法区分仅在结合性上不同的复合表达式，也无法区分某个模式中重复出现的部分。对此，我们只能尽力而为。例如，`a + b - b = a` 既可命名为 `add_sub_self`，也可命名为 `add_sub_cancel`。

有时词“left”或“right”有助于描述一个定理的不同变体。

```
import Mathlib.Algebra.Order.Monoid.Lemmas
import Mathlib.Algebra.Order.Ring.Lemmas

open Nat

#check add_le_add_left
#check add_le_add_right
#check le_of_mul_le_mul_left
#check le_of_mul_le_mul_right
```

当在某个引理名中引用一个不在同一命名空间下的、带命名空间的定义时，应去掉该定义的命名空间。如果去掉命名空间后定义名仍无歧义，则可直接使用。否则，将命名空间以 `lowerCamelCase` 形式重新加回。这是为了确保引理名中由 _ 分隔的字符串对应于某个定义名或联结词。

```
import Mathlib.Data.Int.Cast.Basic
```

```
import Mathlib.Data.Nat.Cast.Basic
import Mathlib.Topology.Constructions

#check Prod.fst
#check continuous_fst

#check Nat.cast
#check map_natCast
#check Int.cast_natCast
```

结构性引理的命名

我们正力图为某些结构性引理标准化特定的命名模式。

外延性

形如 `(∀ x, f x = g x) → f = g` 的引理应命名为 `.ext`，并标注 `@[ext]` 属性。这类引理常常可以通过在某个结构上标注 `@[ext]` 属性来自动生成。（不过，自动生成的引理总是以结构投影来表述，而往往存在一个更好的陈述，例如使用强制转换的陈述，此时应手动书写并标注 `@[ext]`。）

形如 `f = g ↔ ∀ x, f x = g x` 的引理应命名为 `.ext_iff`。

单射性

在可能的情况下，单射性引理应以 `Function.Injective f` 作为结论来表述，并使用完整单词 `injective`，通常形如 `f_injective`。形式 `injective_f` 在 mathlib 中仍频繁出现。

除此之外，通常还应提供一个双向蕴含的变体，例如形如 `f x = f y ↔ x = y`，它可由 `Function.Injective.eq_iff` 得到。此类引理应命名为 `f_inj`（不过若它们位于合适的命名空间中，`.inj` 也很好）。双向单射性引理常常是适合标注 `@[simp]` 的候选。mathlib 中仍有许多名为 `inj` 的单向蕴含，在遇到它们时予以更新和替换是合理的。

不过请注意，归纳类型的构造子有自动生成的单向蕴含，命名为 `.inj`，并且无意更改这一点。当这样一个自动生成的引理已经存在，而又需要一个双向引理时，可将其命名为 `.inj_iff`。

使用“left”或“right”的单射性引理应指向那个“发生变化”的参数。例如，陈述为 `a - b = a - c ↔ b = c` 的引理可命名为 `sub_right_inj`。

归纳与递归原理

归纳/递归原理是为某个类型 `T` 的所有元素构造数据或证明的方式，其途径是提供在更受约束的特定情形下构造此数据或证明的方法。这些原理应被表述为接受一个 `motive` 参数，它声明了我们要对所有 `T` 证明什么性质或构造什么数据。当 motive 消去到 `Prop` 时，它是一个归纳原理，名称中应包含 `induction`。另一方面，当 motive 消去到 `Sort u` 或 `Type u` 时，它是一个递归原理，名称中应改为包含 `rec`。

此外，当且仅当在参数顺序中值先于构造出现时，名称中才应包含 `on`。

下表概括了这些命名约定：

motive 消去到：	`Prop`	`Sort u` 或 `Type u`
值在先	`T.induction_on`	`T.recOn`
构造在先	`T.induction`	`T.rec`

必要时（例如为消歧义）可对这些名称做变体。

作为后缀的谓词

大多数谓词应作为前缀添加。例如 `IsClosed (Icc a b)` 应命名为 `isClosed_Icc`，而非 `Icc_isClosed`。

某些广泛使用的谓词不遵循这一规则。它们是那些与某个已按命名约定加上后缀的原子相类似的谓词。以下是一个非穷尽的列表：* 我们用 `_inj` 表示 `f a = f b ↔ a = b`，因此也用 `_injective` 表示 `Injective f`、`_surjective` 表示 `Surjective f`、`_bijective` 表示 `Bijective f`……* 我们用 `_mono` 表示 `a ≤ b → f a ≤ f b`、`_anti` 表示 `a ≤ b → f b ≤ f a`，因此也用 `_monotone` 表示 `Monotone f`、`_antitone` 表示 `Antitone f`、`_strictMono` 表示 `StrictMono f`、`_strictAnti` 表示 `StrictAnti f` 等等……

作为后缀的谓词之前可以加上 `_left` 或 `_right`，以表明一个二元运算是左单调或右单调的。例如，`mul_left_monotone : Monotone (· * a)` 证明的是乘法的左单调性，而非左乘的单调性。

取值为 Prop 的类

mathlib 有许多取值为 `Prop` 的类以及其他定义。例如“设 R 为一个拓扑环”写作 `variable (R : Type*) [Ring R] [TopologicalSpace R] [IsTopologicalRing R]`，而“设 G 为一个群，H 为一个正规子群”写作 `variable (G : Type*) [Group G] (H : Subgroup G) [Normal H]`。这里 `IsTopologicalRing R` 与 `Normal H` 并非额外的数据，而是对我们已有数据所施加的额外假设。

对于这些取值为 Prop 的类，mathlib 目前力图遵循以下命名约定。如果该类是一个名词，则其名称应以 Is 开头。然而如果它是一个形容词，则其名称无需以 Is 开头。例如，对于“正规子群”类型类，IsNormal 是可接受的，但 Normal 也可以；在非正式语言中我们会说“假设子群 H 是正规的”。然而对于“拓扑环”类型类，IsTopologicalRing 更受青睐，因为我们在非正式语言中不会说“假设环 R 是拓扑的”。

函数的未展开形式与展开形式

两个函数 f 与 g 的乘法可以等价地记作 f * g 或 fun x ↦ f x * g x。这两个表达式在定义上相等，但在语法上不相等（且它们在索引树中不共享同一键），这意味着诸如 rw、fun_prop 或 apply? 之类的工具不会将一种形式的定理用于另一种形式的表达式。因此，有时同时提供使用这两种形式的陈述变体是方便的。如果需要区分它们，涉及第一种未展开形式的陈述仅用 mul 来书写，而使用第二种展开形式的陈述则应改用 fun_mul。如果由于某个引理只以展开形式给出而无需消歧义，则前缀 fun_ 不是必需的。

例如，两个连续函数的乘法是连续的这一事实为

```
theorem Continuous.fun_mul (hf : Continuous f) (hg : Continuous g) : Continuous fun x ↦ f
```

以及

```
theorem Continuous.mul (hf : Continuous f) (hg : Continuous g) : Continuous (f * g)
```

这两个定理都值得标注 fun_prop 属性。

加法、减法、取负、幂以及函数的复合也是如此。

库风格指南

除了命名约定之外，Lean 库中的文件通常还遵循以下指南和约定。统一的风格使得浏览库和阅读内容更加容易，但这些只是指南，而非僵化的规则。

变量约定

- `u`、`v`、`w`、……用于宇宙
- `α`、`β`、`γ`、……用于一般类型
- `a`、`b`、`c`、……用于命题
- `x`、`y`、`z`、……用于一般类型的元素
- `h`、`h₁`、……用于假设
- `p`、`q`、`r`、……用于谓词和关系
- `s`、`t`、……用于列表
- `s`、`t`、……用于集合
- `m`、`n`、`k`、……用于自然数
- `i`、`j`、`k`、……用于整数

具有数学内涵的类型使用通常的数学记号来表示，通常用大写字母（`G` 表示群，`R` 表示环，`K` 或 `𝕜` 表示域，`E` 表示向量空间，……）。在较旧的文件中并未遵循这一约定，那里所有类型都使用希腊字母。我们欢迎对这些文件中类型变量进行重命名的拉取请求。

Unicode 用法

请使用特殊的 Unicode 字符，尤其是数学符号，凡是有助于形成良好记号并提升可读性之处皆可使用。请避免使用以下字符：- 改变文本方向的字符 - 不可见字符（空格和换行符除外）- 修饰其他字符的字符

Mathlib 有一个 linter，它会对照允许列表检查所有字符，必要时可向该列表添加新字符。

行长度

每行长度不应超过 100 个字符。这使得文件更易于阅读，尤其是在小屏幕或小窗口中。如果你使用 VS Code 进行编辑，会有一个指示 100 字符上限的可视标记。

文件头与导入

文件头应包含版权信息、所有对该文件做出重要贡献的作者列表，以及对内容的描述。将 `module` 关键字单独放在文件头之后的一行，空一行，然后将所有 `public import` 归为一组，再空一行，然后将所有 `import` 归为一组。请尽量在每个导入块内保持字母顺序。

```
/-
Copyright (c) 2024 Joe Cool. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Joe Cool
-/
module

public import Mathlib.Logic.Defs

import Mathlib.Algebra.Group.Defs
import Mathlib.Data.Nat.Basic
```

（提示：如果你在 VS Code 中编辑 mathlib，可以输入 `copy` 然后按 TAB 来生成版权头的框架。）

关于作者列表：即使只有一位作者，也使用 `Authors`。行尾不要加句点，并使用逗号（`,`）分隔所有作者姓名（因此不要在倒数第二位与最后一位作者之间使用 `and`）。我们对于哪些贡献有资格列入其中并没有严格的规则。总的思路是，列在那里的人应当是当我们对该 Lean 代码的设计或开发有疑问时会去联系的人。

注意还有一些不太常见的关键字组合，例如 `public meta import` 或 `import all`。鉴于它们较为罕见，我们尚未规定它们在导入语句中的相对位置。

模块文档字符串

在版权头和导入之后，请添加一个模块文档字符串（以 `/-!` 和 `-/` 分隔），其中包含

- 文件的标题，
- 内容的摘要（主要定义和定理、证明技巧等……）
- 文件中使用的记号（如果有的话）
- 对文献的引用（如果有的话）

总的来说，模块文档字符串应当看起来像这样：

```
/-!
# Foos and bars

In this file we introduce `foo` and `bar`,
two main concepts in the theory of xyzzyology.

## Main results

- `exists_foo`: the main existence theorem of `foo`s.
- `bar_of_foo_of_baz`: a construction of a `bar`, given a `foo` and a `baz`.
  If this doc-string is longer than one line, subsequent lines should be indented by two s
  (as required by markdown syntax).
- `bar_eq`    : the main classification theorem of `bar`s.

## Notation

 - `|_|` : The barrification operator, see `bar_of_foo`.

## References

See [Thales600BC] for the original account on Xyzzyology.
-/
```

新的参考文献条目应添加到 `docs/references.bib` 中。

更多建议和示例请参阅我们的文档要求。

定义和定理的结构

所有声明（例如 `def`、`lemma`、`theorem`、`class`、`structure`、`inductive`、`instance` 等）和命令（例如 `variable`、`open`、`section`、`namespace`、`notation` 等）都被视为顶层内容，这些词应当在文档中靠左对齐。特别地，打开一个命名空间或节并不会导致该命名空间或节的内容缩进。（注意：在 VS Code 中，将鼠标悬停在任何声明上，例如 `def Foo ...`，会显示完全限定的名称，例如若 `Foo` 是在命名空间 `MyNamespace` 打开的情况下声明的，则显示 `MyNamespace Foo`。）

这些指南适用于以 `def`、`lemma` 和 `theorem` 开头的声明。对于“定理陈述”，也应理解为“定义的类型”；对于“证明”，也应理解为“定义体”。

在“:”、“:=”或中缀运算符两侧使用空格。将它们置于换行之前，而不是放在下一行的开头。

下文中，凡是未明确指出量的“缩进”都意为“额外缩进 2 个空格”。

在陈述定理之后，我们将后续证明中的各行缩进 2 个空格。

```
open Nat
theorem nat_case {P : Nat → Prop} (n : Nat) (H1 : P 0) (H2 : ∀ m, P (succ m)) : P n :=
  Nat.recOn n H1 (fun m IH ↦ H2 m)
```

如果定理陈述需要多行，则将后续各行缩进 4 个空格。证明仍然只缩进 2 个空格（而非 6 = 4 + 2）。当以策略模式提供证明时，by 放在第一个策略之前的那一行；然而，by 不应单独占一行。实践中这意味着你会经常在定理陈述末尾看到 `:= by`。

```
import Mathlib.Data.Nat.Basic

theorem le_induction {P : Nat → Prop} {m}
    (h0 : P m) (h1 : ∀ n, m ≤ n → P n → P (n + 1)) :
    ∀ n, m ≤ n → P n := by
  apply Nat.le.rec
  · exact h0
  · exact h1 _

def decreasingInduction {P : ℕ → Sort*} (h : ∀ n, P (n + 1) → P n) {m n : ℕ} (mn : m ≤ n)
    (hP : P n) : P m :=
  Nat.leRecOn mn (fun {k} ih hsk => ih <| h k hsk) (fun h => h) hP
```

当一个证明项接受多个参数时，有时将某些参数放在后续行上会更清晰，且往往是必要的。在这种情况下，缩进每个参数。更一般地，每当一个项跨越多行时，都适用这条规则，即额外缩进 2 个空格。

```
open Nat
axiom zero_or_succ (n : Nat) : n = zero ∨ n = succ (pred n)
theorem nat_discriminate {B : Prop} {n : Nat} (H1: n = 0 → B) (H2 : ∀ m, n = succ m → B) :
  Or.elim (zero_or_succ n)
    (fun H3 : n = zero ↦ H1 H3)
    (fun H3 : n = succ (pred n) ↦ H2 (pred n) H3)
```

不要让括号成为“孤儿”；让它们与其参数保持在一起。

这里有一个较长的示例。

```
import Mathlib.Init.Data.List.Lemmas

open List
variable {T : Type}
```

```
theorem mem_split {x : T} {l : List T} : x ∈ l → ∃ s t : List T, l = s ++ (x :: t) :=
  List.recOn l
    (fun H : x ∈ [] ↦ False.elim ((mem_nil_iff _).mp H))
    (fun y l ↦
      fun IH : x ∈ l → ∃ s t : List T, l = s ++ (x :: t) ↦
      fun H : x ∈ y :: l ↦
      Or.elim (eq_or_mem_of_mem_cons H)
        (fun H1 : x = y ↦
          Exists.intro [] (Exists.intro l (by rw [H1]; rfl)))
        (fun H1 : x ∈ l ↦
          let ⟨s, (H2 : ∃ t : List T, l = s ++ (x :: t))⟩ := IH H1
          let ⟨t, (H3 : l = s ++ (x :: t))⟩ := H2
          have H4 : y  ::  l = (y :: s) ++ (x :: t) := by rw [H3]; rfl
          Exists.intro (y :: s) (Exists.intro t H4)))
```

声明的所有参数的类型都应显式给出，即使 Lean 能够自行推断出这些类型信息。这使得在像 GitHub 这样的网页上看到该定义时更易于理解。出于同样的原因，所有声明的返回类型也应给出（Lean 仅对定理强制要求这一点）。因此你应当遵循此例中 `GoodStatement` 的风格：

```
def BadStatement (n) := ∃ k, n + k = 3
def GoodStatement (n : ℕ) : Prop := ∃ k : ℕ, n + k = 3
```

一个简短的声明可以写在单行上：

```
open Nat
theorem succ_pos : ∀ n : Nat, 0 < succ n := zero_lt_succ

def square (x : Nat) : Nat := x * x
```

当论证简短时，可以将 `have` 放在单行上。

```
example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k := ne_of_lt h
  ...
```

当论证过长时，你应当将其放在下一行，额外缩进 2 个空格。

```
example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k :=
    ne_of_lt h
  ...
```

当 `have` 的论证使用策略模式时，无论论证是否跨越多行，`by` 都应放在同一行上。

```
example (n k : Nat) (h : n < k) : ... :=
```

```
  have h1 : n ≠ k := by apply ne_of_lt; exact h
  ...

example (n k : Nat) (h : n < k) : ... :=
  have h1 : n ≠ k := by
    apply ne_of_lt
    exact h
  ...
```

当参数本身长到需要换行时，对第一行之后的每一行都使用额外的缩进，如下例所示:

```
import Mathlib.Data.Nat.Basic

theorem Nat.add_right_inj {n m k : Nat} : n + m = n + k → m = k :=
  Nat.recOn n
    (fun H : 0 + m = 0 + k ↦ calc
      m = 0 + m := Eq.symm (zero_add m)
      _ = 0 + k := H
      _ = k     := zero_add _)
    (fun (n : Nat) (IH : n + m = n + k → m = k) (H : succ n + m = succ n + k) ↦
      have H2 : succ (n + m) = succ (n + k) := calc
        succ (n + m) = succ n + m   := Eq.symm (succ_add n m)
        _            = succ n + k   := H
        _            = succ (n + k) := succ_add n k
      have H3 : n + m = n + k := succ.inj H2
      IH H3)
```

在类或结构定义中，字段缩进 2 个空格，此外每个字段都应有一个文档字符串，如下所示:

```
structure PrincipalSeg {α β : Type*} (r : α → α → Prop) (s : β → β → Prop) extends r ≺r s
  /-- The supremum of the principal segment -/
  top : β
  /-- The image of the order embedding is the set of elements `b` such that `s b top` -/
  down' : ∀ b, s b top ↔ ∃ a, toRelEmbedding a = b

class Module (R : Type u) (M : Type v) [Semiring R] [AddCommMonoid M] extends
    DistribMulAction R M where
  /-- Scalar multiplication distributes over addition from the right. -/
  protected add_smul : ∀ (r s : R) (x : M), (r + s) • x = r • x + s • x
  /-- Scalar multiplication by zero gives zero. -/
  protected zero_smul : ∀ x : M, (0 : R) • x = 0
```

后续的声明之间应以单个换行符分隔。对于一组类似的单行声明则可作例外。

```
theorem foo : True :=
  sorry

theorem bar : False :=
  sorry

@[simp] theorem one_lt_two : 1 < 2 := sorry
@[simp] theorem two_lt_three : 2 < 3 := sorry
```

在定义中使用一个接受多个参数的构造子时，各参数对齐排列，如下所示:

```
theorem Ordinal.sub_eq_zero_iff_le {a b : Ordinal} : a - b = 0 ↔ a ≤ b :=
  ⟨fun h => by simpa only [h, add_zero] using le_add_sub a b,
   fun h => by rwa [← Ordinal.le_zero, sub_le, add_zero]⟩
```

`@[to_additive]` 和 `@[to_dual]` 属性是两种自动化机制，它们分别为代数中的乘法陈述生成相应的加法版本，以及为序论/范畴论中的陈述生成相应的对偶版本。在适用的情况下，应当使用这些属性，而不是手写第二个陈述。

```
@[to_additive] -- generates `add_rotate`
theorem mul_rotate {G : Type*} (a b c : G) : a * b * c = b * c * a := sorry

-- `to_additive` can't be used here because `ℝ` can't be additivised.
theorem mul_rotate' (a b c : ℝ) : a * b * c = b * c * a := sorry
```

实例

在提供结构的项或类的实例时，应当使用 `where` 语法，以避免需要包围的花括号，如下所示:

```
instance instOrderBot : OrderBot ℕ where
  bot := 0
  bot_le := Nat.zero_le
```

如果已经存在一个实例 `instBot`，则可以写成

```
instance instOrderBot : OrderBot ℕ where
  __ := instBot
  bot_le := Nat.zero_le
```

冒号左侧的假设

一般而言，如果证明以引入这些变量开始，那么将参数放在冒号左侧比放在全称量词或蕴含式中更受青睐。例如：

```
example (n : ℝ) (h : 1 < n) : 0 < n := by linarith
```

优于

```
example (n : ℝ) : 1 < n → 0 < n := fun h ↦ by linarith
```

而

```
example (n : ℕ) : 0 ≤ n := Nat.zero_le n
```

优于

```
example : ∀ (n : ℕ), 0 ≤ n := Nat.zero_le
```

注意模式匹配并不算作证明以引入变量开始。例如，下面就是将假设放在冒号右侧的一个有效用例：

```
lemma zero_le : ∀ n : ℕ, 0 ≤ n
  | 0 => le_rfl
  | n + 1 => add_nonneg (zero_le n) zero_le_one
```

绑定子

在绑定子之后使用空格。此外，一般应显式写出绑定子的类型，即使 Lean 并不需要这一信息。

```
example : ∀ α : Type, ∀ x : α, ∃ y : α, y = x :=
  fun (α : Type) (x : α) ↦ Exists.intro x rfl
```

匿名函数

Lean 为声明匿名函数提供了若干优雅的语法选项。对于非常简单的函数，可以使用居中的点作为函数参数，例如用 `( · ^ 2)` 表示平方函数。然而，有时需要按名称引用参数（例如，如果它们在函数体中出现多次）。Lean 对此的默认写法是 `fun x => x * x`，但 `↦` 箭头（通过 `\mapsto` 输入）同样有效。在 mathlib 中，美观打印器显示 `↦`，我们在源代码中也稍微更倾向于使用它。lambda 记号 `λ x ↦ x * x` 虽然在语法上有效，但在 mathlib 中不被允许，而应使用 `fun` 关键字。

计算

在如何书写计算式证明上有一定的灵活性，尽管 `calc` 本身的语法要求会强制施加一些规则。不过，仍有一些一般性的指南。

与 `by` 一样，`calc` 关键字应放在计算开始之前的那一行上，而计算内容则缩进。无论涉及哪些关系（例如 `=` 或 `≤`），都应在各行之间对齐。用作各项占位符以指示计算延续的下划线 `_` 应当左对齐。

至于论证部分，不必对齐 `:=` 符号，但如果表达式足够短，对齐会显得美观。第一个关系两侧的项既可以放在一行，也可以放在不同行上，这可根据表达式的大小来决定。

下面是一个适当风格的示例，它能更轻松地容纳较长的表达式：

```
import Init.Data.List.Basic

open List

theorem reverse_reverse : ∀ (l : List α), reverse (reverse l) = l
  | []     => rfl
  | a :: l => calc
    reverse (reverse (a :: l))
      = reverse (reverse l ++ [a]) := by rw [reverse_cons]
    _ = reverse [a] ++ reverse (reverse l) := reverse_append _ _
    _ = reverse [a] ++ l := by rw [reverse_reverse l]
    _ = a :: l := rfl
```

下面这种风格有一个显著优势，即所有行都可互换，这在例如 VSCode 中编辑证明时尤为有用：

```
import Init.Data.List.Basic

open List

theorem reverse_reverse : ∀ (l : List α), reverse (reverse l) = l
  | []     => rfl
  | a :: l => calc
        reverse (reverse (a :: l))
    _ = reverse (reverse l ++ [a]) := by rw [reverse_cons]
    _ = reverse [a] ++ reverse (reverse l) := reverse_append _ _
    _ = reverse [a] ++ l := by rw [reverse_reverse l]
    _ = a :: l := rfl
```

如果表达式和证明都相对较短，下面这种风格也是一个选择：

```
import Init.Data.List.Basic
```

```
open List

theorem reverse_reverse : ∀ (l : List α), reverse (reverse l) = l
  | []     => rfl
  | a :: l => calc
    reverse (reverse (a :: l)) = reverse (reverse l ++ [a])         := by rw [reverse_cons
    _                          = reverse [a] ++ reverse (reverse l) := reverse_append _ _
    _                          = reverse [a] ++ l                   := by rw [reverse_reve
    _                          = a :: l                             := rfl
```

策略模式

正如我们已经提到的，在打开一个策略块时，by 放在策略块开始所在行之前那一行的末尾，而不是单独占一行。策略块内的所有内容都缩进，如下所示:

```
theorem continuous_uncurry_of_discreteTopology [DiscreteTopology α] {f : α → β → γ}
    (hf : ∀ a, Continuous (f a)) : Continuous (uncurry f) := by
  apply continuous_iff_continuousAt.2
  rintro ⟨a, x⟩
  change map _ _ ≤ _
  rw [nhds_prod_eq, nhds_discrete, Filter.map_pure_prod]
  exact (hf a).continuousAt
```

可以混用项模式和策略模式，如下所示:

```
theorem Units.isUnit_units_mul {M : Type*} [Monoid M] (u : Mˣ) (a : M) :
    IsUnit (↑u * a) ↔ IsUnit a :=
  Iff.intro
    (fun ⟨v, hv⟩ => by
      have : IsUnit (↑u⁻¹ * (↑u * a)) := by exists u⁻¹ * v; rw [← hv, Units.val_mul]
      rwa [← mul_assoc, Units.inv_mul, one_mul] at this)
    u.isUnit.mul
```

当新目标作为附带条件或步骤出现时，它们被缩进，并在前面加上一个聚焦点 ·（通过 \. 输入）；该点本身不缩进。

```
import Mathlib.Algebra.Group.Basic

theorem exists_npow_eq_one_of_zpow_eq_one' [Group G] {n : ℤ} (hn : n ≠ 0) {x : G} (h : x ^
    ∃ n : ℕ, 0 < n ∧ x ^ n = 1 := by
```

```
  cases n
  · simp only [Int.ofNat_eq_coe] at h
    rw [zpow_ofNat] at h
    refine ⟨_, Nat.pos_of_ne_zero fun n0 ↦ hn ?_, h⟩
    rw [n0]
    rfl
  · rw [zpow_negSucc, inv_eq_one] at h
    refine ⟨_ + 1, Nat.succ_pos _, h⟩
```

某些策略，例如 `refine`，可以创建具名的子目标，这些子目标可以使用 `case` 以任意所需的顺序证明。这一特性也有助于提升可读性。然而，并不强制要求使用它来代替聚焦点 `·`。

```
example {p q : Prop} (h₁ : p → q) (h₂ : q → p) : p ↔ q := by
  refine ⟨?imp, ?converse⟩
  case converse => exact h₂
  case imp => exact h₁
```

经常使用 `t0 <;> t1` 来执行 `t0`，然后对所有新目标执行 `t1`。要么将这些策略写在一行，要么将后续策略缩进。

```
  cases x <;>
    simp [a, b, c, d]
```

对于单行策略证明（或嵌入在项中的简短策略证明），可以使用 `by tac1; tac2; tac3`，用分号代替换行加缩进。

一般而言, 你应当每行只放一个策略调用, 除非你正在用一个完全能放进单行的证明来闭合目标。对应于单个数学思想的简短策略序列也可以放在一行, 用分号分隔, 例如 `cases bla; clear h` 或 `induction n; simp` 或 `rw [foo]; simp_rw [bar]`，但即便在这些情形下，仍更倾向于使用换行。

```
example : ... := by
  by_cases h : x = 0
  · rw [h]; exact hzero ha
  · rw [h]
    have h' : ... := H ha
    simp_rw [h', hb]
    ...
```

非常简短的目标可以立即使用 `swap` 或 `pick_goal`（如有需要）来闭合，以避免在证明其余部分中产生额外的缩进。

```
example : ... := by
  rw [h]
  swap; exact h'
```

...

我们通常使用一个空行来分隔定理和定义，但这也可以省略，例如，为了将若干简短的定义归为一组，或将一个定义和记号归为一组。

压缩 simp 调用

除非性能特别差或证明会因此中断，否则终结性 `simp` 调用（如果一个 `simp` 调用闭合了当前目标，或其后仅跟随诸如 `ring`、`field_simp`、`aesop` 之类的灵活策略，则它是终结性的）不应被压缩（即被 `simp?` 的输出所替换）。

主要有两个原因：1. 压缩后的 `simp` 调用可能比对应的未压缩调用长好几行，因而把添加到未压缩 `simp` 调用中用以闭合目标的那些关键引理这一有用信息淹没在大量基础 simp 引理之中。2. 压缩后的 `simp` 调用按名称引用许多引理，这意味着当其中某个引理被重命名时它就会中断。引理重命名发生得足够频繁，以至于这在维护层面上是个值得关注的问题。

性能剖析

在为 mathlib 做贡献时，作者应当注意其贡献对性能的影响。Lean FRO 维护着基准测试基础设施，可以通过在 PR 上评论 `!bench` 来访问。

作者应确保其贡献不会导致显著的性能退化。特别地，如果该 PR 触及了语言的重要组成部分，例如添加新的类、实例或 `simp` 引理、更改导入、创建新定义，或将 `def` 转为 `abbrev`，那么作者应当主动对其更改进行基准测试。尤其是非平凡的 `refactor` PR 应当进行基准测试，任何显著的负面结果都必须在评审过程中加以解释。

透明度与 API 设计

Lean 作为一个实践中高性能的证明助手，其核心在于避免对非常大的项进行定义性相等的检查。在繁释器（elaborator，即将语法转换为项的语言组件）中，透明度（transparency）的概念是避免在不必要时展开大型定义的主要机制。除 `opaque` 定义外，透明度共有三个级别：- `reducible` 定义总是被展开 - `semireducible` 定义（默认）在诸如 `rw` 和 `simp` 之类的主要策略中通常不被展开，但稍加努力即可展开，例如显式调用 `rfl` 或 `erw`。在计算用于将实例存入实例缓存或将 simp 引理存入 simp 缓存的键时，semireducible 定义也不会被展开。- `irreducible` 定义永远不会被展开，除非用户显式请求（例如使用 `unfold` 策略，或使用 `unseal` 命令）。

`def` 默认创建 `semireducible` 定义，`abbrev` 创建 `reducible`（且 `@[inline]`）定义。

在设计定义时，作者应当对定义的透明度级别加以思考。考虑暴露定义的底层项将如何影响实例搜索和化简。mathlib 的默认做法是，定义应当是 `semireducible`，除非有充分的理由不这样做，且该理由

应当在 PR 描述中清楚地说明。这会给贡献者带来额外负担，他们将需要声明形如

```
instance : Foo myDef := inferInstanceAs (Foo underlyingTermOfMyDef)
```

的新实例，并复用 API 引理，尤其是供 `simp` 使用的引理，例如

```
@[simp] lemma myDef_bar_eq_bizz (x : X) : myDef.bar = bizz :=
    underlyingTermOfMyDef_bar_eq_bizz
```

如果意在使 API 边界完全封闭，则使用形如

```
structure myDef where
    underlying : underlyingTerm
```

的类型同义词是库的惯例，以代替 `irreducible` 定义。这些结构包装器意在用于那些与某个现有类型等价、但在数学语义上明显有别的类型，例如 `Option` 和 `WithTop`。

内核并没有类似的透明度概念，因此其展开规则是不同的。在某些情况下，作者也想阻止内核进行展开。Mathlib 为此提供了一个命令 `irreducible_def`。只有当剖析表明确有必要时才应使用它。

在诸如 `simp` 或 `rw` 这类以 reducible 透明度运作的策略之后使用 `erw` 或 `rfl`，表明缺失了某些 API。请考虑向 API 添加必要的引理以避免这种情况。

库中存在着滥用定义性透明度的现有情形，例如 `erw` 和多余的 `rfl`。我们非常欢迎移除这些的 PR，但其 PR 描述应当清楚地说明移除是如何实现的，尤其要说明底层项的变化，并且必须对其更改进行基准测试。请将此视为改进相关组件 API 设计的机会。

空白与定界符

Lean 对空白敏感，一般而言我们选择一种避免为代码加定界符的风格。例如，在书写策略时，可以将它们写成 `tac1; tac2; tac3`，用 `;` 分隔，以覆盖默认的空白敏感性。然而，如上所述，除少数特殊情况外，我们通常都尽量避免这样做。

类似地，有时可以通过明智地使用 `<|` 运算符（或它的近亲 `|>`）来避免使用括号。注意：虽然 `$` 是 `<|` 的同义词，但在 mathlib 中不允许使用它，而应使用 `<|`，以保持一致性，也因为它与 `|>` 的对称性。这些运算符的作用是为 `<|` 右侧的所有内容加括号（注意 `(` 的弯曲方向与 `<` 相同），或为 `|>` 左侧的所有内容加括号（且 `)` 的弯曲方向与 `>` 相同）。

`|>` 用法的一个常见例子出现在点记号中，当 `.` 之前的项是一个被应用于某些参数的函数时。例如，`((foo a).bar b).baz` 可以重写为 `foo a |>.bar b |>.baz`

`<|` 用法的一个常见例子是，当用户提供的项是一个被应用于多个参数的函数，而其最后一个参数是策略模式下的证明，尤其是跨越多行的证明时。在这种情况下，使用 `<| by ...` 代替 `(by ...)` 是很自然的，如下所示：

```
import Mathlib.Tactic

example {x y : ℝ} (hxy : x ≤ y) (h : ∀ ε > 0, y - ε ≤ x) : x = y :=
  le_antisymm hxy <| le_of_forall_pos_le_add <| by
    intro ε hε
    have := h ε hε
    linarith
```

当使用策略 `rw` 或 `simp` 时，左箭头 `←` 之后应有一个空格。例如 `rw [← add_comm a b]` 或 `simp [← and_or_left]`。（在策略名称及其参数之间也应有一个空格，例如 `rw [h]`。）这条规则同样适用于 `do` 记号：`do return (← f) + (← g)`

声明内部的空行

不鼓励在声明内部使用空行，并且有一个 linter 会强制要求它们不出现。这有助于在整个 mathlib 中维持统一的代码风格。

不过，我们鼓励你为代码添加注释：即便是一句短短的话，也比一个置于证明中间的空行所传达的多得多！

规范形式

某些陈述是等价的。例如，要求某个类型的子集 `s` 非空有若干等价的方式。再举一个例子，给定 `a : α`，`Option α` 中对应的元素既可以等价地写成 `Some a`，也可以写成 `(a : Option α)`。一般而言，我们尽量确定一种标准形式，称为规范形式，并在定理的陈述和结论中都使用它。在上述例子中，这将分别是 `s.Nonempty`（它让我们能使用点记号）和 `(a : Option α)`。通常会注册一些 simp 引理，以将其他等价形式转换为规范形式。

这条规则有一种特殊情况。在带有最小元素的类型中，要求 `hlt : ⊥ < x` 与 `hne : x ≠ ⊥` 是等价的，由于两者各有利弊，并不清楚哪一个作为规范形式更好。在带有最大元素的类型中，`hlt : x < ⊤` 与 `hne : x ≠ ⊤` 也出现类似的情形。由于从 `hlt` 转换到 `hne` 非常容易（根据所需的方向使用 `hlt.ne` 或 `hlt.ne'`），而反向转换则更为冗长，因此我们在定理的假设中使用 `hne`（因为这是更容易检验的假设），而在定理的结论中使用 `hlt`（因为这是更强有力可用的结果）。这条规则的一个常见用法是在自然数上，那里 `⊥ = 0`。

注释

使用模块文档定界符 `/-! -/` 来提供节标题和分隔符，因为它们会被纳入自动生成的文档，并使用 `/- -/` 来书写更具技术性的注释（例如 TODO 和实现说明），或证明中的注释。使用 `--` 来书写简短的或行

内的注释。

声明的文档字符串以 `/-- -/` 定界。当一个声明的文档字符串跨越多行时，不要缩进后续各行。

更多建议和示例请参阅我们的文档要求。

错误或追踪消息中的表达式

在所有打印出的消息中（例如在 linter、自定义繁释器或其他元程序中），名称和内插数据应当要么 - 内联并以反引号包围（例如 `m!"{foo}must have type{bar}"`），要么 - 单独占一行并缩进（例如通过 `indentD`）

第二种风格产生如下的输出

```
Could not find model with corners for domain
  src
nor codomain
  tgt
of function
  f
```

mathlib 并非所有部分都已遵循这条规则；那属于 bug（我们欢迎修复此问题的 PR）。

弃用

删除、重命名或更改声明可能导致依赖这些定义的下游项目编译失败。任何正在被移除的公开暴露的定理和定义都应当通过保留旧声明并附加 `@[deprecated]` 属性来平稳过渡。这会向下游项目警示该更改，并给予他们机会，在这些声明被删除之前作出调整。被重命名的定义应当使用一个弃用的 `alias` 指向新名称。否则，当被弃用的定义没有直接替代品时，应当用一条消息来弃用该定义，如下所示:

```
theorem new_name : ... := ...
@[deprecated (since := "YYYY-MM-DD")] alias old_name := new_name

@[deprecated "This theorem is deprecated in favor of using ... with ..." (since := "YYYY-M
theorem example_thm ...
```

`@[deprecated]` 属性要求给出弃用日期，以及一个指向新声明的别名，或一条字符串，用以说明在不再提供新版本时如何从旧定义过渡开。

`deprecate to` 命令和 `scripts/add_deprecations.sh` 脚本可以帮助生成别名定义。

对于带有 `to_additive` 属性的声明，其弃用应确保该弃用也正确地被打上 `to_additive` 标签，如下所示:

```
@[to_additive] theorem Group_bar {G} [Group G] {a : G} : a = a := rfl

-- Two deprecations required to include the `deprecated` tag on both the additive
-- and multiplicative versions
@[deprecated (since := "YYYY-MM-DD")] alias AddGroup_foo := AddGroup_bar
@[to_additive existing, deprecated (since := "YYYY-MM-DD")] alias Group_foo := Group_bar
```

我们允许、但不鼓励贡献者同时将声明 X 重命名为 Y 以及将 W 重命名为 X。在这种情况下，X 不需要弃用属性，但 W 需要。

具名实例不需要弃用。被弃用的声明可以在 6 个月后删除。

避免 `nonrec`

`nonrec` 关键字告诉 Lean 假定声明体中那些貌似递归的调用实际上并非递归，而是去其他命名空间中查找同名的声明。当递归调用与某个命名空间中的另一个声明冲突时，请避免使用 `nonrec`，因为此时将该命名空间添加到那个声明上更具信息量（对 Lean 和用户都是如此）。如果它与根命名空间中的某个声明冲突，那么 `nonrec` 和 `_root_.[...]` 都是可以接受的。有时，避免使用 `nonrec` 需要放弃在该声明体内使用点记号。（目前 Mathlib 中有许多地方违反了这条规则。）

文档风格

所有拉取请求都必须满足以下文档标准。关于 自动生成的文档页面，请参阅 doc-gen 仓库。

你可以使用 Lean doc preview 页面 预览某个 GitHub 页面或拉取请求的 markdown 处理效果。

头部注释

每个 mathlib 文件都应以下列内容开头：* 一段包含版权信息的头部注释（参见我们风格指南中的建议）；* 导入列表（每行一个）；* 一段包含通用文档的模块文档字符串，使用 Markdown 和 LaTeX 编写。

（参见下面的示例。）

标题使用 atx 风格的标题（带井号，不在下方加横线）。开始与结束的分隔符 `/-!` 和 `-/` 应各自单独占一行。

文件必须有一个一级标题。其后跟随对文件内容的概述。

其他二级标题章节（按此顺序）为：* Main definitions（可选，可在概述中涵盖）* Main statements（可选，可在概述中涵盖）* Notation（仅当本文件未引入任何记号时才省略）* Implementation notes（描述重要的设计决策或接口特性，包括类型类的使用以及新定义的 `simp` 范式）* References（对教科书、论文或维基百科页面的引用）* Tags（一组关键词，在 mathlib 中进行文本搜索以查找某事物在何处涉及时可能有用）

引用应指向 mathlib 引文文件中的 bibtex 条目。请参阅下面的引用其他著作一节。

下面的代码块是文件头部的一个示例。

```
/-
Copyright (c) 2018 Robert Y. Lewis. All rights reserved.
Released under Apache 2.0 license as described in the file LICENSE.
Authors: Robert Y. Lewis
-/
module
```

```
public import Mathlib.Algebra.Order.AbsoluteValue.Basic
public import Mathlib.NumberTheory.Padics.PadicVal.Basic

/-!
# p-adic norm

This file defines the `p`-adic norm on `ℚ`.

The `p`-adic valuation on `ℚ` is the difference of the multiplicities of `p` in the numera
denominator of `q`. This function obeys the standard properties of a valuation, with the a
assumptions on `p`.

The valuation induces a norm on `ℚ`. This norm is a nonarchimedean absolute value.
It takes values in {0} ∪ {1/p^k | k ∈ ℤ}.

## Implementation notes

Much, but not all, of this file assumes that `p` is prime. This assumption is inferred aut
by taking `[Fact p.Prime]` as a type class argument.

## References

* [F. Q. Gouvêa, *p-adic numbers*][gouvea1997]
* [R. Y. Lewis, *A formal proof of Hensel's lemma over the p-adic integers*][lewis2019]
* <https://en.wikipedia.org/wiki/P-adic_number>

## Tags

p-adic, p adic, padic, norm, valuation
-/
```

文档字符串

每个定义和重要定理都必须有文档字符串。（也鼓励在引理上添加文档字符串，尤其是当该引理具有某些数学内容或可能在其他文件中有用时。）这些文档字符串使用 `/--` 引入，并以 `-/` 结束，置于定义之上，标记与文本之间可用换行或单个空格分隔。文档字符串的后续行不应缩进。它们同样可以包含 Markdown 和 LaTeX：参见下一节。如果文档字符串是一个完整的句子，则应以句号结尾。具名定理，

例如 **mean value theorem**，应使用粗体（即前后各加两个星号）。

文档字符串应传达定义的数学含义。它们可以与实际实现略有出入。下面是一个文档字符串示例：

```
/-- If `q ≠ 0`, the `p`-adic norm of a rational `q` is `p ^ (-padicValRat p q)`.
If `q = 0`, the `p`-adic norm of `q` is `0`. -/
def padicNorm (p : ℕ) (q : ℚ) : ℚ :=
  if q = 0 then 0 else (p : ℚ) ^ (-padicValRat p q)
```

一个略有出入但仍描述了数学内容的示例如下：

```
/-- `padicValRat` defines the valuation of a rational `q` to be the valuation of `q.num` m
valuation of `q.den`. If `q = 0` or `p = 1`, then `padicValRat p q` defaults to `0`. -/
def padicValRat (p : ℕ) (q : ℚ) : ℤ :=
  padicValInt p q.num - padicValNat p q.den
```

策略文档

策略应具有符合 Lean 文档风格指南的文档字符串。要点是：完整且自包含，但要简明。文档字符串应以一个完整的句子开头，该句子以该策略为主语。（例如：“`rewrite [e]` uses the expression `e` as a rewrite rule on the main goal.”）该策略的所有不同选项和形式都应出现在项目符号列表中。“Examples:”一节（如果有）应是一系列代码块。

代码检查（Linting）

`docBlame` 检查器会列出所有没有文档字符串的定义。`docBlameThm` 检查器会列出没有文档字符串的定理和引理。`tacticDocs` 检查器会列出所有没有文档字符串的策略。

要仅运行 `docBlame` 检查器，请在你的 lean 文件末尾添加以下内容：

```
#lint only docBlame
```

要仅运行 `docBlame` 和 `docBlameThm` 检查器，请在你的 lean 文件末尾添加以下内容：

```
#lint only docBlame docBlameThm
```

要运行所有默认检查器（包括 `docBlame` 和 `tacticDocs`），请在你的 lean 文件末尾添加以下内容：

```
#lint
```

要运行所有默认检查器（包括 `docBlame`）并运行 `docBlameThm`，请在你的 lean 文件末尾添加以下内容：

```
#lint docBlameThm
```

LaTeX 与 Markdown

我们通常将对 Lean 声明或变量的引用放在反引号之间。写出全限定名称（例如 `finset.card_pos` 而不只是 `card_pos`）会在我们的在线文档中将该名称变为一个链接。

原始 URL 应用尖括号 `<...>` 括起来，以确保它们在线时可点击。（某些 URL，尤其是那些带括号或其他特殊符号的，可能无法被 markdown 渲染器正确解析。）

而在谈及数学符号时，则可能更宜使用 LaTeX。可以通过三种方式在文档字符串中包含 LaTeX：- 使用单美元符号 `$ ... $` 以内联方式渲染数学公式，- 使用双美元符号 `$$ ... $$` 以”显示模式”渲染数学公式，或 - 使用环境 `\begin{*} ... \end{*}`（不带美元符号）。

这些对应于我们在线文档的 MathJax 设置。那里 Markdown 与 LaTeX 之间的交互方式类似于 https://math.stackexchange.com 和 https://mathoverflow.net 上的情形，因此你可以把一个文档字符串粘贴到 那里的一个编辑沙盒中以预览最终结果。另请参阅 math.stackexchange 的 MathJax 教程。

分节注释

通常会将一个文件组织成若干节，每节包含相关的声明。通过在开头用模块文档 `/-! ... -/` 描述这些节，这些节便能在文档中显现。

虽然这些分节注释往往会对应于 `section` 或 `namespace` 命令，但这并非必需。你可以在一个 section 或 namespace 内部使用分节注释，也可以让多个 section 或 namespace 跟随在一个分节注释之后。

分节注释仅用于显示和可读性。它们没有语义含义。

在分节注释内部的标题应使用三级标题 `###`。

如果注释长度超过一行，分隔符 `/-!` 和 `-/` 应各自单独占一行。

实际示例参见 Lean/Expr/Basic.lean。

```
namespace BinderInfo

/-! ### Declarations about `BinderInfo` -/

/-- The brackets corresponding to a given `BinderInfo`. -/
def brackets : BinderInfo → String × String
  | BinderInfo.implicit => ("{", "}")
  | BinderInfo.strictImplicit => ("`", "`")
  | BinderInfo.instImplicit => ("[", "]")
  | _ => ("(", ")")
```

```
end BinderInfo

namespace Name

/-! ### Declarations about `name` -/

/-- Find the largest prefix `n` of a `Name` such that `f n != none`, then replace this pre
with the value of `f n`. -/
def mapPrefix (f : Name → Option Name) (n : Name) : Name := Id.run do
  if let some n' := f n then return n'
  match n with
  | anonymous => anonymous
  | str n' s => mkStr (mapPrefix f n') s
  | num n' i => mkNum (mapPrefix f n') i
```

理论文档

除了存在于 Lean 文件中的文档之外，我们还有理论文档，其中给出跨越多个 Lean 文件的综述，以及在形式化需要略为奇特视角的情形下提供更多数学解释，例如可参见拓扑文档。

引用其他著作

要在文档字符串中引用论文和书籍，首先应将引用条目添加到 BibTeX 文件：`docs/references.bib`。要使用 `bibtool` 规范化该文件，你可以运行：

```
bibtool --preserve.key.case=on --preserve.keys=on --print.use.tab=off --pass.comments=on -
```

要确保你的引用在在线文档中成为链接，你可以使用以下两种风格中的任意一种：

第一，你可以将 `docs/references.bib` 中使用的引用键放在方括号中：

```
The proof can be found in [Boole1854].
```

在在线文档中，这会变成类似于：

> The proof can be found in [Boo54]

（该键会变为一个 `alpha` 风格的标签，并成为指向文档参考文献页面的链接。）

或者，你也可以通过在引用键前面的方括号中放入文本，来为引用使用自定义文本：

```
See [Grundlagen der Geometrie][hilbert1999] for an alternative axiomatization.
```

See Grundlagen der Geometrie for an alternative axiomatization.

注意，你目前不能在链接文本中使用右方括号 `]` 符号。因此下面的写法不会生成可用的链接：

```
We follow [Euclid's *Elements* [Prop. 1]][heath1956a].
```

We follow [Euclid’s Elements [Prop. 1]][heath1956a].

语言

文档应使用英语撰写。任何常见的拼写（例如英式、美式或澳式英语）都可接受。不应仅为了改变所用的拼写而提交拉取请求，但作为某个大幅增强文档的 PR 的一部分而改变拼写则是可接受的。这与声明名称的规则形成对比，后者应使用美式英语拼写。

示例

以下文件作为良好文档风格的范例加以维护：

- Mathlib.NumberTheory.Padics.PadicNorm
- Mathlib.Topology.Basic
- Analysis.Calculus.ContDiff.Basic

拉取请求标题与描述规范

我们采用以下规范来撰写拉取请求的标题与描述。

格式

注意:“Title:”与“Description:”并不会实际出现

```
Title:
<type>(<optional-scope>): <subject>

Description:
<body>
<NEWLINE>
<footer>
<NEWLINE>
<dependencies>
```

`<type>` 为:

- feat (feature, 新功能)
- fix (bug fix, 缺陷修复)
- doc (documentation, 文档)
- style (formatting, missing semicolons, …, 格式调整、缺失的分号等)
- refactor (重构)
- test (when adding missing tests, 补充缺失的测试时)
- chore (maintain, 维护)
- perf (performance improvement, optimization, …, 性能改进、优化等)
- ci (for changes to github workflows, other automation, 对 github workflows 及其他自动化的修改)

`<optional-scope>` 是包含所修改模块的某个模块名或目录名。它并非必须包含, 但当 `<subject>` 不足以说明时可能会有用。`Mathlib` 目录前缀总是省略。例如, 它可以是

- Data/Nat/Basic
- Algebra/Group/Defs
- Topology/Constructions

`<subject>` 有以下约束:

- 使用祈使语气、现在时态: 用“change”而非“changed”或“changes”
- 首字母不大写
- 结尾不加句点 (.)

`<body>` 有以下约束:

- 与 `<subject>` 一样, 使用祈使语气、现在时态
- 包含本次修改的动机, 并与之前的行为作对比

`<footer>` 是可选的, 可能包含两项内容:

- 破坏性变更 (Breaking changes): 所有破坏性变更都必须在 footer 中提及, 并附上变更的描述、理由以及迁移说明
- 引用 issue(Referencing issues): 已关闭的缺陷应在 footer 中单独成行列出, 并以“Closes”关键字为前缀, 例如:Closes #123, #456

`<dependencies>` 如果本 PR 依赖于其他 PR, 则应以复选框格式列出, 即 `- [ ] depends on: #XXXX`

示例

一个无需 `<scope>` 的示例可能是:

```
feat: have library search use the whole range for replacement

previously `apply? using h` would replace to `refine blah using h` rather than `refine blah

This also changes the diagnostic message to be on the whole syntax `apply? using h` rather
```

而一个包含 `<scope>` 确实能增加价值的示例:

```
doc(CategoryTheory/EssentialImage): typo and punctuation

Fix a typo, add two periods.
```

一个带有依赖 PR 的示例:

```
feat: the norm on `Unitization` is a C*-norm
```

```
This shows that C⋆-algebras are always `RegularNormedAlgebra`s, so that their `Unitization
```

- [ ] depends on: #5330
- [ ] depends on: #5741
- [ ] depends on: #5742
- [ ] depends on: #5743

拉取请求审查指南

本指南详细介绍了如何为 mathlib 进行 PR 审查。你也许会疑惑本指南是否适用于你，答案是“适用！”

虽然只有 mathlib 维护者（maintainer）才有权限合并拉取请求，但每个人都欢迎、甚至被鼓励去审查拉取请求。（注意：实际上还有另一类人，即 mathlib 审查者（reviewer），他们已经证明自己能够提供有价值的 PR 审查；来自这些用户、带有 `maintainer merge` 的赞成审查会被维护者更快地合并）。

一段有帮助的审查历史是入选 mathlib 审查者或 mathlib 维护者团队的关键标准。

本指南首先给出进行审查的总体准则，对审查者应当考虑哪些方面给出一个高层次的概览，然后聚焦于若干实际的示例。

虽然本指南篇幅相当长，但并不要求你在开始审查之前通晓所有内容。部分审查本身就是有帮助的，你可以随着对 Lean 和 mathlib 理解的加深，零散地逐步了解各种考量因素。

审查准则

尊重与鼓励

与 mathlib 社区中的所有互动一样，请务必遵守 行为准则。简而言之，要保持尊重。然而，在审查时也请务必保持鼓励的态度。大多数贡献者只提交过寥寥几个拉取请求，这甚至可能是他们的第一个！因此避免诸如“这个结果没用，我们已经有它的一个版本了”这样的评论是很重要的。相反，你可以更温和地说，比如，“感谢你证明了这个结果，但我想我们已经有一个具有同样效果的引理了，它是 `my_generic_lemma`。请尝试改用它。”尽量发现他们工作中的优点，即便在指出可改进之处时也是如此。

谦逊

我们当中没有人，包括 mathlib 维护者，是完美的，或对最佳实践拥有垄断权。因此，审查者应当始终为他人提出更好的方案留出余地。此外，认识到 你也可能是错的并接受这种可能性，是很重要的。当然，完美是良好的敌人，所以没有必要为了更好的方案或新想法而无限期地等待。

审查时应考虑什么

审查本质上是审视代码并向自己提问。那些基础性的问题，大致按从易到难的顺序排列，依次是：风格、文档、位置、改进以及库的整合。

注意，新审查者完全有能力对前两个或前三个问题发表评论，而回答库整合方面的问题通常需要数月之久才能培养出对 mathlib 的熟悉度，并对审查过程有所贡献。

以下是你作为审查者可以向自己提出的一些具体问题。这只是一个提纲；在后续章节中，我们将通过示例更详细地探讨每一个问题。

- 它是否遵循风格规范?
  - 代码格式
  - 命名约定
  - PR 标题和描述是否提供了恰当的信息?
- 是否有有用的文档?
  - 这些定义是否有足够翔实的文档字符串（docstring)?
  - 是否有指向相关声明的交叉引用?
  - 复杂的证明是否在其中穿插了注释来给出一个梗概?
  - 重要的定理是否有文档字符串?
  - 当代码只应以特定方式使用时，是否对用户有警示?
  - 它是否在形式化文献中的某些内容?
- 位置，位置，位置
  - 这些声明是否位于恰当的文件中? 在这里 `#find_home` 会很有用。
  - 这些结果是否已经存在了? 可能以更一般的形式、以不同的名字存在? `apply?` 或 `exact?` 策略有时能帮助回答这个问题。
  - 是否引入了新的 `import`，如果是，它们是否为本文件引入了过多的内容?
  - 是否应当将某些结果放到一个新文件中，以尽量减少导入需求?
  - 是否应当将一个文件拆分成多个部分，因为它变得过长（例如，超过 1000 行），或涉及过多不同的主题?
- 是否有明显可以做出的改进?
  - 能否将某些部分拆分为辅助引理或定义（尤其是对于较长的证明)?
  - 能否使用不同的／更好的策略来提升可读性(例如,使用 `gcongr` 而非 `mul_le_mul_of_nonneg_left`)? 注意：只要不牺牲可读性，代码精简（code golfing）是可以的，不过精简平凡的结果通常也无妨。
  - 不同的证明结构是否能极大地简化论证?
  - 所引入的定义是否是形式化该概念的最佳方式（非常困难!)?
- 它是否推进或改进了库?
  - 它是否提供了合理的 API?
  - 它是否足够一般，以支持已知的未来需求?
  - 它是否契合 mathlib 的设计和集体愿景?

- 更具体的考量
  - 声明是否使用 `α β : Type*` 而非 `α β : Type _` 来指代任意的宇宙层级？（注意：这是一个性能问题，因为使用 `Type _` 会引入需要求解的合一问题。）
  - 引理在应当标注 `@[simp]`、`@[ext]` 等属性的地方是否做了标注？或者在不应当标注的地方是否避免了标注？
  - 新定义是否附带了关于它们的引理（也许仅仅是那些由 `@[simps]` 生成的引理）？
  - 任何新声明的实例是否会造成菱形（diamond）？是非定义相等（non-defeq）还是非命题相等（non-propeq）的？

示例

本指南的其余部分专门探讨审查过程中既有的虚构示例，也有真实案例。我们试图为上述每一个问题提供示例。真实案例中涉及的相关方已被征询过将其纳入本风格指南的许可。

并非每个问题都有对应的示例，但我们试图至少为每种情形提供一段关于应当考虑什么的讨论。

风格

代码格式

```
-- do NOT write code in this style!
theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α)
: a*b*c*d=a*(b*(c*d)) :=
by
rw [mul_assoc,
    mul_assoc]
```

上面的代码违反了若干格式准则：二元运算周围没有空格，行首以 `:` 开头而不是在上一行结尾处结束，`by` 应当移到上一行而不是单独占一行（无论如何，CI 中的风格 linter 应当能捕获这一点），而且 `rw` 策略被不必要地拆分到了多行。

在这种情形下，该 PR 的作者由于违反了如此多的风格准则，很可能是一位新贡献者，并且不熟悉、或不记得风格指南。一条恰当的审查评论大致可以是这样的：

```
In case you're unaware, please familiarize yourself with the mathlib
[style guide](https://leanprover-community.github.io/contribute/style.html).
You need spaces around `*`, `:` at the end of the line and the `rw` to
be on the same line.
```suggestion
theorem mul_assoc_assoc {α : Type*} [Semigroup α] (a b c d : α) :
```
```

```
    a * b * c * d = a * (b * (c * d)) := by
  rw [mul_assoc, mul_assoc]
```

命名约定

```
theorem inv_is_unit_times_self_eq_1 {M : Type*} [Monoid M] {a : M} (h : IsUnit a) :
    ↑(IsUnit.unit h)⁻¹ * a = 1 := sorry
```

上面的引理直接取自库，你能猜到它实际的名字吗？它是 `IsUnit.inv_val_mul`。一条建议新名字的恰当审查大致可以是这样的：

```
In order to accord with the
[naming conventions](https://leanprover-community.github.io/contribute/naming.html)
for mathlib, I suggest renaming this to: `IsUnit.inv_val_mul`. Note that:

- we use `mul` instead of `times`, and `one` instead of `1`
- the lemma is sufficiently clear without the reference to `1`
- If we are referencing an `IsUnit` hypothesis, we would use `isUnit`, not `is_unit`
- However, since we have an `IsUnit` hypothesis, putting it in the `IsUnit.`
  namespace allows for use with dot notation.
- we should reference the coercion that appears here, which is `Subtype.val`, hence
  the `val` in the suggested name.
```

这为 PR 作者提供了一个指向命名约定的链接，以防他们尚未看过它，同时也指出了具体的问题，这样他们就不必再通读整份指南。如果他们只在命名约定上犯了一个错误，这种做法也很可能是有帮助的。

注意：并非所有声明都恰好只有一个合适的名字，可能会有几个，每个都有各自的优点和缺点。

PR 标题和描述是否提供了恰当的信息？

考虑以下 PR 标题和描述：

```
Title: feat(Analysis/SpecificLimits)
Description: Where should we put these lemmas?
```

这有两个问题：标题没有提供任何关于改动的信息，而描述包含的是一个讨论性问题，而不是关于改动的信息。一条合理的审查评论大致可以是这样的：

```
Please update the PR title and description to be more informative about what
you have added or changed as these will be permanently included in the git
history when this is merged. Questions or topics for discussion are allowed
```

```
in the PR description, but should be placed after the `---`, as then they
will be treated as comments and not included in the git history.
```

当然，你可以提供建议，甚至自己更新 PR 标题和描述，但你或许会想要指出这一点，尤其是当 PR 作者是一位相对较新的贡献者时。

文档

这些定义是否有足够翔实的文档字符串?

`docBlame` linter 应当确保用户为他们所有的定义添加文档字符串。然而，文档字符串存在并不必然意味着它是有用且准确的。审查者应当尽力确保所提供的文档字符串以易于理解的方式准确描述了该 `def`。

重要的定理是否有文档字符串?

下面的示例参考了 #5580 中的审查。在那个 PR 中，添加了以下定理:

```
protected theorem _root_.WithSeminorms.equicontinuous_TFAE {κ : Type*}
    {q : SeminormFamily 𝕜₂ F ι'} [UniformSpace E] [UniformAddGroup E] [u : UniformSpace F]
    [hu : UniformAddGroup F] (hq : WithSeminorms q) [ContinuousSMul 𝕜 E]
    (f : κ → E →ₛₗ[σ₁₂] F) : TFAE
    [ EquicontinuousAt ((↑) ∘ f) 0,
      Equicontinuous ((↑) ∘ f),
      UniformEquicontinuous ((↑) ∘ f),
      ∀ i, ∃ p : Seminorm 𝕜 E, Continuous p ∧ ∀ k, (q i).comp (f k) ≤ p,
      ∀ i, BddAbove (range fun k ↦ (q i).comp (f k)) ∧ Continuous (⨆ k, (q i).comp (f k))
  sorry
```

这个定理是有用的，但也有点长，需要花时间（对人类而言）去解析。因此，它或许应当有一个文档字符串，这导致了 以下评论

```
Can you please add a docstring explaining the statement of the theorem, as well as a cross
`NormedSpace.equicontinuous_TFAE`?
```

随后 PR 作者用以下翔实的文档字符串更新了该定理，在这里我们能够看到巨大的附加价值:

```
/-- Let `E` and `F` be two topological vector spaces over a `NontriviallyNormedField`, and
that the topology of `F` is generated by some family of seminorms `q`. For a family `f` of
maps from `E` to `F`, the following are equivalent:
* `f` is equicontinuous at `0`.
```

```
* `f` is equicontinuous.
* `f` is uniformly equicontinuous.
* For each `q i`, the family of seminorms `k ↦ (q i) ∘ (f k)` is bounded by some continuo
  seminorm `p` on `E`.
* For each `q i`, the seminorm `⨆ k, (q i) ∘ (f k)` is well-defined and continuous.
In particular, if you can determine all continuous seminorms on `E`, that gives you a comp
characterization of equicontinuity for linear maps from `E` to `F`. For example `E` and `F
both normed spaces, you get `NormedSpace.equicontinuous_TFAE`. -/
```

是否有指向相关声明的交叉引用?

参见上一个示例，其中请求为一个相关声明添加交叉引用。

复杂的证明是否在其中穿插了注释来给出一个梗概?

在这个示例中，我们仅展示一个现成的例子，说明穿插的注释如何能够显著增加 Lean 中证明的价值。
这直接复制自 GromovHausdorff.instSecondCountableTopologyGHSpace 的源码。

每当一个复杂的证明没有像这样被注释时，请鼓励 PR 作者这样做。你可以向他们指出这段代码。

```
/-- The Gromov-Hausdorff space is second countable. -/
instance : SecondCountableTopology GHSpace := by
  refine secondCountable_of_countable_discretization fun δ δpos => ?_
  let ε := 2 / 5 * δ
  have εpos : 0 < ε := mul_pos (by simp) δpos
  have (p : GHSpace) : ∃ s : Set p.Rep, s.Finite ∧ univ ⊆ ⋃ x ∈ s, ball x ε := by
    simpa only [subset_univ, true_and] using
      finite_cover_balls_of_compact (X := p.Rep) isCompact_univ εpos
  -- for each `p`, `s p` is a finite `ε`-dense subset of `p` (or rather the metric space
  -- `p.Rep` representing `p`)
  choose s hs using this
  -- cardinality of the nice finite subset `s p` of `p.Rep`, called `N p`
  let N := fun p : GHSpace => Nat.card (s p)
  -- equiv from `s p`, a nice finite subset of `p.Rep`, to `Fin (N p)`, called `E p`
  let E := fun p : GHSpace => (hs p).1.equivFin
  -- A function `F` associating to `p : GHSpace` the data of all distances between points
  -- in the `ε`-dense set `s p`.
  let F : GHSpace → Σ n : ℕ, Fin n → Fin n → ℤ := fun p =>
    ⟨N p, fun a b => ⌊ε⁻¹ * dist ((E p).symm a) ((E p).symm b)⌋⟩
  refine ⟨Σ n, Fin n → Fin n → ℤ, inferInstance, F, fun p q hpq => ?_⟩
```

```
  /- As the target space of F is countable, it suffices to show that two points
  `p` and `q` with `F p = F q` are at distance `≤ δ`.
  For this, we construct a map `Φ` from `s p ⊆ p.Rep` (representing `p`)
  to `q.Rep` (representing `q`) which is almost an isometry on `s p`, and
 with image `s q`. For this, we compose the identification of `s p` with `Fin (N p)`
  and the inverse of the identification of `s q` with `Fin (N q)`. Together with
  the fact that `N p = N q`, this constructs `Ψ` between `s p` and `s q`, and then
  composing with the canonical inclusion we get `Φ`. -/
  have Npq : N p = N q := (Sigma.mk.inj_iff.1 hpq).1
  let Ψ : s p → s q := fun x => (E q).symm (Fin.cast Npq ((E p) x))
  let Φ : s p → q.Rep := fun x => Ψ x
  -- Use the almost isometry `Φ` to show that `p.Rep` and `q.Rep`
  -- are within controlled Gromov-Hausdorff distance.
  have main : ghDist p.Rep q.Rep ≤ ε + ε / 2 + ε := by
    refine ghDist_le_of_approx_subsets Φ ?_ ?_ ?_
    · show ∀ x : p.Rep, ∃ y ∈ s p, dist x y ≤ ε
      -- by construction, `s p` is `ε`-dense
      intro x
      have : x ∈ ⋃ y ∈ s p, ball y ε := (hs p).2 (mem_univ _)
      obtain ⟨y, ys, hy⟩ := mem_iUnion₂.1 this
      exact ⟨y, ys, hy.le⟩
    · show ∀ x : q.Rep, ∃ z : s p, dist x (Φ z) ≤ ε
      -- by construction, `s q` is `ε`-dense, and it is the range of `Φ`
      intro x
      have : x ∈ ⋃ y ∈ s q, ball y ε := (hs q).2 (mem_univ _)
      obtain ⟨y, ys, hy⟩ := mem_iUnion₂.1 this
      let i : ℕ := E q ⟨y, ys⟩
      let hi := ((E q) ⟨y, ys⟩).is_lt
    have ihi_eq : (⟨i, hi⟩ : Fin (N q)) = (E q) ⟨y, ys⟩ := by rw [Fin.ext_iff, Fin.val_mk]
      have hiq : i < N q := hi
      have hip : i < N p := by rwa [Npq.symm] at hiq
      let z := (E p).symm ⟨i, hip⟩
      use z
      have C1 : (E p) z = ⟨i, hip⟩ := (E p).apply_symm_apply ⟨i, hip⟩
      have C2 : Fin.cast Npq ⟨i, hip⟩ = ⟨i, hi⟩ := rfl
      have C3 : (E q).symm ⟨i, hi⟩ = ⟨y, ys⟩ := by
        rw [ihi_eq]; exact (E q).symm_apply_apply ⟨y, ys⟩
      have : Φ z = y := by simp only [Φ, Ψ]; rw [C1, C2, C3]
      rw [this]
      exact hy.le
```

```
    · show ∀ x y : s p, |dist x y - dist (Φ x) (Φ y)| ≤ ε
    /- the distance between `x` and `y` is encoded in `F p`, and the distance between
    `Φ x` and `Φ y` (two points of `s q`) is encoded in `F q`, all this up to `ε`.
      As `F p = F q`, the distances are almost equal. -/
      intro x y
      -- introduce `i`, that codes both `x` and `Φ x` in `Fin (N p) = Fin (N q)`
      let i : ℕ := E p x
      have hip : i < N p := ((E p) x).2
      have hiq : i < N q := by rwa [Npq] at hip
    have i' : i = (E q) (Ψ x) := by simp only [i, Ψ, Equiv.apply_symm_apply, Fin.coe_cast]
      -- introduce `j`, that codes both `y` and `Φ y` in `Fin (N p) = Fin (N q)`
      let j : ℕ := E p y
      have hjp : j < N p := ((E p) y).2
      have hjq : j < N q := by rwa [Npq] at hjp
      have j' : j = ((E q) (Ψ y)).1 := by
        simp only [j, Ψ, Equiv.apply_symm_apply, Fin.coe_cast]
      -- Express `dist x y` in terms of `F p`
      have : (F p).2 ((E p) x) ((E p) y) = ⌊ε⁻¹ * dist x y⌋ := by
        simp only [F, (E p).symm_apply_apply]
      have Ap : (F p).2 ⟨i, hip⟩ ⟨j, hjp⟩ = ⌊ε⁻¹ * dist x y⌋ := by rw [← this]
      -- Express `dist (Φ x) (Φ y)` in terms of `F q`
     have : (F q).2 ((E q) (Ψ x)) ((E q) (Ψ y)) = ⌊ε⁻¹ * dist (Ψ x) (Ψ y)⌋ := by
        simp only [F, (E q).symm_apply_apply]
      have Aq : (F q).2 ⟨i, hiq⟩ ⟨j, hjq⟩ = ⌊ε⁻¹ * dist (Ψ x) (Ψ y)⌋ := by
        simp [← this, *]
    -- use the equality between `F p` and `F q` to deduce that the distances have equal
      -- integer parts
      have : (F p).2 ⟨i, hip⟩ ⟨j, hjp⟩ = (F q).2 ⟨i, hiq⟩ ⟨j, hjq⟩ := by
        have hpq' : (F p).snd ≍ (F q).snd := (Sigma.mk.inj_iff.1 hpq).2
        rw [Fin.heq_fun₂_iff Npq Npq] at hpq'
        rw [← hpq']
      rw [Ap, Aq] at this
    -- deduce that the distances coincide up to `ε`, by a straightforward computation
      -- that should be automated
      have I :=
        calc
              ε⁻¹ * |dist x y - dist (Ψ x) (Ψ y)| = |ε⁻¹ * (dist x y -
dist (Ψ x) (Ψ y))| := by
            rw [abs_mul, abs_of_nonneg (inv_pos.2 εpos).le]
          _ = |ε⁻¹ * dist x y - ε⁻¹ * dist (Ψ x) (Ψ y)| := by congr; ring
```

```
          _ ≤ 1 := le_of_lt (abs_sub_lt_one_of_floor_eq_floor this)
      calc
        |dist x y - dist (Ψ x) (Ψ y)|
        _ = ε * (ε⁻¹ * |dist x y - dist (Ψ x) (Ψ y)|) := by grind
        _ ≤ ε * 1 := by gcongr
        _ = ε := mul_one _
  calc
    dist p q = ghDist p.Rep q.Rep := dist_ghDist p q
    _ ≤ ε + ε / 2 + ε := main
    _ = δ := by ring
```

当代码只应以特定方式使用时，是否对用户有警示?

某些声明只打算在特定文件内使用，也许是因为它们是辅助性的。另一些声明可以在任何地方使用，但应当谨慎使用，且只在首选方法不可用、或使用它的弊端无关紧要时才使用。

在特定文件中使用 前者的一个例子是：PiLp.iSup_edist_ne_top_aux，它包含以下文档字符串。

```
/-- An auxiliary lemma used twice in the proof of `PiLp.pseudoMetricAux` below. Not intend
```

这个引理通过它的名字（包含 aux）和它的文档字符串向读者表明它并不打算用于通用目的。其原因在于，紧接其前的那个声明，PiLp.pseudoEmetricAux，是一个仅被临时激活以构建恰当的伪扩展度量结构的实例。它的文档字符串也阐明了这一点：

```
/-- Endowing the space `PiLp p β` with the `L^p` pseudoemetric structure. This definition
satisfactory, as it does not register the fact that the topology and the uniform structure
with the product one. Therefore, we do not register it as an instance. Using this as a tem
pseudoemetric space instance, we will show that the uniform structure is equal (but not de
the product one, and then register an instance in which we replace the uniform structure b
product one using this pseudoemetric space and `PseudoEMetricSpace.replaceUniformity`. -/
```

使用条件 后者的一个例子是：completeLatticeOfSup，它包含以下文档字符串。

```
/-- Create a `CompleteLattice` from a `PartialOrder` and `SupSet`
that returns the least upper bound of a set. Usually this constructor provides
poor definitional equalities.  If other fields are known explicitly, they should be
provided; for example, if `inf` is known explicitly, construct the `CompleteLattice`
instance as
```

```
instance : CompleteLattice my_T :=
```

```
  { inf := better_inf,
    le_inf := ...,
    inf_le_right := ...,
    inf_le_left := ...
    -- don't care to fix sup, sInf, bot, top
    ..completeLatticeOfSup my_T _ }
```
-/

在这种情形下，构造了一个 `CompleteLattice`，但承载数据的 `sup`、`sInf`、`bot` 和 `top` 字段是用 `sSup` 来定义的，因此当需要访问其定义时，将不便于对它们进行证明。所以这条文档字符串充当了对用户的一个重要警示：此构造器应当谨慎使用，并且如果这些承载数据的字段是已知的，就应当明确地提供它们。

它是否在形式化文献中的某些内容?

mathlib 中有些对象仅仅是为了服务于形式化而存在的（例如 `AddMonoidWithOne`），但大多数被形式化的想法直接来自数学文献。在这种情况下，尤其是当贡献者在模仿一篇已发表的纸面证明时，应当鼓励他们在 `references.bib` 文件中添加一个条目，并在模块文档和／或相关定理的文档字符串中引用它。

此外，让 mathlib 中的数学与文献相关联，是对所添加内容的相关性的一项额外的合理性检查，并且但愿日后会有人关心它并使用它。将数学纳入 mathlib 会带来维护所添加内容的负担，如果某些内容永远不会被用到，就没有理由承担这一负担。

位置

这些声明是否位于恰当的文件中?

考虑来自 #5742 的以下示例，其中 PR 作者正在为 `Unitization` 配置一个范数结构。作者正在创建一个新文件 `Analysis.NormedSpace.Unitization`，并在某处声明了该实例：

```
instance Unitization.instNontrivial {𝕜 A} [Nontrivial 𝕜] [Nonempty A] :
    Nontrivial (Unitization 𝕜 A) :=
  nontrivial_prod_left
```

注意, 这个实例与范数毫无关系, 因此它很可能应当属于一个更早的文件。审查者可以使用 `#find_home Unitization.instNontrivial` 来确定放置此声明的自然位置是 `Algebra.Algebra.Unitization`。一位有帮助的审查者可以这样评论：

```
It seems like this instance doesn't have anything to do with the norm structure on the
`Unitization`. Perhaps you could place this instance in `Algebra.Algebra.Unitization` inste
```

这些结果是否已经存在了？可能以更一般的形式、以不同的名字存在？

mathlib 到现在已是一个相当庞大的库，任何人，尤其是新用户，都难以熟悉它所有不同的角落以及究竟哪些结果是可用的。这一点因我们追求一般性、摒弃代码重复这一事实而加剧；这导致了新用户经常提出的问题："mathlib 真的缺少向量空间和群同态吗？"，以及它的答案："不，它们分别是 `Module` 和 `MonoidHom`。"

因此，新贡献者（甚至是经验丰富的贡献者！）为一个已经存在的结果创建 PR 的情况并不罕见，有时是逐字逐句地相同，有时则是以更大的一般性出现。

`apply?` 和 `exact?` 策略有时能帮助回答这个问题。如果你怀疑某个结果已经存在，只需将它复制到一个带有 `import Mathlib` 的新文件中并尝试 `exact?`。

是否引入了新的 `import`？它们是否导入了过多的内容？

维护 mathlib 导入层级的组织结构是一项重要的任务，但如果不仔细审查，它很容易失控。一般来说，问题以下面这种方式出现。

一位贡献者想："我想添加 `my_theorem`，它完全是关于 `Z` 的，所以我会把它添加到 `X.Y.Z`。"在尝试把这个定理添加到那里时，贡献者意识到："哦，我没有访问 `helper_lemma` 的权限，我需要 `import A.B.C`。"在审查期间，审查者专注于其他事情，于是这个 PR 连同这一导入改动一起被合并了。这就是某个时刻 `Analysis.NormedSpace.Star.Basic` 导入了 `Analysis.NormedSpace.OperatorNorm` 的来龙去脉！这发生在 #16964 中，随后不得不在 #18194 中加以修复。

再举一个例子，在 #6239 中，贡献者向 `LinearAlgebra.Matrix.DotProduct` 添加了导入 `Data.IsROrC.Basic`。对于一位新贡献者来说，这很可能是一件难以察觉的事情，因为它有时需要对库的组织方式有相当程度的熟悉。

当然，审查者应当尽力捕获最为离谱的例子（例如，把 `Analysis` 文件导入到 `Algebra` 文件中通常相当可疑），但提出这个问题总归是有道理的。它往往可能意味着这些结果应当属于别处，意味着该文件应当沿一条自然边界拆分，或者意味着新结果应当放到一个新文件中。

是否应当将一个文件拆分成多个部分？

拆分一个文件本质上有三个理由：

1. 它实在太长，以致用起来不舒服。这里一个不错的经验法则是它超过了 1000 行。
2. 该文件被分成多个只是松散相关的部分。

3. 为了避免因引入与既有结果密切相关的新结果而导致的导入蔓延。

假设有一些结果，约 500 行的量，被添加到一个本就已经包含 700 行的既有文件中，并进一步假设这些新材料与该文件中某些既有材料密切相关。审查者应当留意一条自然边界，可以沿着它把该文件拆分成连贯的部分。

改进

拆分为辅助引理或定义（尤其是对于较长的证明）？

冗长的独立证明常常是一个迹象，表明手边附近就潜藏着一次值得进行的重构。新贡献者往往不知道库中既有的引理，或者可能不知道如何把他们的定理拆分成更易于管理的小块。在这些情形下，审查者有几种选择，包括：撸起袖子，以 GitHub 上的 `suggestion`（建议）形式亲自把结果重构成多个引理；寻找一个潜在的重构方向并提及它，比如“你或许可以考虑把第 xx 行的论证拆分成它自己的引理，这将简化证明”；或者干脆问，“这个证明似乎相当冗长且笨重，你是否考虑过它可以如何被拆分成更易于管理的部分?”；或者甚至，“如果你利用定理 X，这个证明似乎可能会更容易。”

用不同的策略来提升可读性

一个使用更好的策略或精简代码能够提升可读性的好例子可以在 #6140 的这条建议中找到。在这个例子中，证明里原始的子策略序列是：

```
have h₀ : log b = log (- -b) := by simp
rw [h₀, log_neg_eq_log]
have hb' : 0 < -b := by linarith
have h₁ : log (-b) < 0 := by rw [log_neg_iff hb']; linarith
refine tendsto_exp_atBot.comp ?_
rw [tendsto_const_mul_atBot_of_neg h₁]
show atTop ≤ atTop
rfl
```

而建议是将其精简为：

```
refine tendsto_exp_atBot.comp <| (tendsto_const_mul_atBot_of_neg ?_).mpr tendsto_id
rw [←log_neg_eq_log, log_neg_iff (by linarith)]
linarith
```

从精简后的版本中，我们可以轻易看出，这本质上只是把一些关于 `Filter.Tendsto` 的库引理复合起来，外加为其中一个定理提供一个假设，而该假设是用一些基本的重写和对 `linarith` 的调用来证明的。

一个使用更好的策略能够提升可读性的例子可以在 #4702 的整个 diff 中找到，它使用新的 gcongr 策略精简了整个库中的引理。我们将突出一个特别漂亮的例子以供参考。原始代码是:

```
  _ ≤ ε / 2 * ‖∑ i in range n, g i‖ + ε / 2 * ∑ i in range n, g i := by
    rw [← mul_sum]
    exact add_le_add hn (mul_le_mul_of_nonneg_left le_rfl (half_pos εpos).le)
```

它被用 gcongr 改进为:

```
  _ ≤ ε / 2 * ‖∑ i in range n, g i‖ + ε / 2 * ∑ i in range n, g i := by rw [← mul_sum]; gc
```

或者，来自同一个 PR 的这个例子，它使用 positivity 从:

```
  · have ha' := mul_le_mul_of_nonneg_left ha (inv_pos.2 hab).le
    rwa [MulZeroClass.mul_zero, ← div_eq_inv_mul] at ha'
  · have hb' := mul_le_mul_of_nonneg_left hb (inv_pos.2 hab).le
    rwa [MulZeroClass.mul_zero, ← div_eq_inv_mul] at hb'
```

变为:

```
  · positivity
  · positivity
```

不同的证明结构是否能极大地简化论证?

一个极好的例子出现在 #5602 的审查中。在这个例子中，PR 作者证明了:

```
variable {R S A : Type*} [CommSemiring R] [Semiring A] [Algebra R A] [SetLike S A]
  [hSA : NonUnitalSubsemiringClass S A] [hSRA : SMulMemClass S R A] (s : S)

-- `NonUnitalSubalgebra.unitization s : Unitization R s →ₐ[R] Algebra.adjoin R (s : Set A)
theorem NonUnitalSubalgebra.unitization_surjective :
    Function.Surjective (NonUnitalSubalgebra.unitization s) := by
  apply Algebra.adjoin_induction'
  · refine' fun x hx => ⟨(0, ⟨x, hx⟩), Subtype.ext _⟩
    simp only [NonUnitalSubalgebra.unitization_apply_coe, Subtype.coe_mk]
    change (algebraMap R { x // x ∈ Algebra.adjoin R (s : Set A) } 0 : A) + x = x
    rw [map_zero, Subsemiring.coe_zero, zero_add]
  · exact fun r => ⟨algebraMap R (Unitization R s) r, AlgHom.commutes _ r⟩
  · rintro _ _ ⟨x, rfl⟩ ⟨y, rfl⟩
    exact ⟨x + y, map_add _ _ _⟩
  · rintro _ _ ⟨x, rfl⟩ ⟨y, rfl⟩
    exact ⟨x * y, map_mul _ _ _⟩
```

审查者评论道：

```
I see why it's not immediate (you'd have to get back to the full codomain),
but is there really no good way of using `Algebra.adjoin_le` here?
```

这带来了大为改进的三行证明：

```
theorem NonUnitalSubalgebra.unitization_surjective :
    Function.Surjective (NonUnitalSubalgebra.unitization s) := by
  have : Algebra.adjoin R s ≤ ((Algebra.adjoin R (s : Set A)).val.comp (unitization s)).ra
    Algebra.adjoin_le fun a ha ↦ ⟨(⟨a, ha⟩ : s), by simp⟩
  fun x ↦ match this x.property with | ⟨y, hy⟩ => ⟨y, Subtype.ext hy⟩
```

在这个例子中，诉诸引理 `Algebra.adjoin_le` 相比于使用 `Algebra.adjoin_induction'`，是一次巨大的简化。

所引入的定义是否是形式化该概念的最佳方式（非常困难！）？

这在一般意义上极难描述，但也许能给出的最简单的经验法则是：定义越是避免使用依值类型，就越好。

举例来说，考虑 Vector 的定义。许多依值类型论的入门讲解将 `Vector` 定义为一个归纳类型，并把它作为依值类型的典型例子。虽然依值类型是不可避免的，但 mathlib 的定义转而选择了 `List` 的一个简单子类型，而对 `ℕ` 的依赖只出现在 `List.length` 的相等命题中。这具有这样的优点：我们可以通过强制转换到 `List` 轻松地跳出依值类型的世界，然后生活就轻松多了。事实上，`Vector` 上的大多数运算恰恰就是 `List` 上对应的运算结合一个关于长度的相等证明。

库的整合

这里的许多问题都有点含糊，并且通常需要对 mathlib 的结构有大量的熟悉，或者至少在形式化方面有显著的先前经验。如果你发现自己在审查时难以处理这些问题，不必气馁。

它是否提供了合理的 API？

- 属性是否被恰当地添加（例如 `@[simp]`、`@[ext]`、`@[gcongr]`、`@[aesop]` 等）？
- 是否提供了重写引理，以避免总是需要在定义层面上穿过相等关系？
- 一个新类型是否在常见用例中提供了便利的构造器？

它是否足够一般，以支持已知的未来需求?

这需要知道一些未来的需求可能是什么! 活跃于 Zulip 有助于让审查者意识到这些需求。然而, 人们可以为这个问题使用一个替代物，即“文献中是否存在这个结果的某个更一般的版本，且它调用了 mathlib 中预先存在的概念?”如果有，那么或许现有的 PR 应当被一般化。

它是否契合 mathlib 的设计和集体愿景?

同样地，这是一个含糊的问题，需要知道其设计和集体愿景是什么！然而，举一些实际的例子:

- 如果一位用户正在添加一种新的态射（不在范畴论库中），那么他们很可能应当定义一个捆绑式（bundled）的态射类型，并且大概还要使用 `FunLike` API 定义一个相关联的态射类。
- 类似地，如果一位贡献者正在添加一个新的子对象，那么他们大概应当使用捆绑式子对象，并利用 `SetLike` API。作为参考，请见 Mathematics in Lean 中的相关章节。
- 遵循任何既有的 library note（库说明）的建议。

Lean 的 Github 生态系统

关于向 Lean、Batteries 和 Mathlib 提交拉取请求时所涉及的分支、标签与 CI 工作流的说明文档。

- 你需要了解的事项 对所有人都适用
- 标签与分支 仅面向”专家”，即那些在 Lean 中制造或修复破坏性变更的人，或希望理解 Mathlib CI 内部工作原理的人。

你需要了解的事项

- 如果你向 `leanprover/lean4` 提交一个可能涉及破坏性变更的拉取请求，请将你的 PR 变基（rebase）到 `nightly-with-mathlib` 分支上。这将启用与 Mathlib 的联合 CI。
- 如果你向 `leanprover-community/mathlib4` 提交拉取请求，请从一个 fork 中提交。Mathlib 的 `.olean` 缓存现在可以与来自 fork 的 PR 一同工作。

标签与分支

`leanprover/lean4`

- 开发在 `master` 分支上进行。
- 稳定版本和候选发布版本带有标签，例如 `v4.2.0` 或 `v4.3.0-rc1`。
  - 若要在某个项目中使用这些版本之一，你的 `lean-toolchain` 文件应包含例如 `leanprover/lean4:v4.2.0`。
- 稳定版本在每个月末发布，并且与最后一个候选发布版本完全相同。
- 下一个版本的第一个候选发布版本会在稳定版本发布后立即发布。
- 每个版本都有一个 `releases/v4.X.0` 功能分支，它可能包含
  - 用于发布说明的额外提交
  - 从 `master` 精选（cherry pick）而来、通过候选发布版本发布的关键修复提交。

- 我们会定期从 master 制作每夜构建版本（nightly release), 它在 leanprover/lean4-nightly 仓库上带有诸如 nightly-2023-11-01 这样的标签。
  - 若要在项目中使用某个每夜构建版本,你的 lean-toolchain 文件应包含例如 leanprover/lean4:nightl 2023-11-01。（注意它不应是 leanprover/lean4-nightly:nightly-2023-11-01，因为 elan 在此处施加了一些神奇的智慧。）
  - 每夜构建版本可以通过手动触发发布工作流来修订。修订后的每夜构建版本在 leanprover/lean4-nightly 上带有形如 nightly-YYYY-MM-DD-revK 的标签（K 从 1 开始）。若要在项目中使用某个修订后的每夜构建版本，你的 lean-toolchain 文件应包含例如 leanprover/lean4:nightly-2023-11-01-rev1。修订后的每夜构建版本在基础每夜构建版本之后排序：base < rev1 < rev2 < 次日的每夜构建版本。Mathlib 的每夜构建测试基础设施会自动处理修订后的每夜构建版本。
- 在 leanprover/lean4 上有一个 nightly 分支，它跟随用于构建某个每夜构建版本的最新提交。
- 每个 PR 在成功构建后都会自动获得一个工具链。该 PR 随后会带有标签 toolchain-available。若要在项目中使用 PR #NNNN，你的 lean-toolchain 文件应包含 leanprover/lean4-pr-releases:pr-release-NNNN。
- 对于任何可能影响 Batteries 或 Mathlib 的 PR, 你应将你的 PR 建立在 nightly-with-mathlib 分支的 HEAD 之上。在这种情况下，会创建一个 lean-pr-testing-NNNN Mathlib 分支（下文详述），并且来自该分支的结果会通过 PR 讨论区中的评论进行报告。

leanprover-community/batteries（即‘Batteries’）

- 开发在 main 上进行。
- Batteries 在其 lean-toolchain 中使用最新的稳定版本或候选发布版本。
  - 由于我们在发布 v4.X.0 之后立即发布 v4.X+1.0-rc1, Batteries 仅在极短的时间内处于稳定版本上。
- main 上使用新工具链的第一个提交会被打上该工具链版本号的标签（例如 v4.2.0）。
- 有一个 stable 分支跟随 v4.X.Y 标签。
- Batteries 有一个 bump/v4.X.0 分支，用于 Lean 即将到来的稳定版本,
  - 它包含已获得维护者批准的、针对破坏性变更的适配
  - 并且将使用 leanprover-lean4:nightly-YYYY-MM-DD 工具链。
- Batteries 有一个 nightly-testing 分支，它
  - 使用最近的每夜构建版本（这会自动更新）
  - 自动将 main 的所有提交合并进来
  - 可以手动将 bump/v4.X.0 的任何变更合并进来
  - 可以包含任何其他提交，包括未经审查的提交，只要这些提交是使 nightly-testing 分支能够在最近的每夜构建版本上正常工作所必需的。
- nightly-testing 分支上的 CI 失败会由一个机器人报告到 zulip 的 nightly-testing-batteries 频道。
- nightly-testing 分支上的 CI 成功会导致创建一个与该提交相匹配的标签 nightly-

testing-YYYY-MM-DD，如果该标签尚不存在的话。
  - 因此，如果 nightly-testing-YYYY-MM-DD 存在，我们就知道在它上面：
    * lean-toolchain 是 leanprover/lean4:nightly-YYYY-MM-DD，并且
    * CI 成功。
- 当需要修改 Batteries 以适配 Lean 中的破坏性变更时，你需要创建一个分支，并随后从该分支开启一个 PR。（注意，下面的步骤在 Mathlib 处会自动发生，但对于 Batteries 则需要手动完成。）
  - 如果该变更是在 leanprover/lean4#NNNN 中做出的，那么 Batteries 的适配分支应当命名为 lean-pr-testing-NNNN。
  - Batteries 的适配分支应基于标签 nightly-testing-YYYY-MM-DD，其中 YYYY-MM-DD 是你的 Lean PR 所基于的每夜构建版本的日期。
  - 如果 nightly-testing-YYYY-MM-DD 标签尚不存在，你需要等待（并可能推进到后续的某个每夜构建版本）。如有需要，请联系 @kim-em 寻求帮助。
  - 理想情况下，你会将 lean-pr-testing-NNNN 分支推送到 Batteries 的主仓库；如有需要，我们可以提供写入权限。
  - 此分支上的 lean-toolchain 必须包含 leanprover/lean4-pr-releases:pr-release-NNNN。
  - 你可以从 lean-pr-testing-NNNN 分支开启一个 PR，无论是在完成所需的适配之前还是之后。
  - 开启 PR 时，记得将基础分支设置为 nightly-testing。
  - 请为该 PR 打上 v4.X.0 和‘depends on core changes’标签。（如果你没有写入权限，可请求他人代为完成。）
  - 一旦 Lean PR 被合并并在某个每夜构建版本中发布，Batteries 的适配 PR
    * 应将其 lean-toolchain 更新为 leanprover/lean4:nightly-YYYY-MM-DD
    * 其变更可以根据需要手动合并到 nightly-testing 中，以保持 nightly-testing 正常工作（不要更改基础分支并合并该 PR，我们仍然需要它）。
  - 一旦 Batteries 的适配 PR 获得批准，维护者会将其合并到 bump/v4.X.0（而非 nightly-testing-YYYY-MM-DD）。
- 始终允许将 bump/v4.X.0 合并到 nightly-testing 中，但反之则不行。（对 bump/v4.X.0 的变更已经过审查，但对 nightly-testing 的变更可能尚未审查。）
- 当需要将 Batteries 更新到 Lean 的新版本时，希望所需做的只是创建一个新的 PR，其内容为将 bump/v4.X.0 压缩合并（squash merge）到 main。

leanprover-community/mathlib4（即‘Mathlib’）

- 上面关于 Batteries 所说的一切都适用于 Mathlib，但有以下例外：
  - 开发在 master 上进行。
  - nightly-testing 状态更新会发布在 #nightly-testing-mathlib 的此话题中。
  - 向 Mathlib 提交的 PR 应从 fork 中发起。Mathlib 的 .olean 缓存现在可以与来自 fork 的 PR 一同工作。

- `lean-pr-testing-NNNN`、`nightly-testing`、`nightly-testing-*` 标签以及 `bump/v4*` 分支全都位于 `leanprover-community/mathlib4-nightly-testing`，它是 mathlib4 的一个 fork。如果你需要经常对这些分支进行写入访问，你可以在 Zulip 上的 `nightly-testing-mathlib` 频道 中请求被添加到 `nightly-testing` GitHub 团队。
- 注意，Mathlib 的 `nightly-testing` 分支可以根据需要使用 Batteries 的 `nightly-testing` 分支。
- 类似地，Mathlib 的 `bump/v4.X.0` 分支可以根据需要使用 Batteries 的 `bump/v4.X.0` 分支。
- 对于任何通过 CI 且基于某个每夜构建版本的 Lean PR，都会自动创建 `lean-pr-testing-NNNN` 分支。（与 Batteries 不同，那里必须手动创建。）
- `lean-pr-testing-NNNN` 分支上的 Mathlib 适配 PR 可能需要更改 Batteries 的依赖，以使用 Batteries 的 `lean-pr-testing-NNNN` 分支，如果 Batteries 也遭遇了破坏的话。

Mathlib 的 nightly 与 bump 分支

每个月都有一个新的 Lean 版本发布，而 Mathlib 力求尽快迁移到新的 Lean 版本。为了使这一过程尽可能顺畅，我们遵循以下流程:

- `nightly-testing` 分支位于 `leanprover-community/mathlib4-nightly-testing`，并使用 Lean 的每夜构建工具链版本。换言之，该分支上的 `lean-toolchain` 文件包含诸如 `leanprover/lean4:nightly-2024-09-26` 之类的内容。
  - 该分支不保证能够无错误地构建。
  - 对该分支的变更不会经过 Mathlib 维护者团队的审查。
  - 该分支不受保护：`nightly-testing` GitHub 团队的成员可以向其推送修复。
  - 该分支的目的是使 Mathlib 适配 Lean 每夜构建工具链版本中的变更。
  - 通常，向 Lean 核心提交的 PR `#NNNN` 会伴随着 Mathlib 在 `lean-pr-testing-NNNN` 分支中的适配。一旦 Lean 核心 PR 进入某个每夜构建工具链，Mathlib 的 `lean-pr-testing-NNNN` 分支就可以合并到 `nightly-testing` 中。通常需要解决 `lean-toolchain`、`lakefile.lean` 和/或 `lake-manifest.json` 中的合并冲突。
  - 如果 CI 在该分支上失败，它会在 Zulip 上的 “nightly-testing-mathlib > Mathlib status updates” 中发布一条消息，指明该失败。
  - 如果 CI 在该分支上通过，则会在同一话题中发布一条消息，指明成功，并给出创建 PR 以审查这些适配的说明。（见下文。）
- `leanprover-community/mathlib4-nightly-testing` 中的 `nightly-testing-green` 分支跟踪 `nightly-testing` 最后一个成功构建的提交。用于 `nightly-testing` 构建的工具会从该分支获取。
- `bump/v4.X.Y` 分支同样位于 `leanprover-community/mathlib4-nightly-testing`，并使用 Lean 的每夜构建工具链版本。
  - 该分支应始终能够无错误地构建。
  - 对该分支的变更会经过 Mathlib 维护者团队的审查。

  - 该分支受保护：只有 Mathlib 维护者和某些机器人才能向其推送。
  - 该分支的目的是准备一个 Mathlib `master` 分支的并行版本，使其能在 Lean 的即将到来的版本上构建。一旦该版本发布，`bump/v4.X.Y` 分支就会合并到 `master`。这次合并本质上是原子性的，因为其差异已经通过所有的每日适配 PR 得到审查。（见下文。）
- 当 `nightly-testing` 通过 CI 时，一个机器人会向 Zulip 发布消息，说明如何创建一个”适配 PR”，以将 `nightly-testing` 上的变更合并到 `bump/v4.X.Y`。
  - 这个 PR 可以按照 Zulip 消息中的指示，使用 `scripts/create-adaptation-pr.sh` 来准备。
  - 这个 PR 应由 Mathlib 维护者团队审查。
- 在 Lean 发布周期（即一个月）的过程中，`bump/v4.X.Y` 会累积针对未来 Lean 版本的适配。
  - 但 `master` 也会累积数千行的变更。
  - 因此应当定期将 `master` 合并到 `bump/v4.X.Y` 中。
  - 在撰写本文时，这一步骤已被整合进 `scripts/create-adaptation-pr.sh` 流程中。
  - 偶尔会发生合并冲突。这些冲突应当由 Mathlib 维护者团队审查，尽管目前并未做到这一点。

Lean 与 Mathlib 之间的联合 CI

- 对于每一个向 Lean 提交的 PR，我们都会尝试针对所产生的工具链运行 Mathlib CI。
- 为使其正常工作，你需要将你的 PR 变基到 `nightly-with-mathlib` 分支上。`nightly-with-mathlib` 分支指向最新的、能通过 Mathlib CI 的每夜 Lean 构建版本，并且 Mathlib（可能还有 Batteries）上存在对应的 `nightly-testing-YYYY-MM-DD` 标签。
- 机器人会从 `nightly-testing-YYYY-MM-DD` 标签在 `leanprover-community/mathlib4-nightly-testing` 上创建一个 `lean-pr-testing-NNNN` 分支，如果它已存在则向其推送一个空提交。
- 来自该 Mathlib 分支的后续 CI 结果会以评论的形式报告回 Lean PR。
- 如果你的 PR 不是从一个能成功构建 Mathlib 的每夜构建版本分叉而来，机器人会在你的 PR 上发表评论。每当你向 PR 推送时，它都会重新尝试。
- 如果 `nightly-with-mathlib` 对你的用途而言过于陈旧，你可以基于 `nightly`，那么一旦该每夜构建版本本身通过 Mathlib CI 并且你向 PR 推送，Mathlib CI 就会开始运行。
- 也有可能那个每夜构建版本永远不会通过 Mathlib CI。在这种情况下，你可能不得不等待 `nightly-with-mathlib` 更新，然后变基到它上面。

社区准则

我们致力于建设一个开放和包容的社区，欢迎每个人的参与。任何形式的冒犯性、歧视性或攻击性行为都不会被容忍。我们采用贡献者公约行为准则。本准则适用于 Lean Zulip 聊天 以及 leanprover-community GitHub 组织。

为澄清上述内容：可能导致被暂停或封禁于 Lean 社区 Zulip 的行为包括：骚扰、歧视性或不尊重的行为、持续的离题或扰乱性帖子、反复发布低质量帖子、利用社区完成课程作业或工作任务、使用马甲账号、私信刷屏、向用户发送未经请求的群发私信、大量使用 AI 而不加以注明、未经请求地发布 AI 生成的”垃圾”代码、就 AI 生成的代码做出毫无根据且不正确的声明，以及无视版主的指导。

这里不是免费的代码审查服务。那些仅相当于”看看我的项目”而没有具体问题或先前社区参与的帖子将被移除，发帖者将被暂停。

请勿在 GitHub 或 Zulip 上撰写评论时使用大语言模型（LLM）。如果英语不是你的母语，请不必担心：对大多数用户而言都是如此；只要你能够被理解，就没有问题。建立社区涉及人与人之间的联系；使用 LLM 替你撰写内容会抹去这种人性的成分。看起来由 LLM 生成的消息和评论将被删除，并可能导致暂停。

此列表并非详尽无遗，维护者在用户管理方面保留广泛的裁量权。

反复违规将导致临时暂停，若行为持续，暂停时长将随之增加。情节严重的个别事件将导致封禁。

行为准则团队是报告任何疑虑的首要联系点。你可以直接致信该团队的成员，或使用匿名表单来报告违反社区准则的事件。具体而言，在 Zulip 上，你还可以向整个版主团队（由 Mathlib 和 CSLib 维护者以及部分 Lean FRO 成员组成）报告有问题的消息，参见相关的 Zulip 文档。只有版主可以看到你报告了某条消息，以及你在报告中所写的内容。

我们鼓励在出现不受欢迎的行为时采取降温（de-escalation）的策略。如果你察觉到有人的行为违反了我们的行为准则，请不要以同样的方式回应；而应采取行动来纠正该行为，例如向版主报告。

版主团队与行为准则团队的职权范围

本节旨在区分行为准则团队与版主团队的目的与角色。此外，本节还将阐明这些角色的一些职责，因此是一份面向公众的文件。

Lean Zulip 拥有近 16,000 名订阅用户（截至 2026 年 6 月），而在 2022 年约为 6,000 名。如此庞大的用户群需要大量的管理工作，我们有一份行为准则，并附有若干额外准则加以扩充。管理工作分为两个团队，后者是前者的子集：版主团队和行为准则团队。

版主团队

版主团队处理 GitHub（在 `leanprover-community` 组织内）和 Zulip 上的评论和消息。在 Zulip 上，这包括创建新的私有或公共频道、整理现有频道并将消息移至其恰当的主题，或在某个话题离题或变得无成效时提醒用户。在这两个平台上，这都意味着当用户因他人的行为而感到不快时与他们进行交流，或隐藏、删除违反行为准则的评论。总体而言，版主团队致力于帮助用户与社区进行积极的互动，并尽可能友好地解决争端。

Zulip 上的用户可被版主暂停（临时停用）；彻底的封禁（永久停用）针对马甲账号（个人为隐藏其使用而开设的额外账号）实施。因其他违规行为而封禁则属于行为准则团队的职权范围。首次违规的暂停时长从 1 天起算，可在无事先警告的情况下实施，但会附带说明理由，并对后续违规逐次翻倍。当用户使用 Zulip 上的”报告消息”功能时，包含报告者姓名的报告会通过私有频道发送给版主，此时版主可采取前述任何行动。从规模上看，此类行动几乎每天都会发生。

由于管理行动的频繁性，以及为避免针对个别版主的报复，版主团队可使用若干工具来帮助执行其中的某些行动。当版主使用这些工具采取行动时，这些行动会对其余版主公开（因为有一个机器人在私有频道中发布消息）；因此，任何特定版主都会在一定程度上受到团队其余成员的监督。请注意，所有这些工具都由版主手动触发；过程中始终有人参与。管理工作主要聚焦于对行为准则和 Zulip 准则的明显违反，以及在可以假定各方均出于善意的个人之间的冲突。

行为准则团队

行为准则团队承担着相关但略有不同的职能。版主团队规模庞大，处理 Zulip 上绝大多数的管理工作，但它仅针对那些明显的违规，或经版主团队其余成员稍加讨论后大体明确的违规。相比之下，行为准则团队旨在处理具有以下特征的情形：

1. 标准管理无法解决的个人之间的冲突。
2. 持续的扰乱性行为，其程度未达到对行为准则的明确违反。
3. 任何通过匿名报告表单报告的行为。
4. 值得永久封禁而不仅仅是暂停的恶劣行为。

简而言之，行为准则团队处理的问题是那些比标准管理所应对的更严重、更持久、更微妙，或需要更高隐私保护的问题。行为准则团队在作为这些更复杂情形的首要联系点并在其内部展开讨论的同时，可在确保原始投诉人匿名的前提下，向更广泛的版主团队征求意见和指导。由于投诉具有更严重和／或更持久的性质，行为准则团队也可实施永久封禁，或超出默认翻倍周期的暂停。在行为准则团队发出书面警告之前，不会实施封禁或额外的暂停。与版主一样，行为准则团队致力于帮助用户与社区进行积极的互动，并尽可能友好地解决争端。

行为准则团队将每半年向公众分享一份有限的报告，详细说明收到的投诉数量、收到、处理和解决的日期，以及最终结果。采取行动和最终结果的详细理由将与相关方分享，但为保护隐私，不会纳入半年度报告。

使用 Lean 进行教学

如果你有兴趣使用 Lean 进行教学，可以在这里找到各种资源和建议。

- 课程列表及简介
- 教学资源列表：材料、工具等
- 在教学中使用 Lean 的技巧与提示

教学资源

我们收集了各类资源，它们可能对使用 Lean 开设课程有所帮助。

讲义与教材

本网站收集了各种学习资源。这里我们列出专门为大学课程设计的文本。

- The Mechanics of Proof，作者 Heather Macbeth，是一套讲义，讲述如何撰写细致、严谨的数学证明，并配有相应的 Lean 材料。
- How To Prove It With Lean，作者 Daniel Velleman，是《How To Prove It》一书的补充材料。
- The Hitchhiker’s Guide to Logical Verification，作者 Jasmin Blanchette，是一本教材，以 Lean 4 证明助手为载体，向读者介绍交互式定理证明。该教材附有 Lean 演示文件和习题文件。

游戏

- The Natural Number Game，作者 Kevin Buzzard、Jon Eugster 和 Mohammad Pedramfar，是一个广受欢迎的 Lean 入门游戏。
- NNG 背后的引擎 可用于为课程设计定制化的游戏。

自动评分系统

- 面向 Lean 4 的 Gradescope 自动评分系统
- 面向 Lean 4 的 GitHub Classrooms 自动评分系统

云端 Lean 配置

- 将 mathlib4 的 `.devcontainer` 目录 插入你的课程项目中，即可为你的项目启用 GitHub Codespaces。鼓励学生通过 GitHub 的教育福利注册（免费的）专业版账户，以获得更多的 Codespaces 使用时长。
- 同样地，插入 mathlib4 的 `.gitpod.yml` 和 `.docker/gitpod/Dockerfile` 即可启用 Gitpod。

使用 Lean 进行教学的技巧与建议

为了帮助有意使用 Lean 进行教学的人，我们在此收集了一些在社区中行之有效的策略和方法。根据你课程的具体情况，这些方法可能适用，也可能并不适用。

规划你的课程

如果你正在考虑将 Lean 引入某门课程，或者围绕 Lean 设计一门课程，那么你应该问自己一些重要的问题。你对这些问题的回答有助于找到他人已经讲授过的类似课程，也有助于你选择所使用的教学材料。

- 你是想教授 Lean，还是想使用 Lean 进行教学？换句话说，你这门课程的学习目标是什么？有些课程尝试以 Lean 作为一种”实现语言”来讲授数学或计算机科学主题，并不期望学生掌握扎实的 Lean 技能。另一些课程则明确讲授验证与形式化证明。
- 你的课程实际会用到多少 Lean？尤其是在尝试使用 Lean 进行教学的课程中，形式化的工作量可以非常之少，例如仅作为加分的可选练习，或仅由教师制作演示。
- 你会假定学生具备哪些技术先备知识？他们应当拥有 GitHub 账号并理解 `git` 吗？他们是否用函数式语言编过程，或用任何语言编过程？能否期望他们在自己的电脑上本地安装 Lean?
- 有些课程以介绍 Lean 的类型论作为开端，或许会将其与数学家所熟悉的集合论加以对比。另一些课程则假定学生会自行领会其中的类比——如果他们对集合论有所了解的话——而不着重于二者的差异。两种做法都合理，但最好尽早择其一，并在你的教学材料中始终如一地坚持下去。
- Lean 的语法以及种类繁多的选项，可能会让学生不知所措。为你的课程选择一种”方言”是很重要的，它可以随着课程的推进而扩展。限制你所引入的策略、避免使用库中的引理，以及只教策略模式或项模式（而非两者兼授），都是人们用来限制学生一次需要吸收的语言细节量的方法。

组织课程项目

人们常常使用一个”课程项目”来分发讲义、示例、作业等，同时确保学生都使用固定版本的 Lean 和 mathlib。这里有来自 Fordham 的一门课程 和 Brown 的一门课程 的示例。前者包含可编译为 HTML

的详细讲义；后者则依赖于一本外部教科书作为参考。

如果没有某种类似的结构——例如，如果学生收到的是裸露的.lean 文件——就很难确保他们都使用同一版本的 Lean。这也是提供一个或多个”库”文件的好办法，其中包含对你的课程有用的基本定义、策略等内容。

这些项目通常托管在 GitHub 上。在课程开始时，学生需要克隆该项目，或创建一个 Codespace 或 Gitpod 实例。他们会被告知定期拉取更新，例如在新作业发布时。如果不要求学生熟练使用 `git`，你可以提供 辅助脚本 来尝试自动完成这些管理工作。

有些教师建议学生在开始作业之前先复制作业文件，在副本上进行作答，以避免在作业内容发生变更时出现合并冲突。

另一些教师则使用 GitHub Classrooms 来发布作业。在这种设置下，每份作业都必须是其自身独立的 Lean 项目。

云端 Lean 配置

我们的资源页面提供了为课程项目配置 GitHub Codespaces 和 Gitpod 的指引。尤其是对于面向可能难以在本地安装 Lean 的学生的大型课程，使用云端资源来运行 Lean 可以极大地简化课程的起步阶段。

在这两种方法中，学生都能够在浏览器中使用 VSCode 来编辑 Lean 文件，而 Lean 服务器在远程运行。对于 Codespaces，一个便捷的 VSCode 插件 使得从本地 VSCode 安装环境进行工作也成为可能。显而易见的好处在于，所有学生都拥有统一的环境，而无需为安装而烦恼，并且没有学生会使用性能不足的机器。缺点则包括：学生的文件保存在云端，学生可能难以下载并提交它们；这些云服务限制了每位学生每月可使用的免费时长；而且进行课程作业需要联网。

有些教师已经成功地利用这些资源开设了大型课程，还有许多教师将其作为一种选项提供给学生。对于 GitHub Codespaces，重要的是提醒学生注册 GitHub 的学生福利，以利用额外的 Codespaces 时长。

为了在创建新的 codespace 时为学生节省时间，GitHub 提供了”预构建”（prebuild）选项。在你的课程仓库中，进入 Settings -> Codespaces。你很可能希望在每次推送时进行预构建并存储 1 个版本。存储这些镜像会产生少量费用：目前（2025 年 1 月），一门包含完整 mathlib 构建的课程，其镜像每月费用约为 0.40 美元。

重命名与重新定义策略

Lean/mathlib 为策略所取的名称，可能与你在课堂上呈现这些主题的方式不一致。在 Lean 4 中，可以很容易地在课程项目内为某些策略调用创建别名，或改变现有策略的行为。（在所链接的示例中，在任何导入了 `Tactics.lean` 的课程文件里，`linarith` 的行为都将被重新定义。）

用 done 结束证明

有时很难判断一个策略证明何时完成，因为错误信息出现在证明的开头而不是末尾。有些教师成功地教学生在编写证明时，先在末尾写上策略 done。

```
example (x : ℕ) : x = x := by
  -- fill in your proof here
  done
```

另一种做法是在 by 之后使用花括号：

```
example (x : ℕ) : x = x := by {
  -- fill in your proof here
}
```